

DistriSearch - Guía de Estudio Completa

Sistema de Búsqueda Distribuida

Generado: 2026-01-07 11:15

Tabla de Contenidos

- 0. [Introduccion](#)
- 1. [Arquitectura General](#)
- 2. [Vectorizacion Busqueda](#)
- 3. [Particionamiento](#)
- 4. [Rebalanceo](#)
- 5. [Replicacion](#)
- 6. [Tolerancia Fallos](#)
- 7. [Consenso Raft](#)
- 8. [Docker Swarm](#)
- 9. [DNS Descubrimiento](#)
- 10. [Load Balancer](#)
- 11. [Almacenamiento](#)
- 12. [Gossip Protocol](#)
- 13. [CAP Theorem](#)
- 14. [Despliegue](#)
- 15. [API Backend](#)
- 16. [Frontend](#)
- 17. [Testing](#)
- 18. [Control Center](#)

00 Introduccion

Introducción a DistriSearch

Sistema de Búsqueda Distribuida con Vectorización Semántica

1. ¿Qué es DistriSearch?

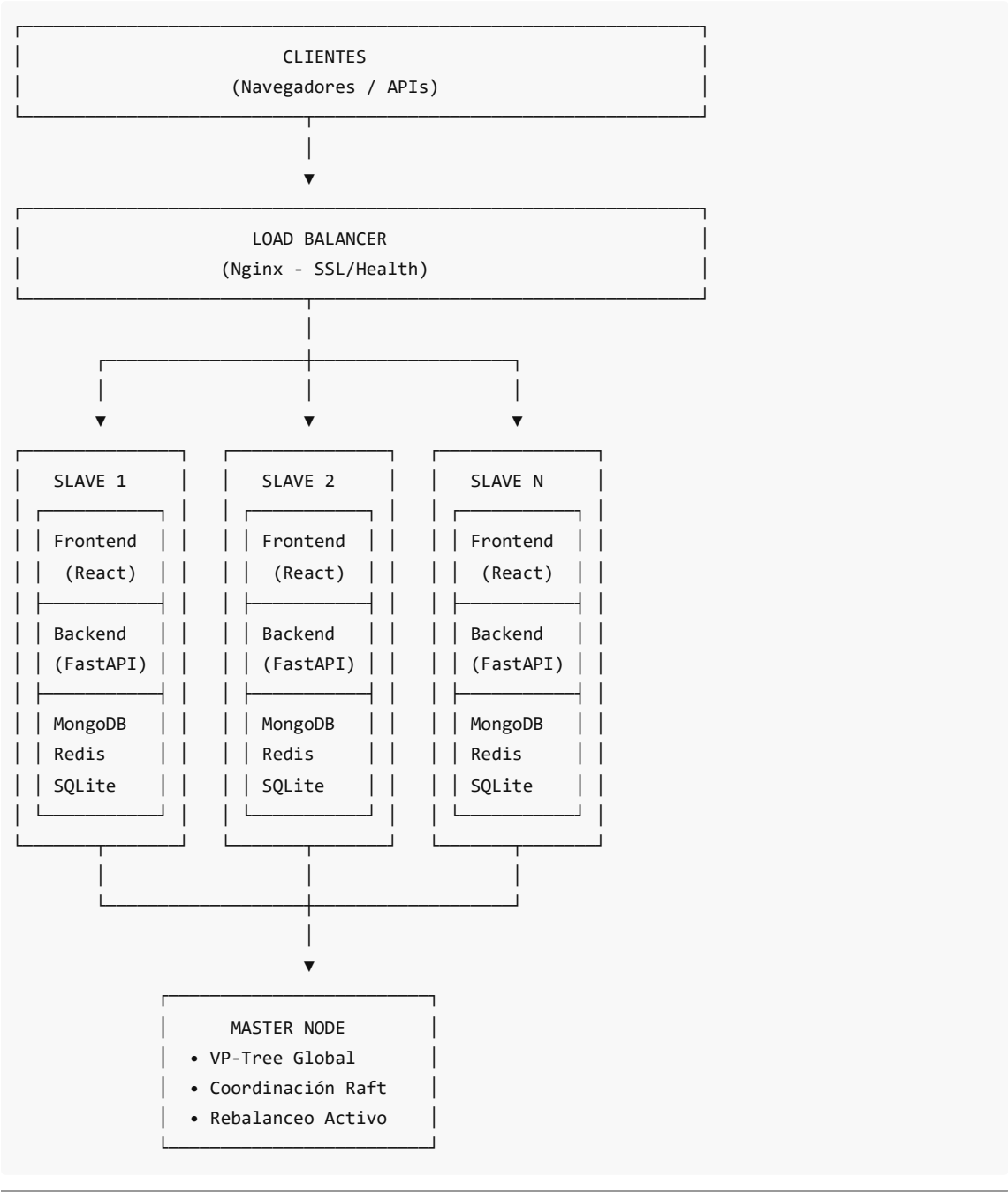
DistriSearch es un sistema de búsqueda distribuida diseñado para almacenar, indexar y buscar documentos de manera eficiente a través de múltiples nodos. A diferencia de los motores de búsqueda tradicionales, DistriSearch utiliza **vectorización semántica adaptativa** para encontrar documentos similares basándose en su significado, no solo en coincidencias exactas de palabras.

Características Principales

Característica	Descripción
Búsqueda Semántica	Encuentra documentos por similitud de significado usando TF-IDF + MinHash

Arquitectura Distribuida	Escala horizontalmente añadiendo más nodos al cluster
Alta Disponibilidad	Continúa operando incluso durante fallos de nodos o particiones de red
Rebalanceo Automático	Redistribuye documentos inteligentemente cuando cambia la topología
Sin Embeddings Pre-entrenados	Vectorización adaptativa que se ajusta al corpus local

Diagrama de Alto Nivel



2. Problema que Resuelve

2.1 ¿Por qué Búsqueda Distribuida?

Los sistemas de búsqueda centralizados enfrentan limitaciones críticas:

Problema	Impacto	Solución DistriSearch
Escalabilidad limitada	Un servidor no puede indexar millones de documentos	Particionamiento horizontal con VP-Tree
Punto único de fallo	Si cae el servidor, todo el sistema falla	Replicación con afinidad semántica
Latencia en búsquedas	Búsquedas lentas con corpus grandes	Índices locales + búsqueda paralela
Búsqueda por keywords	No encuentra documentos semánticamente similares	Vectorización TF-IDF + MinHash

2.2 ¿Por qué Sin Embeddings Pre-entrenados?

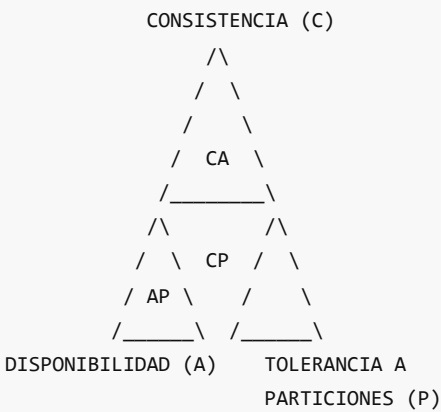
Los embeddings pre-entrenados (como Word2Vec, BERT) tienen limitaciones:

- **Dimensión fija:** Pierden información en documentos extensos
- **Dependencia externa:** Requieren modelos grandes (GB)
- **Dominio genérico:** No se adaptan al vocabulario específico del corpus

Solución DistriSearch: Vectores TF-IDF jerárquicos entrenados localmente en el corpus del cluster, combinados con MinHash para similitud eficiente.

2.3 Teorema CAP y DistriSearch

DistriSearch implementa un sistema **AP (Available & Partition-tolerant)**:



DistriSearch elige: AP

- ✓ Disponibilidad: Nodos operan independientemente
- ✓ Particiones: Soporta fallos de red
- ⚠ Consistencia: Eventual (Gossip protocol)

3. Stack Tecnológico

3.1 Backend (Python)

Tecnología	Propósito
FastAPI	Framework web asíncrono de alto rendimiento
Motor de Vectorización	TF-IDF jerárquico + MinHash + LDA
VP-Tree	Particionamiento del espacio vectorial
Raft-Lite	Consenso para elección de líder
Gossip Protocol	Sincronización eventual del registry

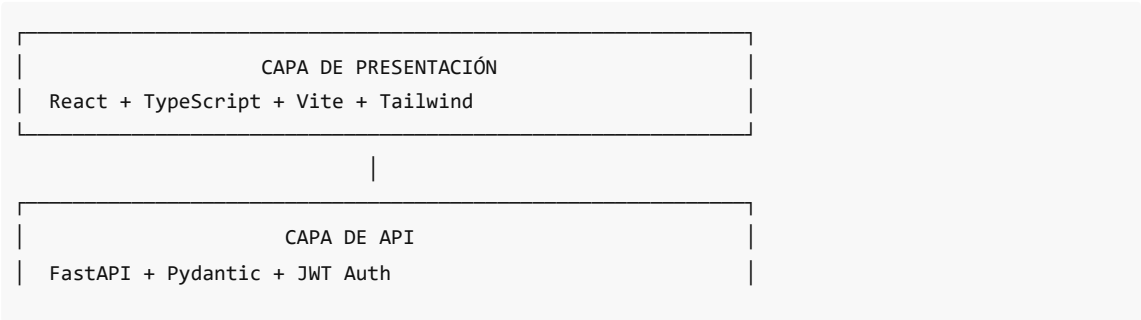
3.2 Frontend (TypeScript)

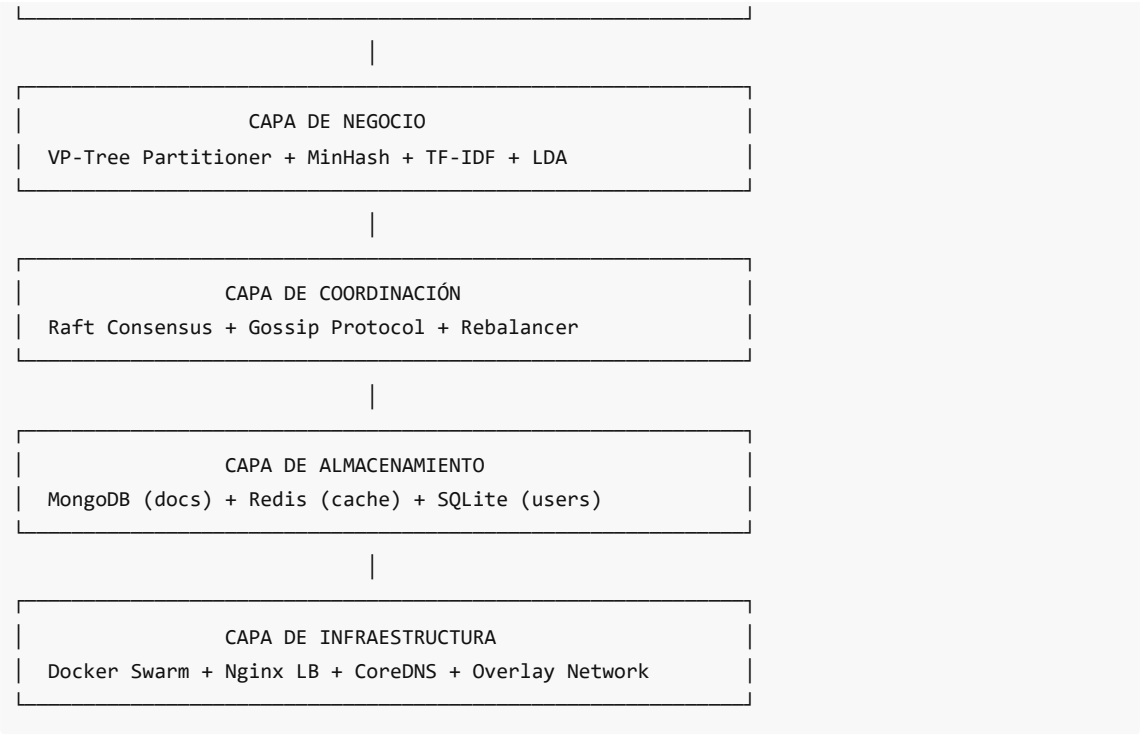
Tecnología	Propósito
React	Librería de UI con componentes
TypeScript	Tipado estático para JavaScript
Vite	Bundler rápido para desarrollo
Tailwind CSS	Framework de estilos utility-first

3.3 Infraestructura

Tecnología	Propósito
Docker Swarm	Orquestación de contenedores distribuidos
Nginx	Load balancer y terminación SSL
CoreDNS	Backup DNS para service discovery
MongoDB	Almacenamiento de documentos (local por nodo)
Redis	Caché local para búsquedas frecuentes
SQLite + Raft	Usuarios y metadatos (replicado)

3.4 Diagrama de Tecnologías por Capa





4. Audiencia Objetivo

Esta guía de estudio está diseñada para **estudiantes de la asignatura de Sistemas Distribuidos** que necesitan:

1. **Comprender la arquitectura:** Entender por qué se tomaron las decisiones de diseño
2. **Dominar los conceptos:** Raft, Gossip, VP-Tree, CAP theorem, etc.
3. **Analizar la implementación:** Código real en Python y TypeScript
4. **Preparar la exposición:** Explicar cada componente con claridad

Conocimientos Previos Recomendados

- Programación en Python y JavaScript/TypeScript
- Conceptos básicos de Docker y contenedores
- Fundamentos de redes (TCP/IP, DNS, HTTP)
- Bases de datos relacionales y NoSQL

5. Estructura de la Guía de Estudio

Esta guía está organizada en **18 secciones** que cubren todos los aspectos del sistema:

Arquitectura y Diseño

Sección	Contenido
01_Arquitectura_General	Visión Master-Slave, componentes principales, estructura del proyecto
02_Vectorizacion_Busqueda	TF-IDF jerárquico, MinHash, LSH, vectores adaptativos
03_Particionamiento	VP-Tree distribuido, algoritmo de asignación, vantage points

Distribución y Coordinación

Sección	Contenido
04_Rebalanceo	Rebalanceo activo, Power of Two Choices, migración de documentos
05_Replicacion	Afinidad semántica, grafo de similaridad, factor de replicación
06_Tolerancia_Fallos	Detección de fallos, re-replicación, recuperación de nodos
07_Consenso_Raft	Elección de líder, replicación de log, SQLite con Raft

Infraestructura

Sección	Contenido
08_Docker_Swarm	Conceptos de Swarm, overlay networks, servicios y réplicas
09_DNS_Descubrimiento	DNS interno de Docker, CoreDNS backup, service discovery
10_Load_Balancer	Configuración Nginx, routing mesh, health checks, SSL

Almacenamiento y Consistencia

Sección	Contenido
11_Almacenamiento	MongoDB local, Redis cache, SQLite usuarios, UserDocumentRegistry
12_Gossip_Protocol	Consistencia eventual, sincronización del registry
13_CAP_Theorem	Availability + Partition Tolerance, trade-offs de DistriSearch

Implementación

Sección	Contenido
14_Despliegue	Preparación de imágenes, inicialización Swarm, verificación
15_API_Backend	FastAPI estructura, endpoints principales, autenticación
16_Frontend	React + TypeScript, componentes, servicios de API
17_Testing	Tests unitarios, integración, tests distribuidos
18_Control_Center	Panel de administración, monitoreo del cluster

6. Cómo Usar Esta Guía

Para Estudio Individual

1. **Lee secuencialmente** las secciones 01-07 para entender la teoría
2. **Explora el código** referenciado en cada sección
3. **Ejecuta el sistema** localmente con `docker-compose.local.yml`
4. **Experimenta** modificando parámetros y observando el comportamiento

Para Preparar la Exposición

1. **Identifica tu tema** asignado y estudia la sección correspondiente
2. **Prepara diagramas** basándote en los ASCII arts proporcionados
3. **Practica explicando** los conceptos a un compañero
4. **Prepara preguntas** que el tribunal podría hacer

Comandos Útiles para Empezar

```
# Clonar el proyecto
git clone https://github.com/Pol4720/DistriSearch.git
cd DistriSearch

# Ejecutar en modo local (desarrollo)
docker-compose -f docker/docker-compose.local.yml up -d

# Ver logs del sistema
docker-compose -f docker/docker-compose.local.yml logs -f

# Ejecutar tests
pytest tests/ -v
```

7. Recursos Adicionales

Documentación del Proyecto

- [docs/Arquitectura de Software.md](#) - Diseño general
- [docs/ARQUITECTURA DISTRIBUIDA.md](#) - Docker Swarm y DNS
- [docs/Soluciones de ubicacion y balanceo.md](#) - Vectorización y particionamiento
- [deploy/manual-swarm/GUIA_DESPLIEGUE.md](#) - Despliegue paso a paso

Conceptos Teóricos (Referencias Externas)

- **Raft Consensus:** [The Raft Paper](#)
- **VP-Trees:** [Vantage-Point Trees](#)
- **CAP Theorem:** [Brewer's Conjecture](#)
- **Gossip Protocol:** [Epidemic Algorithms](#)
- **MinHash/LSH:** [Locality-Sensitive Hashing](#)

Nota: Esta guía está diseñada para complementar el código fuente. Se recomienda tener el proyecto abierto en VS Code mientras se estudia cada sección.

01 Arquitectura General

Arquitectura General de DistriSearch

Visión Global del Sistema Distribuido

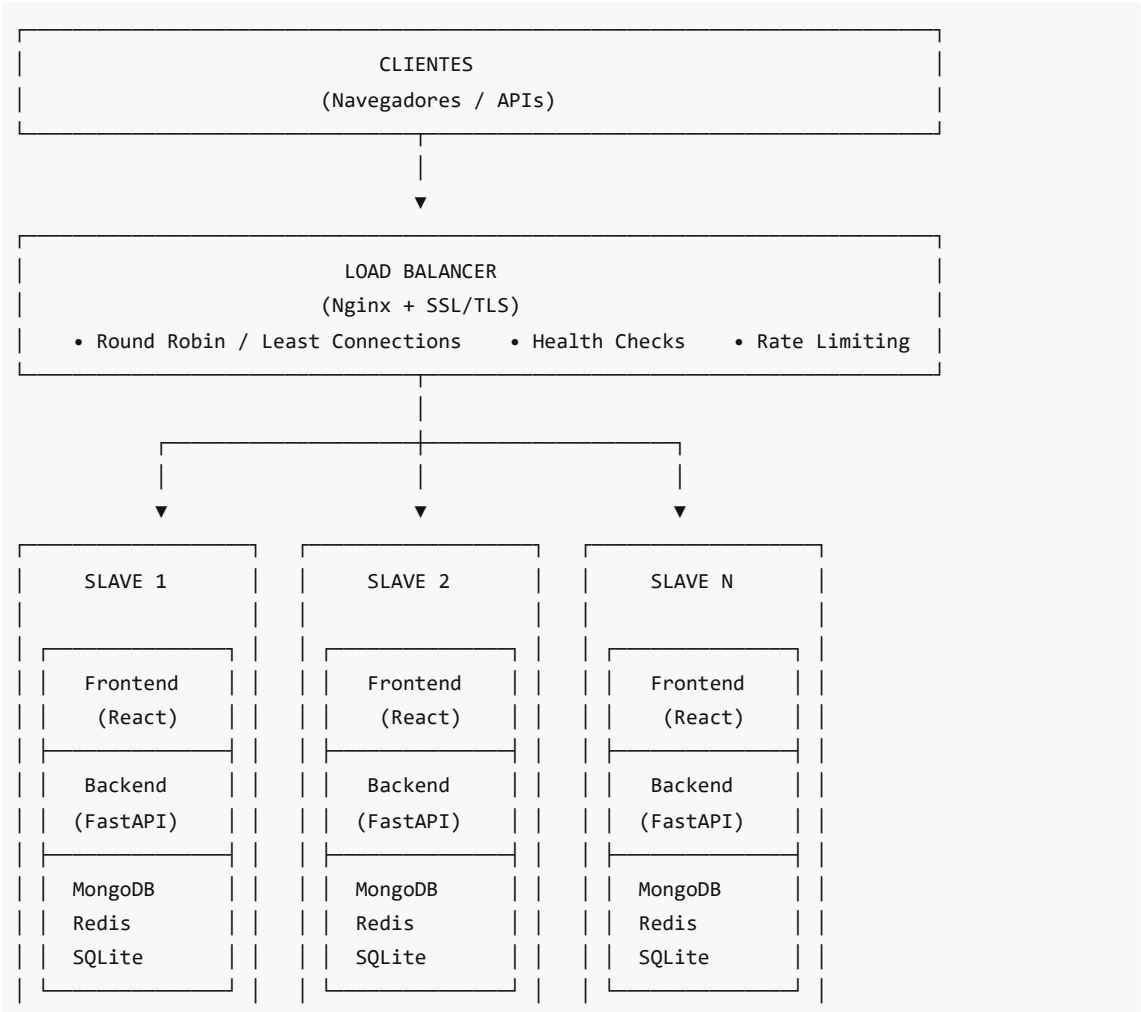
Introducción

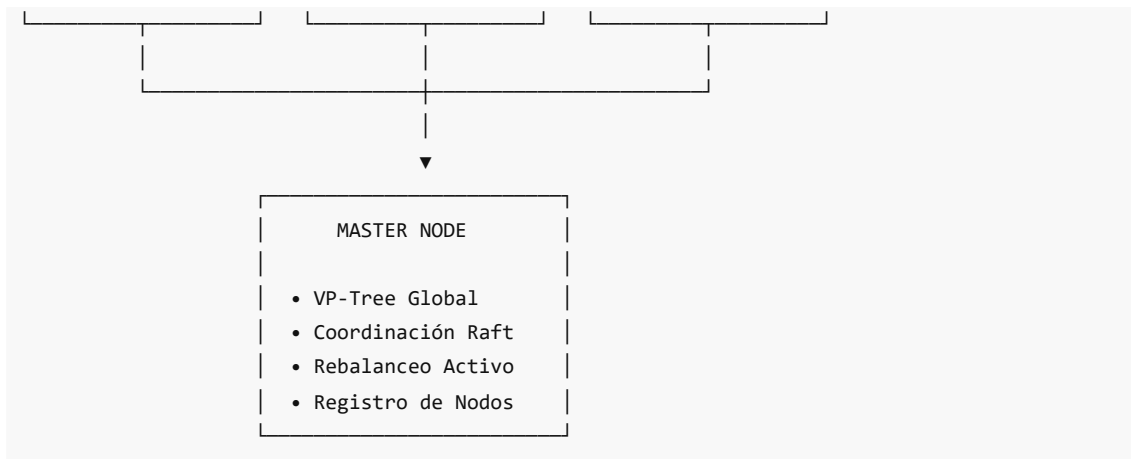
Esta sección explica la arquitectura general de DistriSearch, un sistema de búsqueda distribuida que implementa el patrón **Master-Slave con Load Balancer**. El sistema está diseñado para escalar horizontalmente, tolerar fallos y proporcionar búsqueda semántica sin depender de embeddings pre-entrenados.

Contenido de esta Sección

Archivo	Tema	Descripción
01 Vision Sistema Master Slave.md	Patrón Master-Slave	Roles del Master y Slaves, flujo de comunicación, arquitectura AP
02 Componentes Principales.md	Componentes	Load Balancer, nodos Slave, nodo Master, almacenamiento
03 Estructura Proyecto.md	Estructura de Código	Organización de carpetas, archivos clave, configuraciones

Diagrama de Arquitectura de Alto Nivel





Principios de Diseño

1. Escalabilidad Horizontal

El sistema puede crecer añadiendo más nodos Slave sin modificar el código:

```
Inicial:  [Slave-1] [Slave-2]
          ↓       ↓
Escalado: [Slave-1] [Slave-2] [Slave-3] [Slave-4]
```

2. Tolerancia a Particiones (CAP: AP)

DistriSearch elige **Disponibilidad + Tolerancia a Particiones**:

- Cada nodo puede operar **independientemente** durante particiones de red
- **SQLite local** permite autenticación sin conexión al Master
- **Consistencia eventual** vía protocolo Gossip

3. Autonomía de Nodos

Cada Slave es una **unidad autónoma** que contiene:

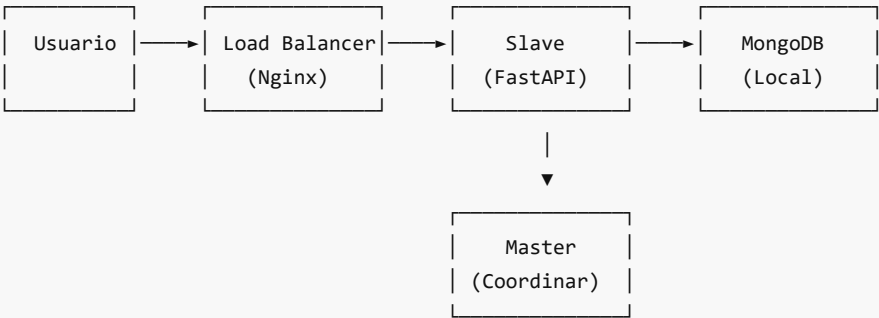
- Su propio Frontend servido por Nginx
- Backend API completo (FastAPI)
- Base de datos MongoDB local
- Caché Redis local
- SQLite para usuarios (replicado con Raft)

4. Coordinación Centralizada

El Master coordina operaciones que requieren visión global:

- Asignación de documentos a nodos (VP-Tree)
- Rebalanceo cuando cambia la topología
- Elección de líder mediante Raft

Flujo de Datos Principal



1. Usuario hace request → Load Balancer
2. Load Balancer selecciona Slave (Round Robin)
3. Slave procesa la petición localmente
4. Para operaciones de escritura: consulta al Master
5. Master coordina particionamiento y replicación

Tecnologías Principales

Capa	Tecnología	Propósito
Presentación	React + TypeScript	Interfaz de usuario
API	FastAPI (Python)	Backend REST
Orquestación	Docker Swarm	Despliegue distribuido
Balanceo	Nginx	Distribución de carga
Búsqueda	VP-Tree + MinHash	Particionamiento semántico
Consenso	Raft-Lite	Elección de líder
Datos	MongoDB + Redis + SQLite	Almacenamiento híbrido

Referencias del Código

Los archivos principales que definen la arquitectura:

```
DistriSearch/
├── backend/app/main.py           # Punto de entrada FastAPI
├── backend/app/config.py        # Configuración del sistema
├── backend/app/distributed/     # Módulos de distribución
│   ├── consensus/              # Implementación Raft
│   ├── coordination/           # Coordinación del cluster
│   └── communication/          # Comunicación entre nodos
├── docker/
│   ├── master/Dockerfile       # Imagen del Master
│   ├── slave/Dockerfile        # Imagen del Slave
│   └── load-balancer/nginx.conf # Configuración Nginx
└── shared/models/              # Modelos compartidos
```

```
|— node.py           # Modelo de nodo
|— cluster.py        # Modelo de cluster
|— document.py       # Modelo de documento
```

Preguntas Frecuentes de Estudio

1. ¿Por qué Master-Slave en lugar de P2P?

- Simplicidad en la coordinación
- VP-Tree requiere visión global
- Más fácil de debuggear y monitorizar

2. ¿Qué pasa si el Master falla?

- Los Slaves continúan operando (modo degradado)
- Raft elige un nuevo líder automáticamente
- Las búsquedas locales siguen funcionando

3. ¿Por qué MongoDB local en cada Slave?

- Elimina punto único de fallo
- Permite operación durante particiones
- Cada nodo es autónomo

Siguiente: [01 Vision Sistema Master Slave.md](#) para entender en detalle el patrón arquitectónico.

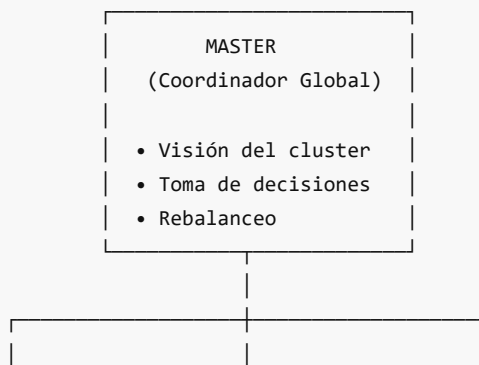
Visión del Sistema Master-Slave

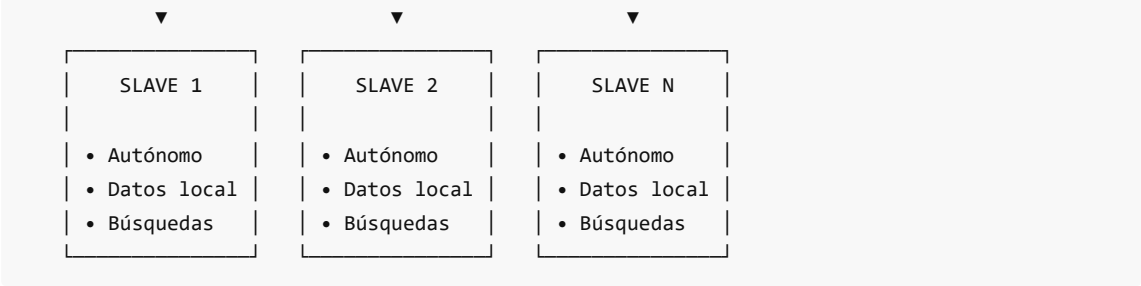
Patrón Arquitectónico de DistriSearch

1. ¿Qué es el Patrón Master-Slave?

El patrón **Master-Slave** (también llamado Primary-Replica) es una arquitectura donde:

- **Un nodo Master** coordina y toma decisiones globales
- **Múltiples nodos Slave** ejecutan el trabajo y almacenan datos
- El Master tiene **visión global** del sistema
- Los Slaves operan de forma **semi-autónoma**





2. Rol del Master Node

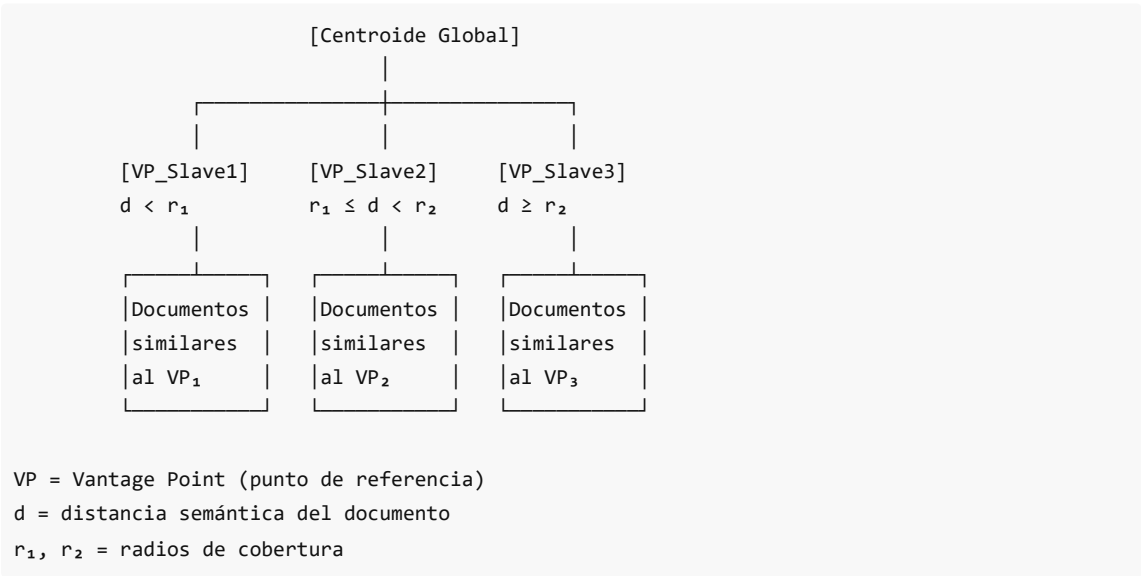
2.1 Responsabilidades Principales

El Master en DistriSearch tiene las siguientes funciones:

Responsabilidad	Descripción	Componente
VP-Tree Global	Mantiene el árbol de particionamiento semántico	backend/app/core/partitioning/
Coordinación de Particiones	Decide qué documentos van a qué nodos	VPtreePartitioner
Consenso Raft	Elección de líder y replicación de estado	backend/app/distributed/consensus/
Rebalanceo Activo	Redistribuye documentos cuando cambia la topología	ActiveRebalancer
Registro de Nodos	Mantiene el estado de todos los Slaves	NodeRegistry

2.2 VP-Tree Global

El Master mantiene un **Vantage-Point Tree** que particiona el espacio vectorial:



¿Cómo funciona la asignación?

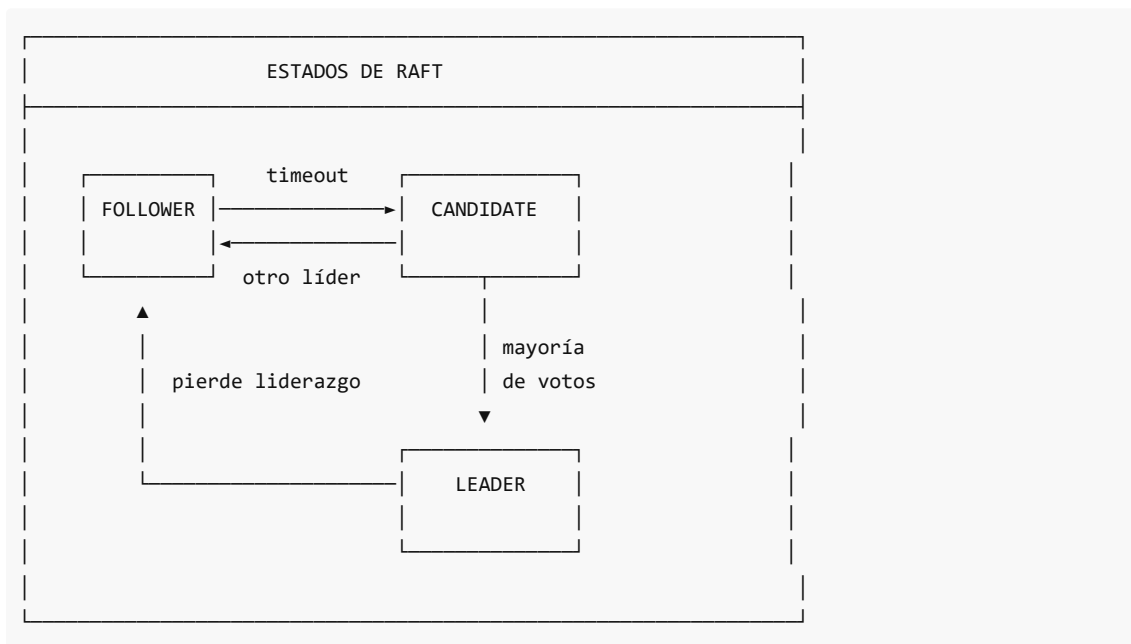
```
# Pseudocódigo del algoritmo de asignación (simplificado)
class VPTreePartitioner:
    def assign_document(self, doc_vector) -> str:
        """Encuentra el nodo más apropiado para un documento."""
        best_node = None
        best_distance = float('inf')

        for node_id, vantage_point in self.vantage_points.items():
            distance = doc_vector.compute_distance(vantage_point)
            if distance < best_distance:
                best_distance = distance
                best_node = node_id

        return best_node
```

2.3 Consenso Raft para Alta Disponibilidad

El Master utiliza **Raft** para garantizar que siempre haya un líder:



Configuración Raft en DistriSearch (de `backend/app/config.py`):

```
# Raft Consensus (milliseconds)
raft_election_timeout_min: int = 150 # Timeout mínimo para elección
raft_election_timeout_max: int = 300 # Timeout máximo para elección
raft_heartbeat_interval: int = 50    # Intervalo de heartbeat del líder
```

2.4 Rebalanceo Activo

Cuando un nodo se une o abandona el cluster:



1. DETECCIÓN: Nuevo nodo N₄ se une al cluster

2. ANÁLISIS: Master calcula carga actual

Slave1: 45% ██████████░░░░░░░░░░

Slave2: 80% ████████████████████░░

Slave3: 75% ████████████████████░░

Slave4: 0% ░░░░░░░░░░░░░░░░░░░░ (nuevo)

3. SELECCIÓN: Identificar documentos a migrar

- Docs en nodos sobrecargados (>70%)
- Docs cuyo VP más cercano ahora es N₄

4. MIGRACIÓN GRADUAL:

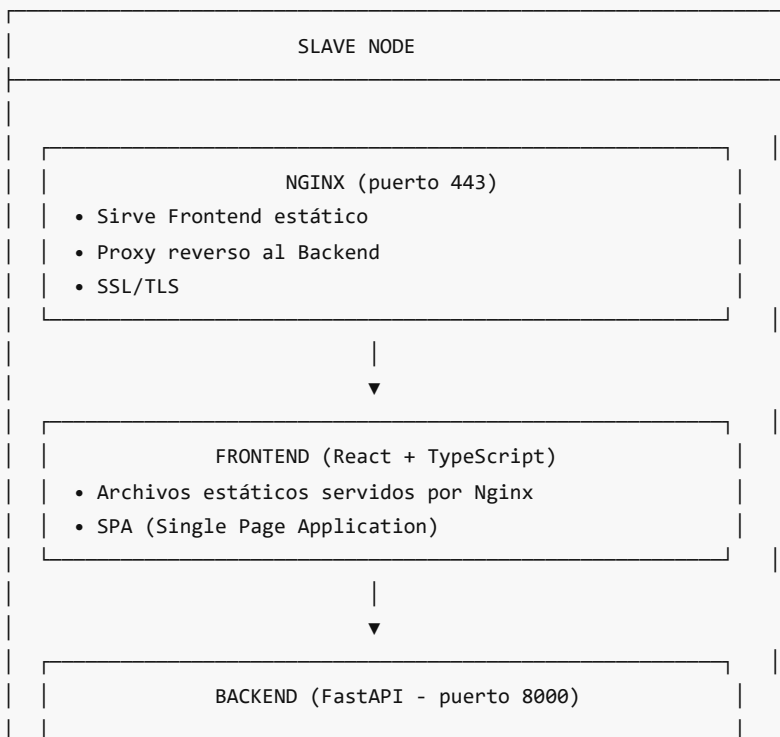
- Transferir en batches de 50 documentos
- Rate limiting: 1 segundo entre batches
- Mantener réplica temporal hasta confirmar

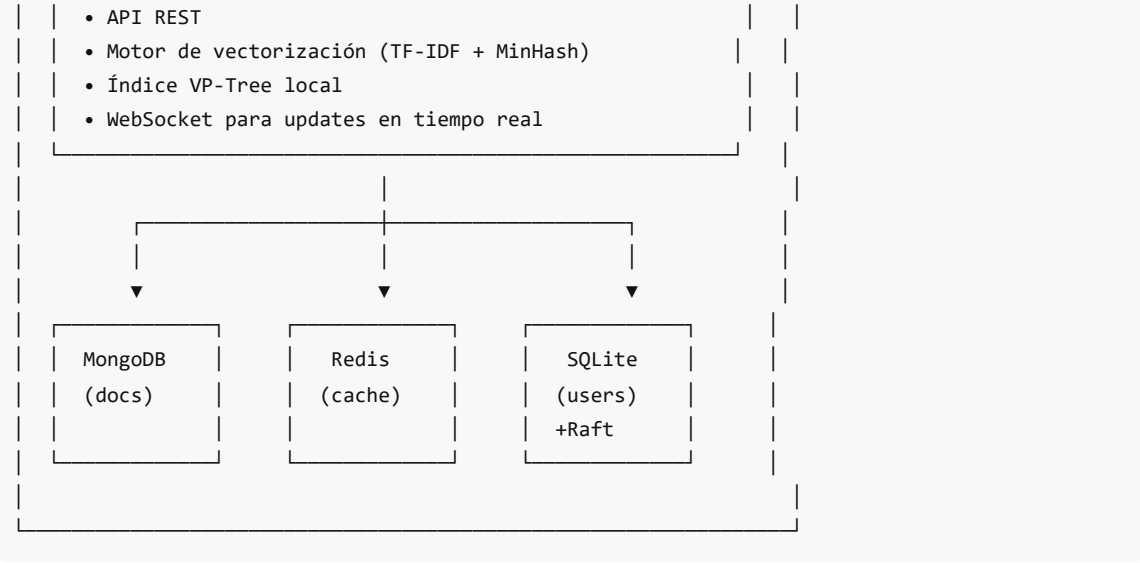
5. ACTUALIZACIÓN: Recalcular VP-Tree

3. Rol de los Nodos Slave

3.1 Arquitectura de un Slave

Cada Slave es una **unidad autónoma** que contiene todo lo necesario:





3.2 Operación Autónoma

Los Slaves pueden funcionar **independientemente** del Master:

Operación	¿Requiere Master?	Motivo
Autenticación de usuarios	✗ No	SQLite local con Raft
Búsqueda en documentos locales	✗ No	MongoDB e índice local
Consultar caché	✗ No	Redis local
Subir nuevo documento	✓ Sí	Master asigna nodo destino
Búsqueda distribuida	✓ Sí	Master coordina agregación
Ver estado del cluster	✓ Sí	Master tiene visión global

3.3 Dockerfile del Slave

El Slave se construye en **multi-stage** para optimizar tamaño:

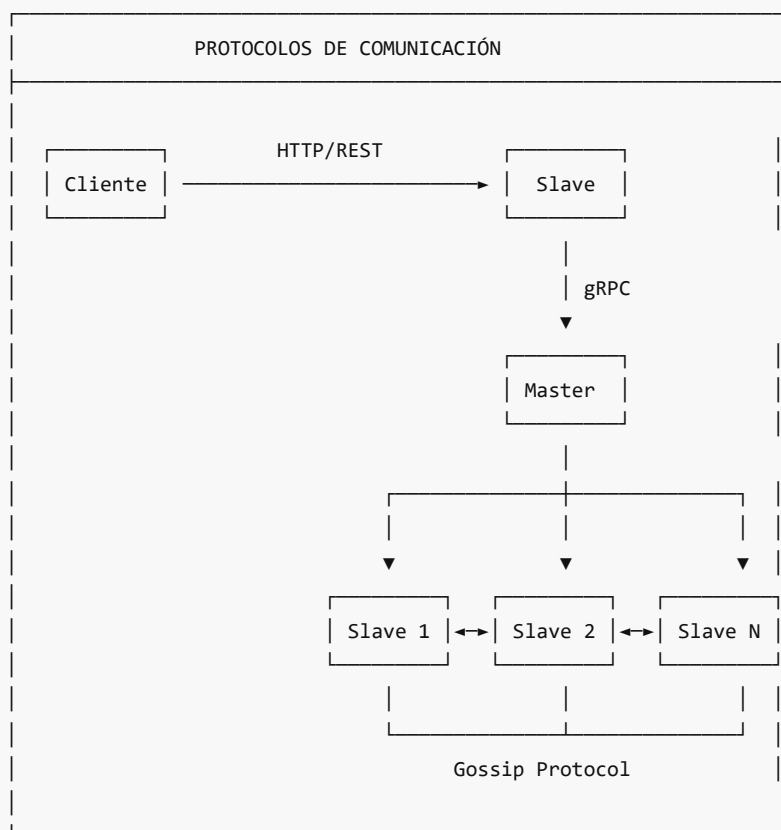
```
# Stage 1: Build Frontend
FROM node:20-alpine AS frontend-builder
WORKDIR /app/frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
RUN npm run build

# Stage 2: Build Backend Dependencies
FROM python:3.11-slim AS backend-builder
WORKDIR /app
RUN python -m venv /opt/venv
COPY backend/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Stage 3: Final Runtime Image
FROM python:3.11-slim
RUN apt-get install -y nginx supervisor curl openssl
COPY --from=frontend-builder /app/frontend/dist /var/www/html
COPY --from=backend-builder /opt/venv /opt/venv
COPY backend/ ./backend/
EXPOSE 443 8000
```

4. Comunicación entre Nodos

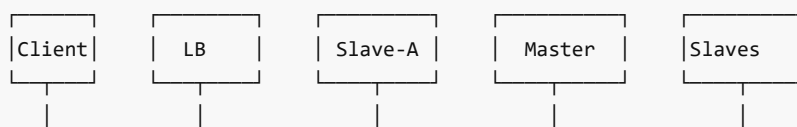
4.1 Protocolos de Comunicación

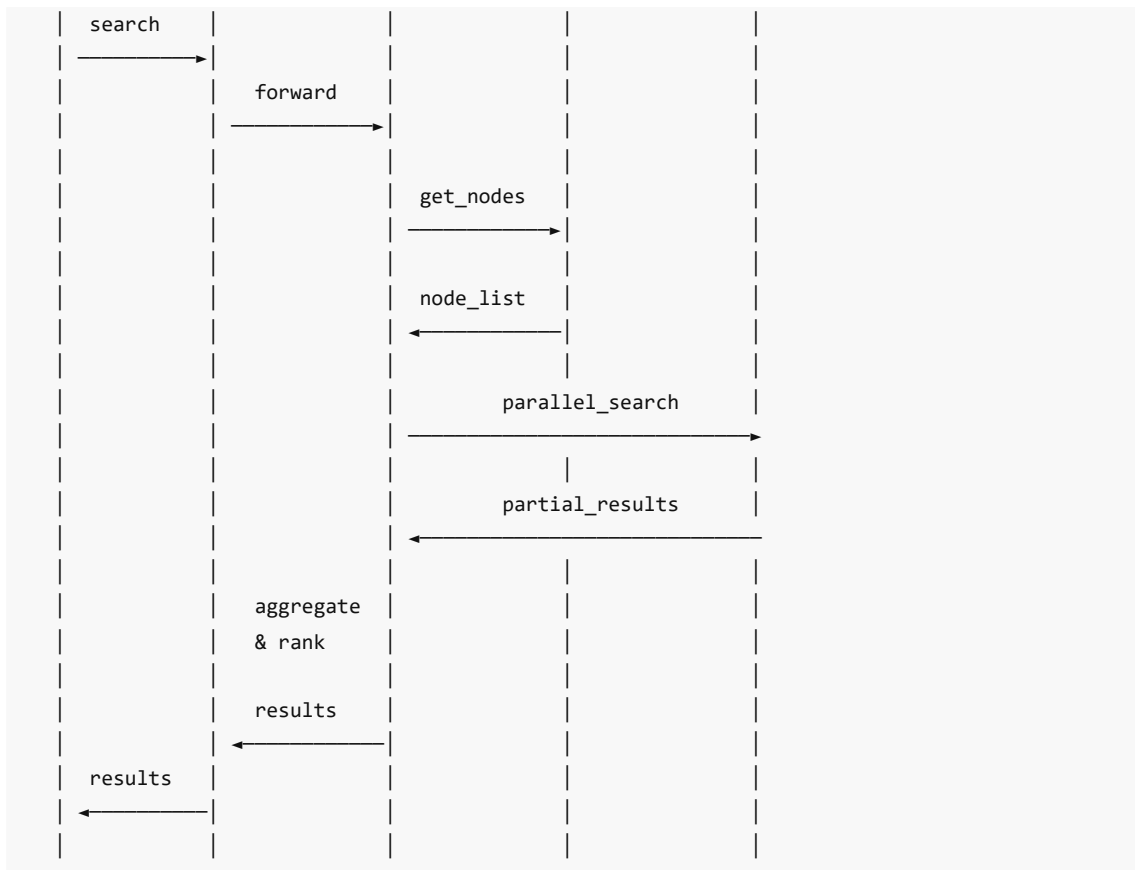


Resumen:

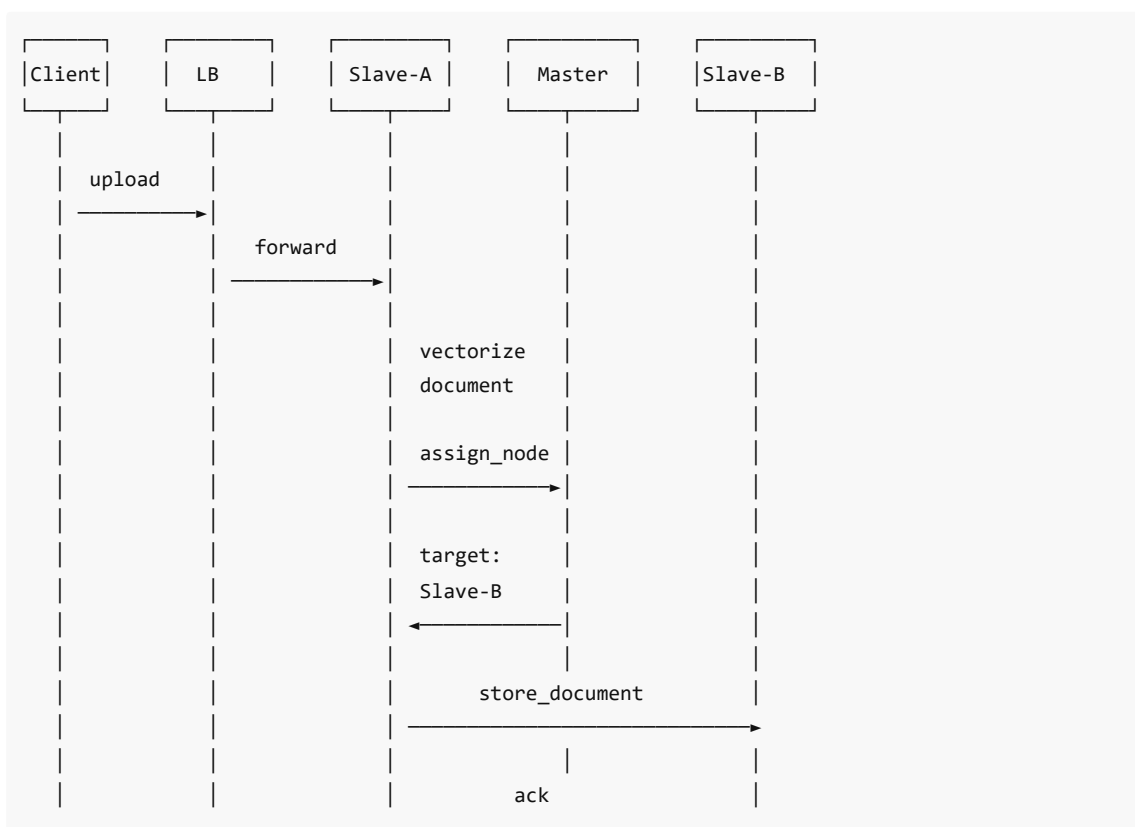
- Cliente → Slave: HTTP/REST (puerto 8000 o 443)
- Slave → Master: gRPC (puerto 50051)
- Slave ↔ Slave: Gossip Protocol (UDP)

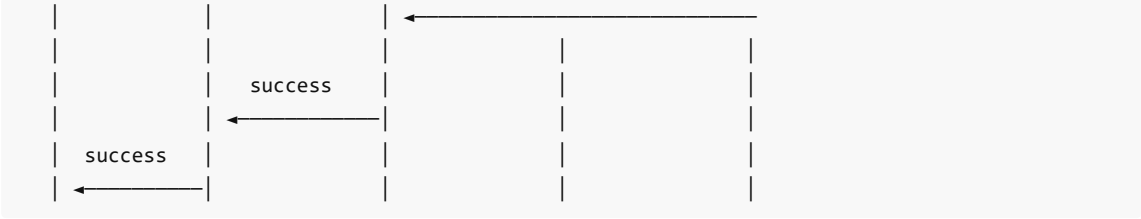
4.2 Flujo de una Búsqueda Distribuida





4.3 Flujo de Subida de Documento

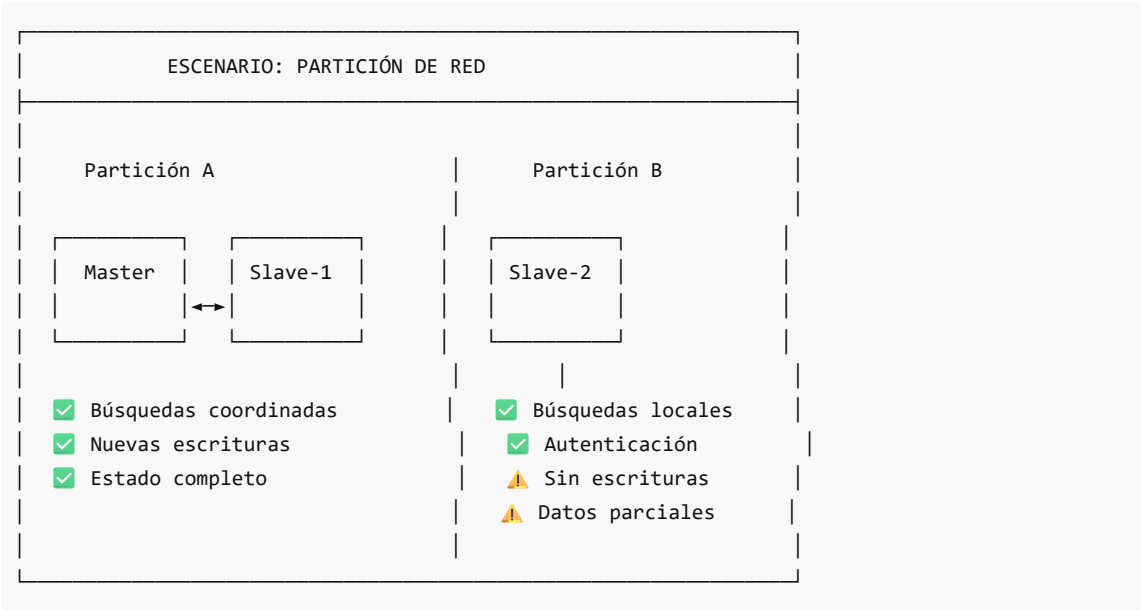




5. Diseño AP (Available & Partition-tolerant)

5.1 Arquitectura que Tolera Particiones

DistriSearch implementa el modelo **AP** del teorema CAP:



5.2 Componentes por Tipo de Consistencia

Componente	Tipo	Consistencia	Motivo
SQLite (Raft)	Usuarios, nodos	Fuerte (Raft)	Autenticación requiere consistencia
MongoDB	Documentos	Local por nodo	Cada slave es autónomo
Redis	Caché	Local	No requiere consistencia
UserDocumentRegistry	Mapeo user→docs	Eventual (Gossip)	Tolera inconsistencias temporales

5.3 Configuración de Replicación

Del archivo `backend/app/config.py` :

```
# Replicación
replication_factor: int = 2          # Número de copias por documento
min_replicas_for_write: int = 1     # Mínimo para confirmar escritura

# Heartbeat (detección de fallos)
```

```
heartbeat_interval: int = 5          # Segundos entre heartbeats
heartbeat_timeout: int = 15         # Timeout para marcar nodo como fallido
max_heartbeat_failures: int = 3     # Fallos antes de declarar nodo muerto
```

6. Resumen: Master vs Slave

Aspecto	Master	Slave
Cantidad	1-2 (activo + standby)	N (escalable)
Función principal	Coordinar	Almacenar y servir
Datos	Metadatos del cluster	Documentos reales
Estado durante partición	Degrada el cluster	Opera autónomamente
Puerto API	8001	8000
Puerto gRPC	50051	50051
Almacenamiento	SQLite (Raft)	MongoDB + Redis + SQLite

Referencias del Código

```
# Modelo de Nodo (shared/models/node.py)
class NodeRole(str, Enum):
    MASTER = "master"
    SLAVE = "slave"

class NodeStatus(str, Enum):
    ACTIVE = "active"
    INACTIVE = "inactive"
    DRAINING = "draining"    # Siendo removido
    FAILED = "failed"
    STARTING = "starting"
    SYNCING = "syncing"      # Sincronizando después de recuperación
```

```
# Configuración de nodo (backend/app/config.py)
node_id: str = Field(default="node-1", alias="NODE_ID")
node_role: str = Field(default="slave", alias="NODE_ROLE")
master_host: str = Field(default="master", alias="MASTER_HOST")
master_port: int = Field(default=8001, alias="MASTER_PORT")
```

Siguiente: [02 Componentes Principales.md](#) para detalles de cada componente técnico.

Componentes Principales de DistriSearch

Análisis Detallado de Cada Componente

1. Load Balancer (Nginx)

1.1 Propósito

El Load Balancer es el **punto de entrada** al sistema. Todas las peticiones de clientes pasan primero por él.

LOAD BALANCER (NGINX)
ENTRADA • Puerto 443 (HTTPS) • Puerto 80 (HTTP → redirect a HTTPS)
FUNCIONES 1. Terminación SSL/TLS 2. Distribución de tráfico (Round Robin / Least Connections) 3. Health Checks a backends 4. Rate Limiting (protección DDoS) 5. Compresión Gzip 6. Caché de archivos estáticos
SALIDA • Backend API: proxy a slaves:8000 • Frontend: archivos estáticos

1.2 Configuración Principal

Del archivo `docker/load-balancer/nginx.conf` :

```
# DNS interno de Docker para descubrimiento dinámico
resolver 127.0.0.11 valid=10s ipv6=off;
resolver_timeout 5s;

# Rate limiting
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=100r/s;
limit_req_zone $binary_remote_addr zone=upload_limit:10m rate=10r/s;

# Upstream para el Master
upstream master_api {
    server master:8001 max_fails=3 fail_timeout=30s;
    keepalive 16;
}
```

```
# Compresión
gzip on;
gzip_comp_level 6;
gzip_types text/plain text/css application/json application/javascript;
```

1.3 Algoritmos de Balanceo

Algoritmo	Descripción	Cuándo Usar
Round Robin	Rotación secuencial	Carga uniforme, nodos similares
Least Connections	Al nodo con menos conexiones activas	Requests de duración variable
IP Hash	Mismo cliente siempre al mismo nodo	Sesiones sticky

```
# Ejemplo: Least Connections
upstream backend_slaves {
    least_conn;
    server slave1:8000 weight=5;
    server slave2:8000 weight=5;
    server slave3:8000 weight=5;
}
```

1.4 Health Checks

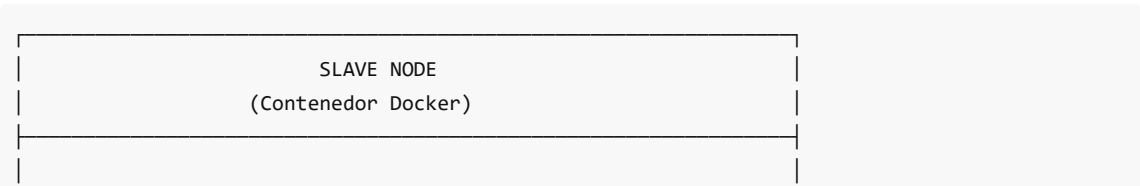
```
# Health check endpoint interno
server {
    listen 8080;
    server_name localhost;

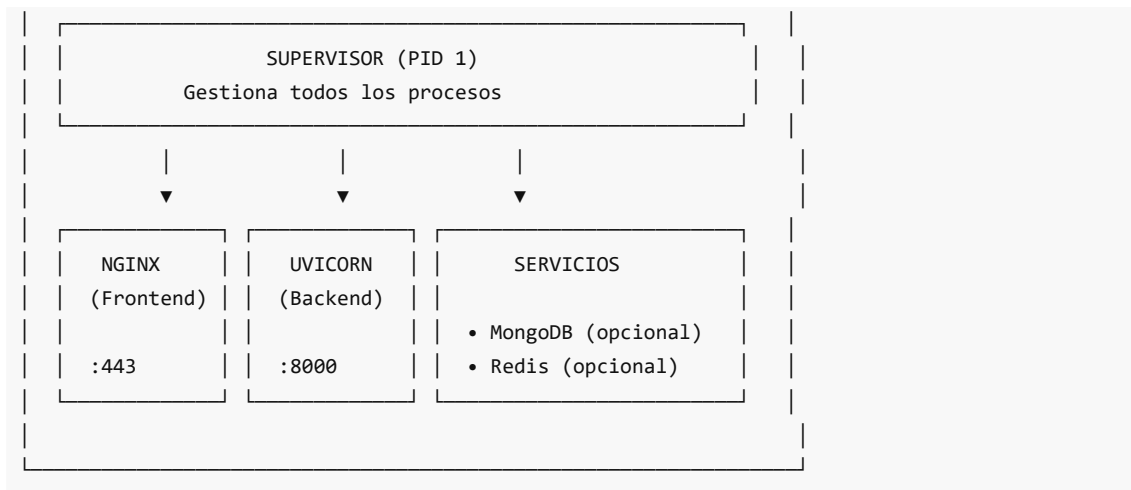
    location /nginx_status {
        stub_status on;
        allow 127.0.0.1;
        allow 10.0.0.0/8;
        deny all;
    }
}
```

2. Nodos Slave

2.1 Arquitectura Interna

Cada Slave combina **Frontend + Backend** en un solo contenedor:





2.2 Frontend (React + TypeScript)

Tecnologías:

- **React 18:** Librería de UI con hooks
- **TypeScript:** Tipado estático
- **Vite:** Bundler rápido para desarrollo y producción
- **Tailwind CSS:** Framework de estilos utility-first

Estructura del Frontend (frontend/src/):

```
frontend/src/
├─ main.tsx           # Entry point
├─ App.tsx            # Componente raíz
├─ index.css          # Estilos globales (Tailwind)
├─
├─ components/        # Componentes React
│  └─ search/         # Búsqueda
│  └─ upload/         # Subida de archivos
│  └─ dashboard/      # Panel de control
│  └─ layout/         # Layout (Header, Sidebar)
├─
├─ pages/             # Páginas/Vistas
│  └─ HomePage.tsx
│  └─ SearchPage.tsx
│  └─ DashboardPage.tsx
├─
├─ hooks/             # Custom hooks
│  └─ useSearch.ts
│  └─ useClusterStatus.ts
├─
├─ services/          # Servicios API
│  └─ api.ts
├─
└─ types/             # TypeScript types
   └─ index.ts
```

Archivos de configuración:

```
// vite.config.ts
export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    sourcemap: true
  }
})
```

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,
    "jsx": "react-jsx"
  }
}
```

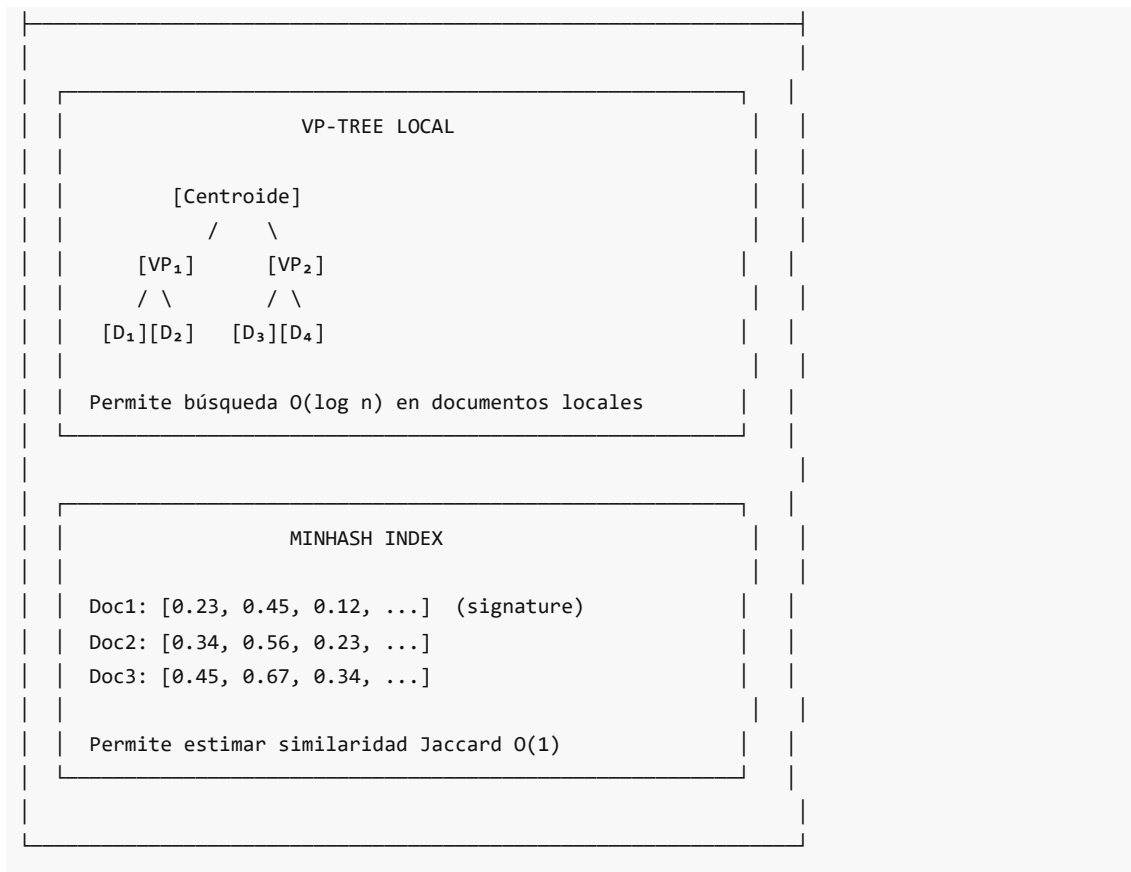
2.3 Backend (FastAPI)

Tecnologías:

- **FastAPI:** Framework web asíncrono de alto rendimiento
- **Pydantic:** Validación de datos con tipos
- **Uvicorn:** Servidor ASGI
- **Motor:** Cliente async para MongoDB

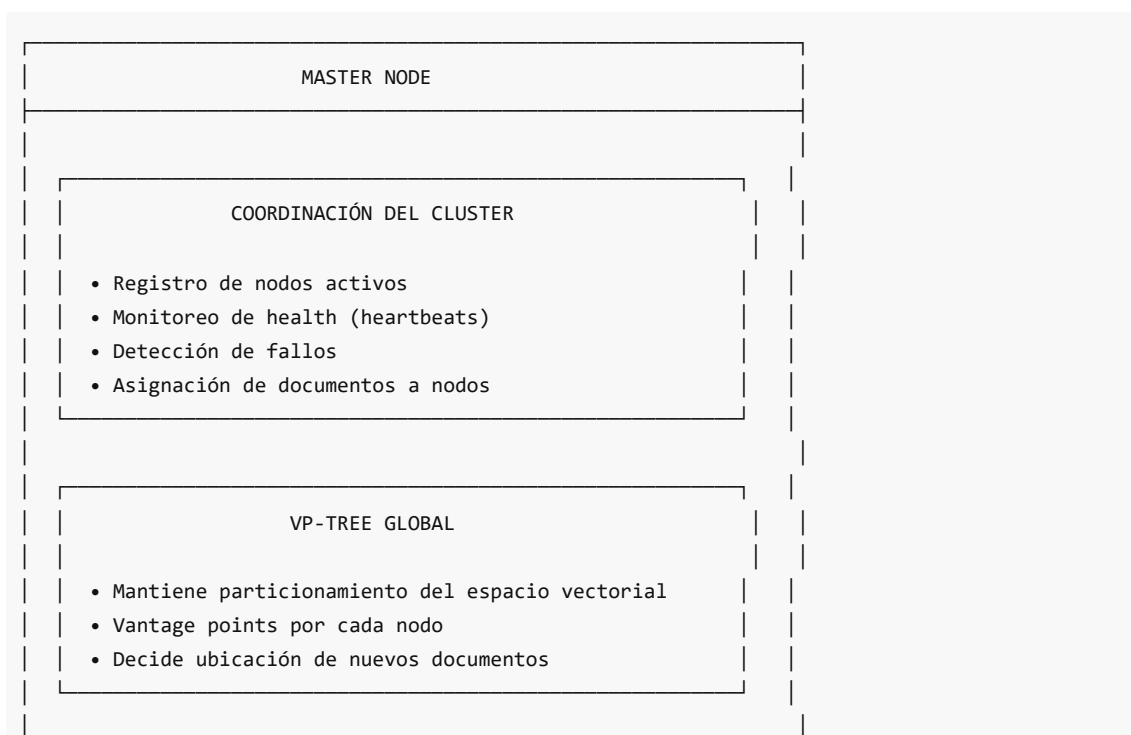
Estructura del Backend (backend/app/):

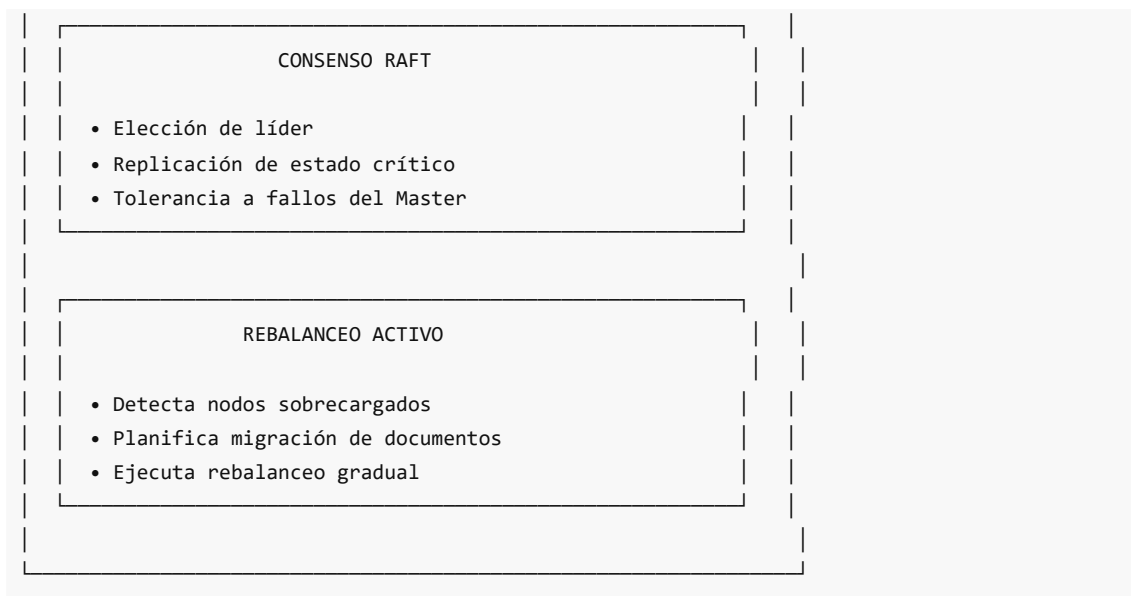
```
backend/app/
├─ main.py           # Entry point FastAPI
├─ config.py         # Configuración (Pydantic Settings)
├─
├─ api/              # Endpoints REST
│  ├─ router.py      # Router principal
│  ├─ dependencies.py # Inyección de dependencias
│  ├─ websocket.py   # WebSocket endpoints
│  └─ v1/
│     └─ endpoints/
│        ├─ search.py   # POST /search
│        ├─ documents.py # CRUD documentos
│        ├─ upload.py   # POST /upload
│        ├─ cluster.py  # Estado del cluster
│        └─ health.py   # Health checks
├─
├─ core/             # Núcleo de negocio
│  ├─ vectorization/  # TF-IDF, MinHash, LDA
│  ├─ partitioning/   # VP-Tree, asignación
│  ├─ rebalancing/    # Rebalanceo activo
│  └─ replication/    # Afinidad semántica
```

3. Nodo Master

3.1 Responsabilidades





3.2 Dockerfile del Master

```
# docker/master/Dockerfile
FROM python:3.11-slim

RUN apt-get update && apt-get install -y curl

WORKDIR /app

# Dependencias Python
COPY backend/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Código fuente
COPY backend/ ./backend/
COPY shared/ ./shared/

# Variables de entorno
ENV NODE_ROLE=master \
    API_PORT=8001

EXPOSE 8001

HEALTHCHECK --interval=10s --timeout=5s --retries=3 \
    CMD curl -f http://localhost:8001/api/v1/health/live || exit 1

CMD ["python", "-m", "uvicorn", "backend.app.main:app", \
    "--host", "0.0.0.0", "--port", "8001"]
```

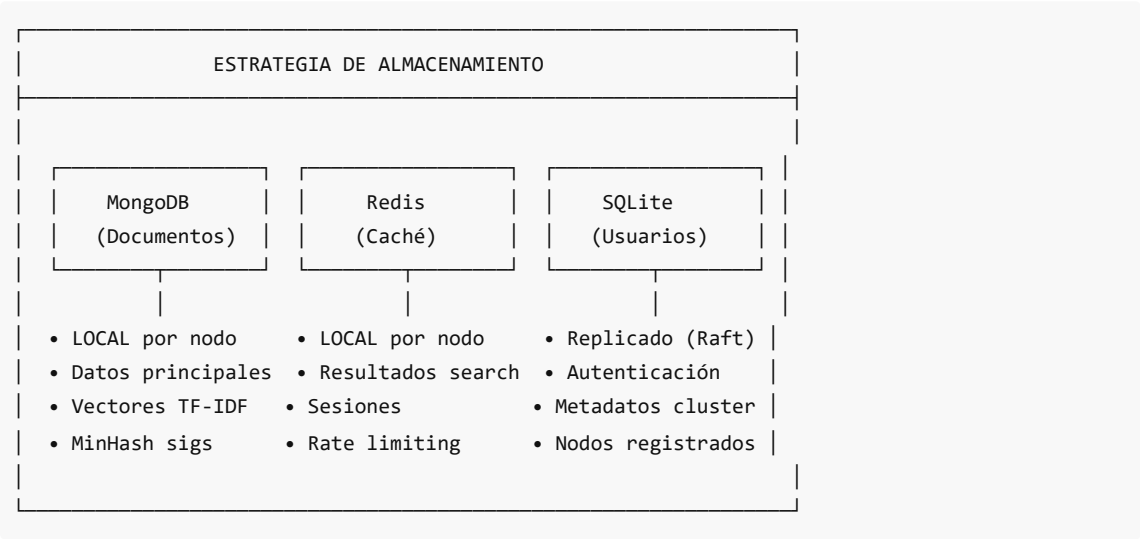
3.3 API del Master

Endpoints específicos del Master:

Endpoint	Método	Descripción
/api/v1/cluster/nodes	GET	Lista de nodos activos
/api/v1/cluster/status	GET	Estado del cluster
/api/v1/cluster/assign	POST	Asignar documento a nodo
/api/v1/cluster/rebalance	POST	Iniciar rebalanceo
/api/v1/health/live	GET	Liveness check
/api/v1/health/ready	GET	Readiness check

4. Almacenamiento

4.1 Arquitectura de Almacenamiento



4.2 MongoDB (Documentos)

Propósito: Almacenar documentos y sus vectores

```
# Esquema del documento en MongoDB
{
  "_id": ObjectId("..."),
  "doc_id": "uuid-string",
  "filename": "reporte_ventas_2024.pdf",
  "content_hash": "sha256:...",
  "created_at": ISODate("2024-01-15T10:30:00Z"),
  "updated_at": ISODate("2024-01-15T10:30:00Z"),
  "owner_id": "user-uuid",
  "node_id": "slave-1",

  # Vectores
  "vectors": {
    "name_tfidf": [...],           # Vector TF-IDF del nombre
```

```

        "content_minhash": [...],      # Signatures MinHash
        "topic_distribution": [...],    # Distribución LDA
    },

    # Metadatos
    "metadata": {
        "size_bytes": 1024000,
        "mime_type": "application/pdf",
        "pages": 15
    }
}

```

Configuración (de `backend/app/config.py`):

```

mongodb_uri: str = Field(
    default="mongodb://mongodb:27017/distrisearch",
    alias="MONGODB_URI"
)
mongodb_database: str = Field(default="distrisearch")
mongodb_max_pool_size: int = Field(default=50)

```

4.3 Redis (Caché)

Propósito: Caché de búsquedas frecuentes y sesiones

```

# Ejemplo de uso de caché
cache_key = f"search:{query_hash}"
cached_result = await redis.get(cache_key)

if cached_result:
    return json.loads(cached_result)

# Ejecutar búsqueda
result = await search_engine.search(query)

# Guardar en caché (TTL: 5 minutos)
await redis.setex(cache_key, 300, json.dumps(result))

```

Configuración:

```

redis_url: str = Field(default="redis://redis:6379")
redis_max_connections: int = Field(default=20)

```

4.4 SQLite con Raft (Usuarios)

Propósito: Datos críticos que requieren consistencia durante particiones

SQLite + RAFT REPLICATION



Tablas replicadas:

- users (autenticación)
- nodes (registro de nodos)
- partitions (mapeo documento → nodo)
- sessions (sesiones activas)

Beneficio: Autenticación funciona durante particiones

5. Modelos Compartidos

5.1 Modelos de Datos

Del directorio `shared/models/` :

```
# shared/models/node.py
class NodeRole(str, Enum):
    MASTER = "master"
    SLAVE = "slave"

class NodeStatus(str, Enum):
    ACTIVE = "active"
    INACTIVE = "inactive"
    DRAINING = "draining"
    FAILED = "failed"
    STARTING = "starting"
    SYNCING = "syncing"

class Node(BaseModel):
    node_id: str
    role: NodeRole = NodeRole.SLAVE
    status: NodeStatus = NodeStatus.STARTING
    host: str
    port: int = 8000
    grpc_port: int = 50051
    cluster_id: Optional[str] = None
    joined_at: Optional[datetime] = None
```

```
# shared/models/document.py
class Document(BaseModel):
    doc_id: str
    filename: str
```

```
content_hash: str
owner_id: str
node_id: str
vectors: DocumentVectors
metadata: DocumentMetadata
created_at: datetime
```

```
# shared/models/cluster.py
class ClusterState(BaseModel):
    cluster_id: str
    leader_id: Optional[str]
    nodes: List[Node]
    total_documents: int
    healthy: bool
```

6. Resumen de Componentes

Componente	Tecnología	Puerto	Propósito
Load Balancer	Nginx	443/80	Entrada, SSL, balanceo
Frontend	React + TS	(Nginx)	UI de usuario
Backend	FastAPI	8000	API REST
Master API	FastAPI	8001	Coordinación
gRPC	grpcio	50051	Comunicación interna
MongoDB	MongoDB 6.0	27017	Documentos
Redis	Redis 7	6379	Caché
SQLite	SQLite + Raft	(file)	Usuarios

7. Configuración Docker Compose

Archivo `docker/docker-compose.yml` :

```
version: '3.8'

services:
  load-balancer:
    build: ./load-balancer
    ports:
      - "443:443"
      - "80:80"
    depends_on:
      - slave

  master:
```

```
build: ./master
environment:
  - NODE_ROLE=master
  - RAFT_PEERS=master
ports:
  - "8001:8001"

slave:
  build: ./slave
  environment:
    - NODE_ROLE=slave
    - MASTER_HOST=master
    - MONGODB_URI=mongodb://mongodb:27017/distrisearch
  deploy:
    replicas: 3

mongodb:
  image: mongo:6.0
  volumes:
    - mongodb-data:/data/db

redis:
  image: redis:7-alpine

networks:
  default:
    driver: overlay
    attachable: true

volumes:
  mongodb-data:
```

Siguiente: [03 Estructura Proyecto.md](#) para ver la organización completa del código fuente.

Estructura del Proyecto DistriSearch

Organización del Código Fuente

1. Visión General de Directorios

```
DistriSearch/
|
├─ 📁 backend/           # API REST Python (FastAPI)
├─ 📁 frontend/          # Aplicación React + TypeScript
├─ 📁 docker/            # Configuración de contenedores
├─ 📁 shared/            # Código compartido entre componentes
├─ 📁 tests/             # Tests (unit, integration, distributed)
├─ 📁 scripts/           # Scripts de utilidad
```

```

├─ 📁 control-center/      # Panel de administración
├─ 📁 deploy/              # Guías y scripts de despliegue
├─ 📁 docs/                # Documentación técnica
├─
├─ pyproject.toml         # Configuración del proyecto Python
├─ requirements-dev.txt   # Dependencias de desarrollo
├─ pytest.ini             # Configuración de pytest
├─ mypy.ini               # Configuración de tipado estático
├─ .pre-commit-config.yaml # Hooks de pre-commit
├─

```

2. Directorio backend/

El corazón del sistema: **API REST con FastAPI**.

```

backend/
├─ .env                  # Variables de entorno (no commitear)
├─ requirements.txt      # Dependencias Python
├─
├─ app/
│   ├── __init__.py
│   ├── main.py          # 🟡 Entry point de FastAPI
│   ├── config.py        # 🟡 Configuración centralizada
│   ├──
│   ├── 📁 api/           # Capa de API REST
│   │   ├── router.py    # Router principal
│   │   ├── dependencies.py # Inyección de dependencias
│   │   ├── websocket.py  # WebSocket para tiempo real
│   │   └─ v1/
│   │       ├── endpoints/
│   │       │   ├── search.py    # POST /api/v1/search
│   │       │   ├── documents.py # CRUD documentos
│   │       │   ├── upload.py    # POST /api/v1/upload
│   │       │   ├── cluster.py   # Estado cluster
│   │       └── health.py       # Health checks
│   ├──
│   ├── 📁 core/          # Lógica de negocio
│   │   ├── vectorization/ # TF-IDF, MinHash, LDA
│   │   ├── partitioning/  # VP-Tree, asignación
│   │   ├── rebalancing/   # Rebalanceo activo
│   │   ├── replication/   # Replicación con afinidad
│   │   ├── recovery/      # Tolerancia a fallos
│   │   └─ search/         # Motor de búsqueda
│   ├──
│   ├── 📁 distributed/   # Componentes distribuidos
│   │   ├── consensus/    # Implementación Raft
│   │   ├── coordination/  # Coordinación cluster
│   │   └─ communication/ # gRPC, Gossip
│   ├──
│   ├── 📁 grpc/          # Definiciones gRPC
│   │   └─ protos/        # Archivos .proto
│   └─

```



```

├── middleware/      # Middlewares FastAPI
│   ├── auth.py     # Autenticación JWT
│   └── logging.py   # Logging estructurado
└── storage/        # Capa de datos
    ├── mongodb.py  # Cliente MongoDB
    ├── redis_cache.py # Cliente Redis
    └── sqlite_users.py # SQLite para usuarios

```

Archivos Clave del Backend

main.py - Punto de entrada:

```

"""
DistriSearch Main Application
Architecture: AP (Available & Partition-tolerant)
"""

from fastapi import FastAPI
from .config import Settings, get_settings
from .api.router import api_router

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Starting DistriSearch...")
    await init_dependencies(settings)
    yield
    await shutdown_dependencies()

def create_application() -> FastAPI:
    app = FastAPI(
        title="DistriSearch API",
        version=settings.app_version,
        lifespan=lifespan
    )
    return app

```

config.py - Configuración:

```

class Settings(BaseSettings):
    # Nodo
    node_id: str = Field(default="node-1", alias="NODE_ID")
    node_role: str = Field(default="slave", alias="NODE_ROLE")

    # Master
    master_host: str = Field(default="master", alias="MASTER_HOST")
    master_port: int = Field(default=8001, alias="MASTER_PORT")

    # MongoDB
    mongodb_uri: str = Field(default="mongodb://mongodb:27017/distrisearch")

    # Raft

```

```
raft_election_timeout_min: int = 150
raft_heartbeat_interval: int = 50

# Rebalanceo
rebalance_threshold: float = 0.8
rebalance_batch_size: int = 50
```

3. Directorio frontend/

Aplicación **React + TypeScript** con Vite.

```
frontend/
├─ index.html          # HTML base
├─ package.json        # Dependencias npm
├─ tsconfig.json       # Configuración TypeScript
├─ vite.config.ts      # Configuración Vite
├─ tailwind.config.js  # Configuración Tailwind CSS
├─ postcss.config.js   # PostCSS para Tailwind
├─
└─ src/
    ├─ main.tsx        # ● Entry point React
    ├─ App.tsx         # ● Componente raíz
    ├─ index.css       # Estilos globales (Tailwind)
    ├─ vite-env.d.ts   # Tipos de Vite
    ├─
    ├─ components/     # Componentes React
    │   ├─ search/     # Componentes de búsqueda
    │   ├─ upload/     # Componentes de subida
    │   ├─ dashboard/  # Panel de control
    │   └─ layout/     # Header, Sidebar, Footer
    │
    ├─ pages/          # Páginas/Vistas
    │   ├─ HomePage.tsx
    │   ├─ SearchPage.tsx
    │   ├─ UploadPage.tsx
    │   └─ DashboardPage.tsx
    │
    ├─ hooks/          # Custom hooks
    │   ├─ useSearch.ts
    │   ├─ useUpload.ts
    │   └─ useClusterStatus.ts
    │
    ├─ services/       # Servicios API
    │   └─ api.ts      # Cliente HTTP
    │
    └─ types/          # TypeScript types
        └─ index.ts
```

Archivos Clave del Frontend

package.json :

```
{
  "name": "distrisearch-frontend",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.x"
  },
  "devDependencies": {
    "typescript": "^5.x",
    "vite": "^5.x",
    "@types/react": "^18.x",
    "tailwindcss": "^3.x"
  }
}
```

vite.config.ts :

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:8000'
    }
  }
})
```

4. Directorio `docker/`

Configuración de **contenedores y orquestación**.

```
docker/
├─ docker-compose.yml          # ● Orquestación principal
├─ docker-compose.local.yml    # Desarrollo local
├─ docker-compose.distributed.yml # Modo distribuido
├─ docker-compose.swarm.yml    # Docker Swarm
├─ docker-compose.test.yml     # Tests
├─
├─ 📁 load-balancer/
│   ├── Dockerfile
│   ├── nginx.conf             # ● Configuración Nginx
│   └─ supervisord-lb.conf
```

```

|   |   | update-upstreams.sh      # Script dinámico
|   |   |   └─ conf.d/
|   |   |       └─ upstreams/      # Upstreams generados
|   |
|   └─ master/
|       │ Dockerfile              # ● Imagen Master
|       └─ entrypoint.sh
|
|   └─ slave/
|       │ Dockerfile              # ● Imagen Slave
|       │ entrypoint.sh
|       │ generate-ssl.sh         # Genera certificados
|       │ nginx-frontend.conf    # Nginx para frontend
|       │ nginx-https.conf       # HTTPS config
|       └─ supervisord.conf      # Supervisor config
|
|   └─ mongodb/
|       └─ init-replica.js        # Inicializa replica set
|
|   └─ coredns/
|       │ Dockerfile
|       │ Corefile                # ● Config CoreDNS
|       └─ zones/
|           └─ distrisearch.local.zone
|
|   └─ dns-sync/
|       │ Dockerfile
|       └─ sync_dns_zone.py       # Sincroniza DNS

```

Archivos Docker Clave

docker-compose.yml (simplificado):

```

version: '3.8'

services:
  load-balancer:
    build: ./load-balancer
    ports:
      - "443:443"
    networks:
      - distrisearch-network

  master:
    build: ./master
    environment:
      - NODE_ROLE=master
    ports:
      - "8001:8001"

  slave:
    build: ./slave

```

```

environment:
  - NODE_ROLE=slave
  - MASTER_HOST=master
deploy:
  replicas: 3

mongodb:
  image: mongo:6.0
  volumes:
    - mongodb-data:/data/db

redis:
  image: redis:7-alpine

networks:
  distrisearch-network:
    driver: overlay

volumes:
  mongodb-data:

```

slave/Dockerfile (multi-stage):

```

# Stage 1: Build Frontend
FROM node:20-alpine AS frontend-builder
COPY frontend/ ./
RUN npm ci && npm run build

# Stage 2: Build Backend
FROM python:3.11-slim AS backend-builder
COPY backend/requirements.txt ./
RUN pip install -r requirements.txt

# Stage 3: Runtime
FROM python:3.11-slim
RUN apt-get install -y nginx supervisor
COPY --from=frontend-builder /dist /var/www/html
COPY --from=backend-builder /opt/venv /opt/venv
COPY backend/ ./backend/
EXPOSE 443 8000

```

5. Directorio **shared/**

Código **compartido** entre componentes.

```

shared/
├── __init__.py
├──
├── constants/
│   ├── __init__.py

```

```

├── config.py          # Constantes globales
├── models/
│   ├── __init__.py
│   ├── cluster.py     # ● Modelo ClusterState
│   ├── document.py    # ● Modelo Document
│   └── node.py        # ● Modelo Node
└── protocols/
    ├── __init__.py
    ├── events.py      # Eventos del sistema
    └── messages.py    # Mensajes entre nodos

```

Modelos Compartidos

node.py :

```

class NodeRole(str, Enum):
    MASTER = "master"
    SLAVE = "slave"

class NodeStatus(str, Enum):
    ACTIVE = "active"
    INACTIVE = "inactive"
    DRAINING = "draining"
    FAILED = "failed"

class Node(BaseModel):
    node_id: str
    role: NodeRole
    status: NodeStatus
    host: str
    port: int = 8000

```

document.py :

```

class Document(BaseModel):
    doc_id: str
    filename: str
    owner_id: str
    node_id: str
    vectors: DocumentVectors
    created_at: datetime

```

6. Directorio tests/

Tests organizados por tipo.

```

tests/
├── __init__.py

```

```

|
├─ unit/                # Tests unitarios
|   └─ test_vectorization.py
|   └─ test_vp_tree.py
|   └─ test_minhash.py
|
├─ integration/         # Tests de integración
|   └─ test_api_search.py
|   └─ test_api_upload.py
|   └─ test_mongodb.py
|
└─ distributed/         # Tests del sistema distribuido
    └─ test_consensus_leader_election.py # ● Test Raft
    └─ test_load_balancing.py           # ● Test balanceo
    └─ test_replication.py
    └─ test_partition_tolerance.py

```

Configuración de Tests

pytest.ini :

```

[pytest]
testpaths = tests
python_files = test_*.py
python_functions = test_*
addopts = -v --tb=short
asyncio_mode = auto

```

backend/tests/test_adaptive_cluster.py :

```

"""Tests for adaptive cluster behavior."""

import pytest
from backend.app.distributed.coordination import ClusterCoordinator

@pytest.mark.asyncio
async def test_node_joins_cluster():
    coordinator = ClusterCoordinator()
    result = await coordinator.register_node(node_info)
    assert result.success
    assert node_info.node_id in coordinator.active_nodes

```

7. Directorio scripts/

Scripts de utilidad y automatización.

```

scripts/
├─ benchmark.py          # Benchmark de rendimiento
├─ deploy.sh             # Script de despliegue
└─ healthcheck.py        # Verificación de salud

```

```

├─ load_data.py          # Carga datos de prueba
├─ migrate_data.py       # Migración de datos
├─ setup_dev.ps1         # Setup Windows (PowerShell)
└─ setup_dev.sh          # Setup Linux/Mac

```

healthcheck.py :

```

"""Health check script for all nodes."""

async def check_node_health(node_url: str) -> bool:
    try:
        response = await client.get(f"{node_url}/api/v1/health")
        return response.status_code == 200
    except Exception:
        return False

```

8. Directorio **control-center/**

Panel de administración con Streamlit.

```

control-center/
├─ docker-compose.yml
├─ Dockerfile
├─ Dockerfile.backend
├─ Dockerfile.frontend
├─ README.md
├─
├─ ── backend/
│   ├── main.py          # API del panel
│   ├── requirements.txt
│   ├── routers/         # Endpoints
│   └─ services/         # Lógica de negocio
├─
└─ ── frontend/
    ├── 🏠_Panel_Principal.py # 🟡 Página principal Streamlit
    ├── requirements.txt
    ├── .streamlit/
    │   └─ config.toml
    └─ pages/
        ├── 1_📊_Cluster.py
        ├── 2_📁_Documents.py
        └─ 3_⚙️_Settings.py

```

9. Directorio **deploy/**

Guías y scripts de **despliegue**.

```

deploy/
└─ manual-swarm/
    └─ GUIA_DESPLIEGUE.md # 🟡 Guía paso a paso

```



```
└─ scripts/
    ├── 01-prepare-node.sh
    ├── 02-init-swarm.sh
    ├── 03-join-swarm.sh
    ├── 04-deploy-mongodb.sh
    ├── 05-deploy-master.sh
    ├── 06-deploy-slave.sh
    └─ 07-verify-cluster.sh
```

10. Archivos de Configuración Raíz

Archivo	Propósito
pyproject.toml	Configuración del proyecto Python (black, isort, etc.)
requirements-dev.txt	Dependencias de desarrollo
pytest.ini	Configuración de pytest
mypy.ini	Configuración de tipado estático
.pre-commit-config.yaml	Hooks de pre-commit
.flake8	Configuración de linter

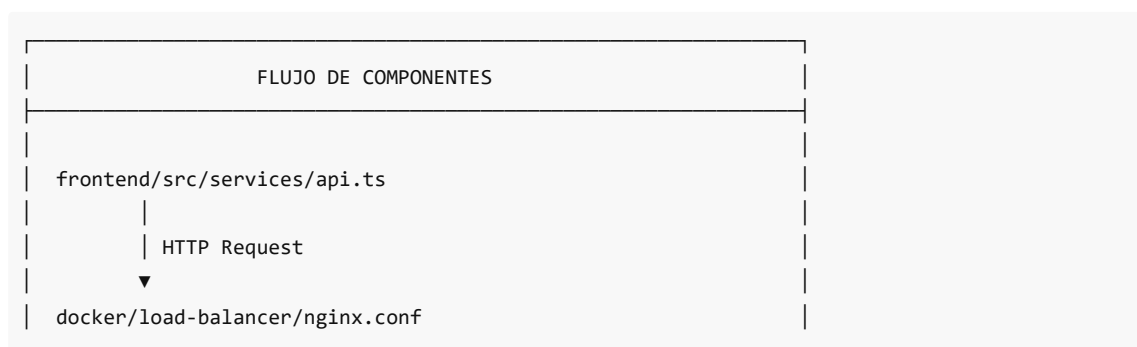
pyproject.toml :

```
[tool.black]
line-length = 88
target-version = ['py311']

[tool.isort]
profile = "black"
line_length = 88

[tool.mypy]
python_version = "3.11"
strict = true
```

11. Flujo de Interacción entre Componentes



```
| | Proxy Pass
| | ▼
| backend/app/api/v1/endpoints/*.py
| | Lógica de negocio
| | ▼
| backend/app/core/search/search_engine.py
| | Vectorización
| | ▼
| backend/app/core/vectorization/document_vectorizer.py
| | Almacenamiento
| | ▼
| backend/app/storage/mongodb.py
| | Coordinación
| | ▼
| backend/app/distributed/coordination/cluster_coordinator.py
```

12. Comandos Útiles para Desarrollo

```
# Backend
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev

# Docker (desarrollo local)
docker-compose -f docker/docker-compose.local.yml up -d

# Tests
pytest tests/ -v

# Linting
black backend/
isort backend/
mypy backend/
flake8 backend/
```

13. Resumen de Archivos Críticos

Archivo	Descripción	Importancia
backend/app/main.py	Entry point de FastAPI	● Crítico
backend/app/config.py	Toda la configuración	● Crítico
docker/docker-compose.yml	Orquestación	● Crítico
docker/slave/Dockerfile	Imagen del Slave	● Crítico
docker/master/Dockerfile	Imagen del Master	● Crítico
docker/load-balancer/nginx.conf	Config Nginx	● Importante
shared/models/node.py	Modelo de nodo	● Importante
frontend/src/App.tsx	Componente raíz React	● Importante

Anterior: [02 Componentes Principales.md](#)

Siguiente sección: [../02 Vectorizacion Busqueda/README.md](#) para entender el sistema de vectorización.

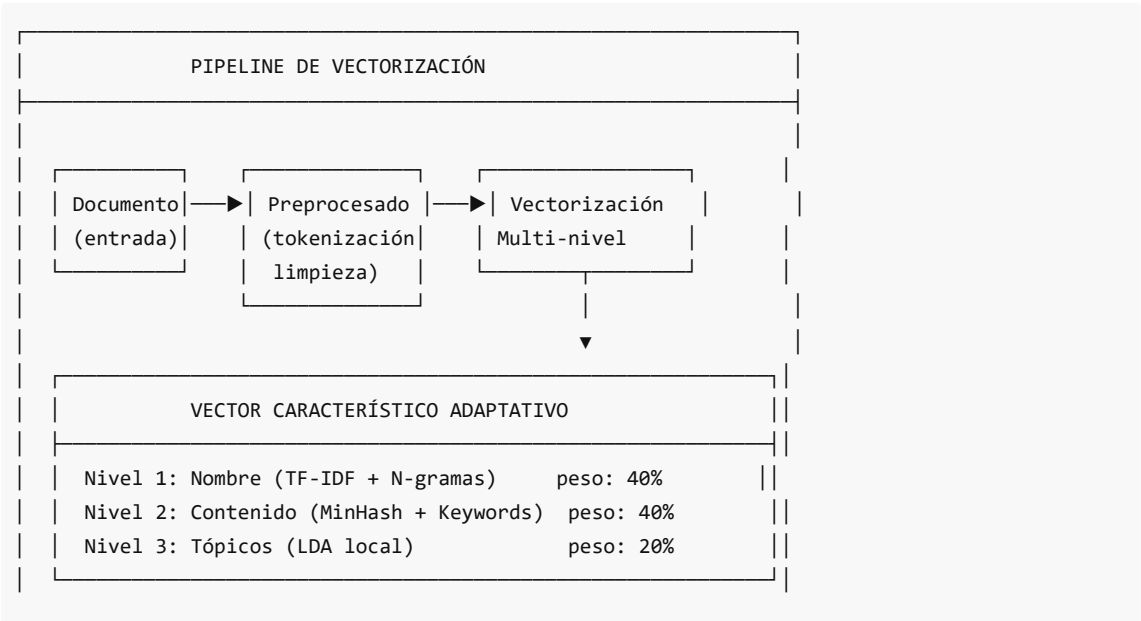
02 Vectorizacion Busqueda

Sistema de Vectorización Adaptativa

Búsqueda Semántica sin Embeddings Pre-entrenados

1. Visión General

DistriSearch implementa un **sistema de vectorización adaptativo** que NO depende de embeddings pre-entrenados de dimensión fija. Esta decisión de diseño resuelve problemas críticos con documentos extensos.



--	--

2. ¿Por Qué No Usar Embeddings Pre-entrenados?

Problema con Embeddings Fijos	Solución en DistriSearch
Dimensión fija (768, 1024)	Vectores sparse adaptativos
Pérdida de información en docs largos	Segmentación con MinHash
Modelos pesados (>500MB)	LDA ligero entrenado localmente
Vocabulario genérico	TF-IDF específico del corpus
Costosos en GPU	CPU-friendly

3. Componentes del Sistema

3.1 Estructura de Directorios

```
backend/app/core/vectorization/
├─ document_vectorizer.py    # 🟡 Clase principal
├─ tfidf_processor.py        # TF-IDF jerárquico
├─ minhash_signature.py      # MinHash + LSH
├─ lda_topics.py             # LDA local
├─ textrank_keywords.py      # Extracción de keywords
├─ char_ngrams.py            # N-gramas de caracteres
└─ __init__.py
```

3.2 Clase Principal: DocumentVectorizer

```
# backend/app/core/vectorization/document_vectorizer.py
class DocumentVectorizer:
    """
    Vectorizador Adaptativo de Documentos.

    Computa vectores multi-nivel:
    - Nivel 1: Vector del nombre (TF-IDF + char n-grams + categoría)
    - Nivel 2: Vector de contenido (MinHash + TextRank + LDA)
    - Nivel 3: Features estructurales (extensión, tamaño, patrones)
    """

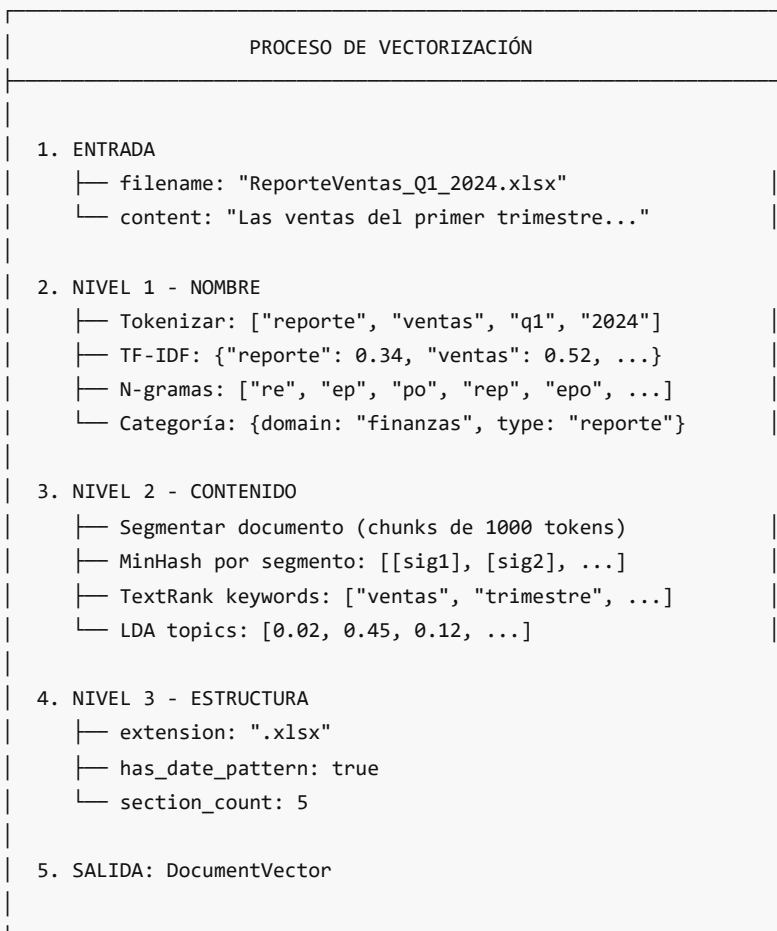
    def __init__(
        self,
        minhash_num_perm: int = 128,
        lda_num_topics: int = 20,
        tfidf_max_features: int = 5000,
        name_weight: float = 0.4,
        content_weight: float = 0.4,
```

```

        topic_weight: float = 0.2
    ):
        # Inicializa procesadores
        self.filename_tfidf = FilenameTFIDFProcessor()
        self.content_tfidf = TFIDFProcessor()
        self.minhash = MinHashSignature(num_perm=minhash_num_perm)
        self.lda = LDATopicModeler(num_topics=lda_num_topics)
        self.keyword_extractor = TextRankKeywordExtractor()
        self.char_ngram = CharNGramProcessor()

```

4. Flujo de Vectorización



5. Cálculo de Similitud

La similitud entre documentos se calcula como combinación ponderada:

$$\text{Similitud} = w_{\text{name}} \cdot S_{\text{name}} + w_{\text{content}} \cdot S_{\text{content}} + w_{\text{topic}} \cdot S_{\text{topic}}$$

Donde por defecto: $w_{\text{name}} = 0.4$, $w_{\text{content}} = 0.4$, $w_{\text{topic}} = 0.2$

```
def compute_similarity(self, vec1: DocumentVector, vec2: DocumentVector) -> float:
    name_sim = self._compute_name_similarity(vec1.name_vector, vec2.name_vector)
    content_sim = self._compute_content_similarity(vec1.content_vector, vec2.content_vector)
    topic_sim = self._compute_topic_similarity(vec1.content_vector, vec2.content_vector)

    return (
        self.name_weight * name_sim +
        self.content_weight * content_sim +
        self.topic_weight * topic_sim
    )
```

6. Adaptación por Tipo de Documento

El sistema ajusta pesos según el tipo de documento:

Tipo de Archivo	w_{name}	w_{content}	w_{topic}
Binario (.exe, .zip)	0.80	0.00	0.20
Texto (.txt, .md)	0.30	0.50	0.20
Documento (.pdf, .docx)	0.40	0.40	0.20
Código (.py, .js)	0.35	0.45	0.20

7. Índice de Contenidos

Archivo	Descripción
01 TF IDF Jerarquico.md	TF-IDF con pesos por nivel
02 MinHash LSH.md	Similitud Jaccard eficiente
03 Vector Caracteristico Adaptativo.md	Estructura del vector multi-nivel
04 LDA Topicos Locales.md	Modelado de tópicos local

Anterior: [../01 Arquitectura General/README.md](#)

Siguiente: [01 TF IDF Jerarquico.md](#)

TF-IDF Jerárquico

Representación Vectorial en Tres Niveles

1. ¿Qué es TF-IDF?

TF-IDF (Term Frequency - Inverse Document Frequency) es una técnica estadística para evaluar la importancia de una palabra en un documento dentro de un corpus.

1.1 Fórmulas Matemáticas

Term Frequency (TF) - Frecuencia normalizada del término en el documento:

$$TF(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Donde:

- $f_{t,d}$ = número de veces que el término t aparece en el documento d
- $\sum_{t' \in d} f_{t',d}$ = total de términos en el documento

Inverse Document Frequency (IDF) - Penaliza términos muy comunes:

$$IDF(t) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

Donde:

- N = número total de documentos en el corpus
- $|\{d \in D : t \in d\}|$ = documentos que contienen el término t

TF-IDF combinado:

$$TFIDF(t,d) = TF(t,d) \times IDF(t)$$

2. ¿Por Qué TF-IDF Jerárquico?

En DistriSearch, aplicamos TF-IDF en **tres niveles** con diferentes propósitos:

TF-IDF JERÁRQUICO
NIVEL 1: NOMBRE DEL ARCHIVO (Alta Prioridad)
├ TF-IDF sobre tokens del nombre
├ N-gramas de caracteres (2,3,4)
└ Peso: 40% en similaridad final
NIVEL 2: CONTENIDO (Segmentado)
├ TF-IDF sobre contenido completo
├ Keywords extraídos con TextRank
└ Peso: 40% en similaridad final
NIVEL 3: METADATOS ESTRUCTURALES
├ Extensión del archivo
├ Patrones detectados (fechas, versiones)
└ Contribuye a categorización

3. Implementación: TFIDFProcessor

```
# backend/app/core/vectorization/tfidf_processor.py

class TFIDFProcessor:
    """
    Procesador TF-IDF para vectorización de texto.
    """

    def __init__(
        self,
        max_features: int = 5000,
        min_df: int = 1,
        max_df: float = 0.95,
        ngram_range: Tuple[int, int] = (1, 2),
        use_idf: bool = True,
        smooth_idf: bool = True,
        sublinear_tf: bool = True, # Usa 1 + log(tf)
        lowercase: bool = True
    ):
        self._vectorizer = TfidfVectorizer(
            max_features=max_features,
            min_df=min_df,
            max_df=max_df,
            ngram_range=ngram_range,
            use_idf=use_idf,
            smooth_idf=smooth_idf,
            sublinear_tf=sublinear_tf,
            lowercase=lowercase,
            token_pattern=r'(?u)\b\w+\b',
            strip_accents='unicode'
        )
```

3.1 Sublinear TF

Cuando `sublinear_tf=True`, se aplica escalado logarítmico:

$$TF_{\text{sublinear}}(t,d) = 1 + \log(f_{\text{t,d}})$$

Esto evita que términos muy frecuentes dominen el vector.

4. Nivel 1: Vector del Nombre

El nombre del archivo tiene **alta prioridad** porque:

- Siempre está disponible (incluso para archivos binarios)
- Contiene semántica clave elegida por el usuario
- Es corto y específico

4.1 Procesamiento del Nombre

```
# backend/app/core/vectorization/tfidf_processor.py
```



```

class FilenameTFIDFProcessor(TFIDFProcessor):
    """TF-IDF especializado para nombres de archivo."""

    def preprocess_filename(self, filename: str) -> str:
        """
        Preprocesa nombre de archivo.

        Ejemplo: "ReporteVentas_Q1_2024.xlsx"
        Resultado: "reporte ventas q1 2024 reporteventas"
        """
        # Remover extensión
        name = filename.rsplit('.', 1)[0] if '.' in filename else filename

        # Separar por caracteres especiales
        tokens = re.split(r'[_\-\s\.]+', name)

        # Agregar nombre sin separadores (para coincidencias parciales)
        tokens.append(re.sub(r'[_\-\s\.]+', '', name.lower()))

        return ' '.join(tokens).lower()

```

4.2 N-gramas de Caracteres

Capturan errores tipográficos y variantes:

```

# backend/app/core/vectorization/char_ngrams.py

class CharNGramProcessor:
    """Procesador de N-gramas de caracteres."""

    def __init__(self, ngram_sizes: List[int] = [2, 3, 4]):
        self.ngram_sizes = ngram_sizes

    def get_ngrams(self, text: str) -> Set[str]:
        """
        Ejemplo: "ventas" con n=3
        Resultado: {"^ve", "ven", "ent", "nta", "tas", "as$"}
        """
        text = f'^{text.lower()}$' # Marcadores de inicio/fin
        ngrams = set()

        for n in self.ngram_sizes:
            ngrams.update(
                text[i:i+n] for i in range(len(text) - n + 1)
            )

        return ngrams

```

Ejemplo de similitud con n-gramas:

"ventas" vs "bentas" (error tipográfico)

N-gramas de "ventas": {^ve, ven, ent, nta, tas, as\$, ^ven, vent, enta, ntas, tas\$, ...}

N-gramas de "bentas": {^be, ben, ent, nta, tas, as\$, ^ben, bent, enta, ntas, tas\$, ...}

Intersección: {ent, nta, tas, as\$, enta, ntas, tas\$, ...}

Jaccard = $\frac{|\text{intersección}|}{|\text{unión}|} \approx 0.75$ (alta similaridad)

5. Nivel 2: Vector de Contenido

Para documentos extensos, aplicamos TF-IDF al contenido completo:

```
def fit(self, documents: List[str]) -> 'TFIDFProcessor':
    """
    Entrena el vectorizador en un corpus.
    """
    valid_docs = [doc for doc in documents if doc and doc.strip()]

    self._vectorizer.fit(valid_docs)
    self._vocabulary = self._vectorizer.vocabulary_
    self._idf_values = self._vectorizer.idf_
    self._is_fitted = True

    return self

def get_tfidf_dict(self, text: str) -> Dict[str, float]:
    """
    Obtiene pesos TF-IDF como diccionario.

    Ejemplo salida:
    {"ventas": 0.52, "trimestre": 0.48, "crecimiento": 0.35}
    """
    vector = self._vectorizer.transform([text])
    feature_names = self._vectorizer.get_feature_names_out()

    # Convertir sparse matrix a diccionario
    result = {}
    for idx in vector.nonzero()[1]:
        result[feature_names[idx]] = vector[0, idx]

    return result
```

6. Nivel 3: Metadatos Estructurales

Extraemos patrones del nombre y estructura:

```
# backend/app/core/vectorization/document_vectorizer.py
```

```
def _compute_structural_features(self, filename: str, ...) -> StructuralFeatures:
    """Extrae features estructurales."""

    return StructuralFeatures(
        extension=filename.rsplit('.', 1)[1].lower(),
        name_length=len(filename),
        has_date_pattern=bool(re.search(r'\d{4}|Q[1-4]', filename)),
        has_version=bool(re.search(r'v\d+|version', filename, re.I)),
        section_count=len(re.findall(r'^#{1,6}', content, re.MULTILINE)),
        has_tables=bool(re.search(r'\|.*\|.*\|', content))
    )
```

7. Ventajas vs Embeddings de Dimensión Fija

Aspecto	Embeddings Fijos	TF-IDF Jerárquico
Dimensión	Fija (768, 1024)	Sparse, variable
Docs largos	Trunca o promedia	Preserva todo
Vocabulario	Pre-entrenado genérico	Específico del corpus
Memoria	~500MB por modelo	~50MB vocabulario
Interpretabilidad	Opaco	Claro (términos con pesos)
Actualización	Re-entrenamiento costoso	Incremental fácil

8. Ejemplo Completo

```
# Vectorización de "ReporteVentas_Q1_2024.xlsx"

# NIVEL 1: Nombre
name_tokens = ["reporte", "ventas", "q1", "2024"]
name_tfidf = {
    "reporte": 0.34,
    "ventas": 0.52,          # Alta importancia
    "q1": 0.71,              # Muy específico (bajo DF)
    "2024": 0.28
}

char_ngrams = {"rep", "epo", "por", "ort", "rte", "ven", "ent", "nta", "tas", ...}
category = {"domain": "finanzas", "type": "reporte", "temporal": "Q1-2024"}

# NIVEL 2: Contenido (si extraíble)
content_tfidf = {
    "ventas": 0.45,
    "trimestre": 0.38,
    "crecimiento": 0.32,
    "margen": 0.28,
    ...
}
```

```

}
keywords_textrank = ["ventas", "trimestre", "crecimiento", "margen", "operativo"]

# NIVEL 3: Estructura
structural = {
    "extension": "xlsx",
    "has_date_pattern": True,
    "has_version": False
}

```

9. Similitud con Coseno

Para comparar vectores TF-IDF usamos **similitud del coseno**:

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| \times |\vec{B}|}$$

```

def compute_similarity_from_dicts(
    self,
    dict1: Dict[str, float],
    dict2: Dict[str, float]
) -> float:
    """Coseno entre dos vectores sparse (diccionarios)."""

    common_terms = set(dict1.keys()) & set(dict2.keys())

    if not common_terms:
        return 0.0

    dot_product = sum(dict1[t] * dict2[t] for t in common_terms)
    norm1 = math.sqrt(sum(v**2 for v in dict1.values()))
    norm2 = math.sqrt(sum(v**2 for v in dict2.values()))

    return dot_product / (norm1 * norm2)

```

Anterior: [README.md](#)

Siguiente: [02 MinHash LSH.md](#)

MinHash y Locality-Sensitive Hashing (LSH)

Estimación Eficiente de Similitud Jaccard

1. El Problema de la Similitud

Comparar dos documentos usando sus conjuntos de tokens es costoso:

- Documento A: 10,000 tokens únicos
- Documento B: 12,000 tokens únicos

- Calcular Jaccard directamente: $O(n + m)$ por par

Con millones de documentos, esto es **prohibitivo**.

2. Similitud de Jaccard

La **similitud de Jaccard** mide el solapamiento entre dos conjuntos:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Ejemplo:

$A = \{\text{ventas, trimestre, crecimiento, margen}\}$

$B = \{\text{ventas, año, crecimiento, ingresos}\}$

Intersección = $\{\text{ventas, crecimiento}\} \rightarrow |A \cap B| = 2$

Unión = $\{\text{ventas, trimestre, crecimiento, margen, año, ingresos}\} \rightarrow |A \cup B| = 6$

$$J(A, B) = 2/6 = 0.33$$

3. MinHash: La Solución

MinHash permite **estimar** la similitud de Jaccard usando firmas compactas.

3.1 Intuición

Si tomamos una función hash h y aplicamos a todos los elementos de un conjunto, el **mínimo** hash tiene la misma probabilidad de provenir de cualquier elemento.

Propiedad clave:

$$P(\min(h(A)) = \min(h(B))) = J(A, B)$$

La probabilidad de que el mínimo hash coincida es exactamente la similitud de Jaccard.

3.2 Firma MinHash

Usamos **múltiples funciones hash** $(h_1, h_2, \dots, h_k)$ para crear una "firma":

$$\text{Signature}(A) = [\min(h_1(A)), \min(h_2(A)), \dots, \min(h_k(A))]$$

FIRMA MINHASH

Documento: {ventas, trimestre, crecimiento}

$h_1(\text{ventas})=42, h_1(\text{trimestre})=78, h_1(\text{crecimiento})=15$
 $\rightarrow \min_1 = 15$

$h_2(\text{ventas})=91, h_2(\text{trimestre})=23, h_2(\text{crecimiento})=67$
 $\rightarrow \min_2 = 23$

...

```
Firma = [15, 23, 56, 8, 102, ...] (128 valores)
```

4. Implementación en DistriSearch

```
# backend/app/core/vectorization/minhash_signature.py

from datasketch import MinHash, MinHashLSH

class MinHashSignature:
    """
    Generador de firmas MinHash para estimación de similaridad.
    """

    def __init__(self, num_perm: int = 128, seed: int = 42):
        """
        Args:
            num_perm: Número de permutaciones (tamaño de firma)
            seed: Semilla para reproducibilidad
        """
        self.num_perm = num_perm
        self.seed = seed

    def compute_signature(self, tokens: Set[str]) -> List[int]:
        """
        Calcula firma MinHash para un conjunto de tokens.
        """
        if not tokens:
            return [0] * self.num_perm

        minhash = MinHash(num_perm=self.num_perm, seed=self.seed)

        for token in tokens:
            minhash.update(token.encode('utf-8'))

        return list(minhash.hashvalues)

    def estimate_similarity(self, sig1: List[int], sig2: List[int]) -> float:
        """
        Estima similaridad Jaccard desde dos firmas.

        Similaridad = (coincidencias) / (total de hashes)
        """
        if len(sig1) != len(sig2):
            raise ValueError("Firmas deben tener mismo tamaño")

        matches = sum(1 for h1, h2 in zip(sig1, sig2) if h1 == h2)
```

```
return matches / len(sig1)
```

4.1 Error de Estimación

El error estándar de la estimación es:

$$\sigma = \sqrt{\frac{J(1-J)}{k}}$$

Con $k = 128$ permutaciones y $J = 0.5$:

$$\sigma = \sqrt{\frac{0.5 \times 0.5}{128}} \approx 0.044$$

Error típico: $\pm 4.4\%$ - suficientemente preciso para búsqueda.

5. LSH: Locality-Sensitive Hashing

LSH acelera la búsqueda de documentos similares **sin comparar todos los pares**.

5.1 Concepto

Dividimos la firma MinHash en **bandas**. Dos documentos son candidatos si coinciden en **al menos una banda completa**.

LSH CON BANDAS
Firma de 128 valores $\rightarrow$ 16 bandas de 8 valores cada una
Doc A: [15,23,56,8 102,45,67,89 12,34,56,78 ...]
Doc B: [15,23,56,8 102,45,67,89 99,11,22,33 ...]
=====
Banda 1: COINCIDE $\leftarrow$ Son candidatos
Doc C: [99,88,77,66 55,44,33,22 11,00,99,88 ...]
Doc A vs C: Ninguna banda coincide $\leftarrow$ NO son candidatos

5.2 Probabilidad de Ser Candidatos

Con b bandas y r filas por banda:

$$P(\text{candidato}) = 1 - (1 - J^r)^b$$

Ejemplo con 16 bandas, 8 filas:

Jaccard Real	P(candidato)
0.2	0.02%
0.4	2.8%

0.5	18%
0.6	57%
0.8	99.7%

6. Implementación LSH en DistriSearch

```
# backend/app/core/vectorization/minhash_signature.py

class MinHashLSHIndex:
    """
    Índice LSH para búsqueda aproximada de vecinos cercanos.
    """

    def __init__(
        self,
        threshold: float = 0.5,
        num_perm: int = 128,
        seed: int = 42
    ):
        """
        Args:
            threshold: Umbral de similaridad Jaccard para matching
            num_perm: Número de permutaciones
        """
        self.threshold = threshold
        self.num_perm = num_perm

        self._lsh = MinHashLSH(threshold=threshold, num_perm=num_perm)
        self._minhashes: dict = {}
        self._signature_gen = MinHashSignature(num_perm=num_perm, seed=seed)

    def add(self, doc_id: str, tokens: Set[str]):
        """Agrega documento al índice."""
        minhash = self._signature_gen.create_minhash(tokens)

        self._lsh.insert(doc_id, minhash)
        self._minhashes[doc_id] = minhash

    def query(self, tokens: Set[str]) -> List[str]:
        """
        Busca documentos similares.

        Retorna IDs de documentos candidatos.
        """
        minhash = self._signature_gen.create_minhash(tokens)
        return list(self._lsh.query(minhash))

    def query_with_scores(
```



```

        self,
        tokens: Set[str],
        top_k: int = 10
    ) -> List[Tuple[str, float]]:
        """
        Busca documentos similares con scores.
        """
        query_minhash = self._signature_gen.create_minhash(tokens)
        candidates = self._lsh.query(query_minhash)

        # Calcular similaridad exacta para candidatos
        results = []
        for doc_id in candidates:
            doc_minhash = self._minhashes[doc_id]
            similarity = query_minhash.jaccard(doc_minhash)
            results.append((doc_id, similarity))

        # Ordenar por similaridad descendente
        results.sort(key=lambda x: x[1], reverse=True)

        return results[:top_k]

```

7. Aplicación: Contenido Segmentado

Para documentos largos, aplicamos MinHash por **segmentos**:

```

# backend/app/core/vectorization/minhash_signature.py

class ContentMinHasher:
    """MinHash para contenido segmentado."""

    def __init__(self, num_perm: int = 128, segment_size: int = 1000):
        self.num_perm = num_perm
        self.segment_size = segment_size
        self._signature_gen = MinHashSignature(num_perm=num_perm)

    def compute_segmented_signatures(self, content: str) -> List[List[int]]:
        """
        Computa firmas MinHash por segmento.

        Documento largo → [firma_seg1, firma_seg2, ...]
        """
        tokens = self._tokenize(content)
        segments = self._segment_tokens(tokens)

        signatures = []
        for segment in segments:
            sig = self._signature_gen.compute_signature(set(segment))
            signatures.append(sig)

```

```

        return signatures

def compute_combined_similarity(
    self,
    sigs1: List[List[int]],
    sigs2: List[List[int]]
) -> float:
    """
    Similaridad combinada entre documentos segmentados.

    Usa el máximo de todas las comparaciones segmento-a-segmento.
    """
    if not sigs1 or not sigs2:
        return 0.0

    max_sim = 0.0
    for s1 in sigs1:
        for s2 in sigs2:
            sim = self._signature_gen.estimate_similarity(s1, s2)
            max_sim = max(max_sim, sim)

    return max_sim

```

8. Ventajas para Búsqueda Distribuida

Aspecto	Beneficio en DistriSearch
Compresión	Documento → 128 enteros (512 bytes)
Comparación rápida	$O(k)$ en lugar de $O(n+m)$
Escalable	LSH reduce candidatos drásticamente
Distribuable	Firmas viajan fácilmente entre nodos
No usa hash para ubicación	Cumple restricción del proyecto

9. Flujo en Búsqueda

BÚSQUEDA CON MINHASH LSH

1. Query: "reporte ventas trimestral"
2. Tokenizar → {reporte, ventas, trimestral}
3. Calcular MinHash → [42, 78, 15, 91, 23, ...]
4. LSH Query → Candidatos: [doc_123, doc_456, doc_789]

(de 100,000 docs, solo 3 candidatos)

5. Calcular similaridad exacta para candidatos

6. Ranking final:

- doc_123: 0.85
- doc_456: 0.72
- doc_789: 0.61

10. Comparación de Complejidad

Operación	Sin LSH	Con LSH
Indexar N docs	$O(N)$	$O(N)$
Buscar en N docs	$O(N)$	$O(1)$ promedio
Comparar par	$O(\text{tokens})$	$O(k) = O(128)$
Espacio por doc	$O(\text{tokens})$	$O(k) = O(128)$

Anterior: [01 TF IDF Jerarquico.md](#)

Siguiente: [03 Vector Característico Adaptativo.md](#)

Vector Característico Adaptativo

Representación Multi-Nivel de Documentos

1. Concepto General

El **AdaptiveDocumentVector** es la estructura central que representa un documento en DistriSearch. Se adapta según el tipo y contenido del documento.

ADAPTIVE DOCUMENT VECTOR

NIVEL 1: NameVector

- tokens_tfidf: Dict[str, float]
- char_ngrams_signature: List[int]
- category: Dict[str, str]

NIVEL 2: ContentVector

```
| | |─ minhash_signatures: List[List[int]]
| | |─ keywords_textrank: List[str]
| | |─ topic_distribution: List[float]
```

```
| NIVEL 3: StructuralFeatures
```

```
| |─ extension: str
| |─ name_length: int
| |─ has_date_pattern: bool
| |─ has_version: bool
```

```
| PESOS ADAPTATIVOS
```

```
| |─ name_weight: 0.4 (ajustable)
| |─ content_weight: 0.4 (ajustable)
| |─ topic_weight: 0.2 (ajustable)
```

2. Definición en Código

```
# shared/models/document.py

from pydantic import BaseModel
from typing import Dict, List, Optional

class NameVector(BaseModel):
    """Vector del nombre del archivo."""
    tokens_tfidf: Dict[str, float] = {}
    char_ngrams_signature: List[int] = []
    category: Dict[str, str] = {}

class ContentVector(BaseModel):
    """Vector del contenido."""
    minhash_signatures: List[List[int]] = []
    keywords_textrank: List[str] = []
    topic_distribution: List[float] = []

class StructuralFeatures(BaseModel):
    """Características estructurales."""
    extension: str = ""
    name_length: int = 0
    file_size: int = 0
    has_date_pattern: bool = False
    has_version: bool = False
    section_count: int = 0
    has_tables: bool = False

class DocumentVector(BaseModel):
```

```

"""Vector completo del documento."""
name_vector: NameVector
content_vector: ContentVector
structural_features: StructuralFeatures

# Pesos adaptativos
name_weight: float = 0.4
content_weight: float = 0.4
topic_weight: float = 0.2

```

3. Cálculo de Similitud Multi-Nivel

La similitud total se calcula combinando los tres niveles:

$$S_{\text{total}} = w_{\text{name}} \cdot S_{\text{name}} + w_{\text{content}} \cdot S_{\text{content}} + w_{\text{topic}} \cdot S_{\text{topic}}$$

3.1 Implementación

```

# backend/app/core/vectorization/document_vectorizer.py

def compute_similarity(
    self,
    vec1: DocumentVector,
    vec2: DocumentVector
) -> float:
    """
    Calcula similitud ponderada entre dos vectores.
    """
    # Similitud del nombre
    name_sim = self._compute_name_similarity(
        vec1.name_vector, vec2.name_vector
    )

    # Similitud del contenido
    content_sim = self._compute_content_similarity(
        vec1.content_vector, vec2.content_vector
    )

    # Similitud de tópicos
    topic_sim = self._compute_topic_similarity(
        vec1.content_vector, vec2.content_vector
    )

    # Combinación ponderada
    total = (
        self.name_weight * name_sim +
        self.content_weight * content_sim +
        self.topic_weight * topic_sim
    )

```

```
return total
```

4. Similitud del Nombre (name_sim)

Combina TF-IDF coseno + n-gramas + bonus de categoría:

```
def _compute_name_similarity(
    self,
    nv1: NameVector,
    nv2: NameVector
) -> float:
    """
    Similaridad entre vectores de nombre.
    """

    # 1. Coseno TF-IDF
    tfidf_sim = self.content_tfidf.compute_similarity_from_dicts(
        nv1.tokens_tfidf,
        nv2.tokens_tfidf
    )

    # 2. Similitud de n-gramas de caracteres
    ngram_sim = self.char_ngram.estimate_similarity_from_signatures(
        nv1.char_ngrams_signature,
        nv2.char_ngrams_signature
    )

    # 3. Bonus por categoría coincidente
    category_bonus = 0.0
    if nv1.category and nv2.category:
        if nv1.category.get('domain') == nv2.category.get('domain'):
            category_bonus += 0.1
        if nv1.category.get('type') == nv2.category.get('type'):
            category_bonus += 0.05

    # Combinar
    base_sim = 0.6 * tfidf_sim + 0.4 * ngram_sim

    return min(1.0, base_sim + category_bonus)
```

Fórmula:

$$S_{\text{name}} = \min(1.0, \underbrace{0.6 \cdot \cos(\text{TF}_1, \text{TF}_2)}_{\text{TF-IDF}} + \underbrace{0.4 \cdot J(\text{ng}_1, \text{ng}_2)}_{\text{n-gramas}} + \underbrace{\text{bonus}}_{\text{categoría}})$$

5. Similitud del Contenido (content_sim)

Usa MinHash Jaccard sobre segmentos:

```
def _compute_content_similarity(
    self,
    cv1: ContentVector,
    cv2: ContentVector
) -> float:
    """
    Similaridad entre vectores de contenido.
    """
    if not cv1.minhash_signatures or not cv2.minhash_signatures:
        return 0.0

    # Usar el máximo entre todos los pares de segmentos
    return self.content_minhasher.compute_combined_similarity(
        cv1.minhash_signatures,
        cv2.minhash_signatures
    )
```

Estrategia de segmentos:

Doc A (largo): [seg1, seg2, seg3]

Doc B (corto): [seg1]

Comparaciones:

A.seg1 vs B.seg1 → 0.45

A.seg2 vs B.seg1 → 0.72 ← Máximo

A.seg3 vs B.seg1 → 0.38

content_sim = 0.72

6. Similaridad de Tópicos (topic_sim)

Usa **divergencia de Jensen-Shannon** entre distribuciones LDA:

$$JS(P \mid Q) = \frac{1}{2} KL(P \mid M) + \frac{1}{2} KL(Q \mid M)$$

Donde $M = \frac{1}{2}(P + Q)$ y KL es la divergencia de Kullback-Leibler.

```
def _compute_topic_similarity(
    self,
    cv1: ContentVector,
    cv2: ContentVector
) -> float:
    """
    Similaridad de tópicos usando Jensen-Shannon.
    """
    if not cv1.topic_distribution or not cv2.topic_distribution:
        return 0.0

    return self.lda.compute_topic_similarity(
        cv1.topic_distribution,
```

```

        cv2.topic_distribution
    )

# En lda_topics.py
def compute_topic_similarity(
    self,
    dist1: List[float],
    dist2: List[float]
) -> float:
    """
    Similaridad = 1 - JS_divergence
    """
    p = np.array(dist1)
    q = np.array(dist2)

    # Evitar log(0)
    p = np.clip(p, 1e-10, 1.0)
    q = np.clip(q, 1e-10, 1.0)

    # Normalizar
    p = p / p.sum()
    q = q / q.sum()

    # Jensen-Shannon
    m = 0.5 * (p + q)
    js_div = 0.5 * np.sum(p * np.log(p / m)) + 0.5 * np.sum(q * np.log(q / m))

    # Convertir a similaridad [0, 1]
    return 1.0 - np.sqrt(js_div)

```

7. Adaptación por Tipo de Documento

El sistema **ajusta los pesos automáticamente** según el tipo de archivo:

```

# backend/app/core/vectorization/document_vectorizer.py

def _get_adaptive_weights(
    self,
    filename: str,
    content: Optional[str]
) -> Tuple[float, float, float]:
    """
    Calcula pesos adaptativos según tipo de documento.
    """
    extension = filename.rsplit('.', 1)[1].lower() if '.' in filename else ''

    # Archivos binarios: peso alto en nombre
    binary_extensions = {'exe', 'dll', 'zip', 'rar', 'jpg', 'png', 'mp3', 'mp4'}
    if extension in binary_extensions or not content:
        return (0.80, 0.00, 0.20) # name, content, topic

```



```

# Código fuente: balance con más contenido
code_extensions = {'py', 'js', 'ts', 'java', 'cpp', 'go', 'rs'}
if extension in code_extensions:
    return (0.35, 0.45, 0.20)

# Documentos de texto
text_extensions = {'txt', 'md', 'rst'}
if extension in text_extensions:
    return (0.30, 0.50, 0.20)

# Por defecto (PDF, DOCX, etc.)
return (0.40, 0.40, 0.20)

```

7.1 Tabla de Pesos por Tipo

Categoría	Extensiones	\$w_{\text{name}}\$	\$w_{\text{content}}\$	\$w_{\text{topic}}\$
Binario	exe, zip, jpg, mp3	0.80	0.00	0.20
Código	py, js, java, cpp	0.35	0.45	0.20
Texto plano	txt, md, rst	0.30	0.50	0.20
Documento	pdf, docx, xlsx	0.40	0.40	0.20

8. Inferencia de Categoría

La categoría se infiere del nombre del archivo:

```

# backend/app/core/vectorization/char_ngrams.py

def infer_category(filename: str) -> Dict[str, str]:
    """
    Infiere categoría del documento desde el nombre.
    """
    name_lower = filename.lower()
    category = {}

    # Inferir dominio
    domain_patterns = {
        'finanzas': ['ventas', 'factura', 'balance', 'presupuesto', 'fiscal'],
        'rrhh': ['nomina', 'empleado', 'contrato', 'vacaciones', 'personal'],
        'legal': ['contrato', 'acuerdo', 'demanda', 'sentencia', 'ley'],
        'tecnico': ['manual', 'guia', 'especificacion', 'requisito', 'api'],
        'marketing': ['campana', 'publicidad', 'leads', 'conversion', 'roi']
    }

    for domain, keywords in domain_patterns.items():
        if any(kw in name_lower for kw in keywords):
            category['domain'] = domain

```

```

        break

# Inferir tipo de documento
type_patterns = {
    'reporte': ['reporte', 'report', 'informe'],
    'factura': ['factura', 'invoice', 'recibo'],
    'contrato': ['contrato', 'contract', 'acuerdo'],
    'manual': ['manual', 'guia', 'guide', 'tutorial'],
    'presentacion': ['presentacion', 'ppt', 'slides']
}

for doc_type, keywords in type_patterns.items():
    if any(kw in name_lower for kw in keywords):
        category['type'] = doc_type
        break

# Detectar patrón temporal
import re
if re.search(r'Q[1-4][-_]?\\d{4}|20\\d{2}[-_]?Q[1-4]', filename):
    match = re.search(r'(Q[1-4])[-_]?(\\d{4})|(\\d{4})[-_]?(Q[1-4])', filename)
    if match:
        quarter = match.group(1) or match.group(4)
        year = match.group(2) or match.group(3)
        category['temporal'] = f"{quarter}-{year}"

return category

```

9. Ejemplo Completo de Vectorización

```

# Documento: "ReporteVentas_Q1_2024.xlsx"
# Contenido: "Las ventas del primer trimestre mostraron crecimiento..."

vector = vectorizer.vectorize(
    filename="ReporteVentas_Q1_2024.xlsx",
    content="Las ventas del primer trimestre mostraron crecimiento..."
)

# Resultado:
DocumentVector(
  name_vector=NameVector(
    tokens_tfidf={
      "reporte": 0.34,
      "ventas": 0.52,
      "q1": 0.71,
      "2024": 0.28
    },
    char_ngrams_signature=[4521, 8723, 1234, ...], # 128 valores
    category={
      "domain": "finanzas",
      "type": "reporte",

```

```

        "temporal": "Q1-2024"
    }
),
content_vector=ContentVector(
    minhash_signatures=[[2341, 5678, ...], [3456, 7890, ...]],
    keywords_textrank=["ventas", "trimestre", "crecimiento"],
    topic_distribution=[0.02, 0.01, 0.45, 0.03, ...] # 20 tópicos
),
structural_features=StructuralFeatures(
    extension="xlsx",
    name_length=24,
    has_date_pattern=True,
    has_version=False
),
name_weight=0.40,
content_weight=0.40,
topic_weight=0.20
)

```

10. Desglose de Similitud

Para debugging, se puede obtener el desglose completo:

```

breakdown = vectorizer.get_similarity_breakdown(vec1, vec2)

# Resultado:
{
    'name_similarity': 0.78,
    'content_similarity': 0.65,
    'topic_similarity': 0.82,
    'total_similarity': 0.73, # 0.4*0.78 + 0.4*0.65 + 0.2*0.82
    'weights': {
        'name': 0.4,
        'content': 0.4,
        'topic': 0.2
    }
}

```

Anterior: [02 MinHash LSH.md](#)

Siguiente: [04 LDA Tópicos Locales.md](#)

LDA: Tópicos Locales

Modelado de Tópicos Entrenado en el Corpus del Cluster

1. ¿Qué es LDA?

Latent Dirichlet Allocation (LDA) es un modelo probabilístico generativo que descubre tópicos latentes en una colección de documentos.

MODELO GENERATIVO LDA

Suposición: Cada documento es una mezcla de tópicos
Cada tópico es una distribución sobre palabras

Ejemplo con 3 tópicos:

Documento "ReporteVentas_Q1.pdf"

- 45% Tópico 2 (Finanzas): ventas, ingresos, margen, ...
- 35% Tópico 7 (Temporal): trimestre, Q1, período, ...
- 20% Tópico 12 (General): reporte, análisis, datos, ...

Distribución de tópicos: [0, 0, 0.45, 0, 0, 0, 0, 0.35, ...]

2. ¿Por Qué Entrenamiento Local?

Modelo Pre-entrenado	LDA Local (DistriSearch)
Tópicos genéricos	Tópicos específicos del corpus
No captura jerga interna	Aprende vocabulario de la empresa
Tamaño fijo (genérico)	Número de tópicos configurable
Requiere descarga ~500MB	Entrenamiento ligero in-situ
Actualización costosa	Re-entrena incrementalmente

2.1 Ventajas del Entrenamiento Local

- Especificidad:** Los tópicos reflejan exactamente los temas del corpus
- Privacidad:** No envía datos a servicios externos
- Adaptabilidad:** Evolucionan con el corpus
- Eficiencia:** Modelo pequeño (~5MB)

3. Modelo Matemático

3.1 Proceso Generativo

Para cada documento d :

- Elegir distribución de tópicos: $\theta_d \sim \text{Dirichlet}(\alpha)$
- Para cada palabra w en el documento:
 - Elegir un tópico: $z \sim \text{Multinomial}(\theta_d)$
 - Elegir la palabra: $w \sim \text{Multinomial}(\phi_z)$

3.2 Parámetros

- α : Prior de Dirichlet para distribución documento-tópico
- β : Prior de Dirichlet para distribución tópico-palabra
- K : Número de tópicos (en DistriSearch: 20 por defecto)

3.3 Distribución de Tópicos

La distribución de tópicos para un documento d :

$$P(\text{tópico } k \mid d) = \frac{n_{d,k} + \alpha}{\sum_{k'} n_{d,k'} + K\alpha}$$

Donde $n_{d,k}$ es el número de palabras en d asignadas al tópico k .

4. Implementación en DistriSearch

```
# backend/app/core/vectorization/lda_topics.py

from gensim import corpora
from gensim.models import LdaModel, LdaMulticore

class LDATopicModeler:
    """
    Modelador de tópicos LDA entrenado localmente.
    """

    def __init__(
        self,
        num_topics: int = 20,
        min_word_length: int = 3,
        min_df: int = 5,
        max_df: float = 0.5,
        passes: int = 10,
        random_state: int = 42,
        use_multicore: bool = True
    ):
        """
        Args:
            num_topics: Número de tópicos a extraer
            min_df: Frecuencia mínima de documento para palabras
            max_df: Frecuencia máxima de documento (ratio)
            passes: Pasadas sobre el corpus
        """
        self.num_topics = num_topics
        self.min_df = min_df
        self.max_df = max_df
        self.passes = passes
        self.use_multicore = use_multicore

        self._model = None
```

```
self._dictionary = None
self._is_trained = False
```

4.1 Entrenamiento

```
def train(self, documents: List[str]) -> 'LDATopicModeler':
    """
    Entrena LDA en el corpus local.
    """
    # Preprocesar documentos
    processed_docs = [self._preprocess(doc) for doc in documents]
    processed_docs = [doc for doc in processed_docs if doc]

    # Ajustar número de tópicos si hay pocos documentos
    if len(processed_docs) < self.num_topics:
        self.num_topics = max(2, len(processed_docs) // 2)

    # Crear diccionario
    self._dictionary = corpora.Dictionary(processed_docs)

    # Filtrar extremos
    self._dictionary.filter_extremes(
        no_below=self.min_df,
        no_above=self.max_df
    )

    # Crear corpus BOW
    corpus = [self._dictionary.doc2bow(doc) for doc in processed_docs]

    # Entrenar LDA
    if self.use_multicore:
        self._model = LdaMulticore(
            corpus=corpus,
            id2word=self._dictionary,
            num_topics=self.num_topics,
            passes=self.passes,
            random_state=self.random_state,
            workers=2
        )
    else:
        self._model = LdaModel(
            corpus=corpus,
            id2word=self._dictionary,
            num_topics=self.num_topics,
            passes=self.passes,
            random_state=self.random_state
        )

    self._is_trained = True
    return self
```

5. Obtener Distribución de Tópicos

```
def get_topic_distribution(self, text: str) -> List[float]:
    """
    Obtiene distribución de tópicos para un documento.

    Retorna: Lista de probabilidades [p_0, p_1, ..., p_{K-1}]
    """
    if not self._is_trained:
        # Distribución uniforme si no está entrenado
        return [1.0 / self.num_topics] * self.num_topics

    # Preprocesar
    tokens = self._preprocess(text)

    if not tokens:
        return [1.0 / self.num_topics] * self.num_topics

    # Convertir a BOW
    bow = self._dictionary.doc2bow(tokens)

    if not bow:
        return [1.0 / self.num_topics] * self.num_topics

    # Obtener distribución
    topic_dist = self._model.get_document_topics(
        bow,
        minimum_probability=0.0
    )

    # Convertir a lista de tamaño fijo
    distribution = [0.0] * self.num_topics
    for topic_id, prob in topic_dist:
        if topic_id < self.num_topics:
            distribution[topic_id] = prob

    return distribution
```

5.1 Ejemplo de Distribución

```
# Documento sobre ventas
text = """
Las ventas del primer trimestre superaron las expectativas.
El margen operativo creció un 15% respecto al año anterior.
Los ingresos totales alcanzaron $2.5 millones.
"""

distribution = lda.get_topic_distribution(text)
```

```
# Resultado (20 tópicos):
# [0.02, 0.01, 0.45, 0.03, 0.02, 0.01, 0.08, 0.12, ...]
#           ^^^^^           ^^^^^
#           Tópico "Finanzas"       Tópico "Temporal"
```

6. Similitud de Tópicos: Jensen-Shannon

La **divergencia de Jensen-Shannon** mide qué tan diferentes son dos distribuciones:

$$JS(P \parallel Q) = \frac{1}{2} D_{KL}(P \parallel M) + \frac{1}{2} D_{KL}(Q \parallel M)$$

Donde:

- $M = \frac{1}{2}(P + Q)$ es la distribución promedio
- D_{KL} es la divergencia de Kullback-Leibler:

$$D_{KL}(P \parallel Q) = \sum_i P_i \log \frac{P_i}{Q_i}$$

6.1 Implementación

```
def compute_topic_similarity(
    self,
    dist1: List[float],
    dist2: List[float]
) -> float:
    """
    Similaridad = 1 - sqrt(JS_divergence)

    JS está acotada en [0, 1], por lo que similaridad también.
    """
    import numpy as np

    p = np.array(dist1)
    q = np.array(dist2)

    # Evitar log(0)
    p = np.clip(p, 1e-10, 1.0)
    q = np.clip(q, 1e-10, 1.0)

    # Normalizar (por si acaso)
    p = p / p.sum()
    q = q / q.sum()

    # Distribución promedio
    m = 0.5 * (p + q)

    # Jensen-Shannon divergence
    js_div = 0.5 * np.sum(p * np.log(p / m)) + 0.5 * np.sum(q * np.log(q / m))

    # Convertir a similaridad [0, 1]
```



```
# sqrt porque JS está en [0, log(2)] para distribuciones discretas
return 1.0 - np.sqrt(js_div / np.log(2))
```

6.2 Ejemplo de Similitud

Documento A (Finanzas): [0.02, 0.01, 0.45, 0.03, ...]
Documento B (Finanzas): [0.03, 0.02, 0.42, 0.05, ...]
Documento C (Legal): [0.01, 0.50, 0.02, 0.01, ...]

JS(A, B) = 0.03 → Similitud = 0.97 (muy similares)

JS(A, C) = 0.45 → Similitud = 0.33 (diferentes)

7. Visualización de Tópicos

```
def get_top_words(
    self,
    topic_id: int,
    n_words: int = 10
) -> List[Tuple[str, float]]:
    """
    Obtiene las palabras más representativas de un tópico.
    """
    if not self._is_trained or topic_id >= self.num_topics:
        return []

    return self._model.show_topic(topic_id, n_words)
```

7.1 Ejemplo de Tópicos Descubiertos

Tópico 0 (Tecnología):

software: 0.082, sistema: 0.065, datos: 0.058,
aplicacion: 0.045, servidor: 0.042, ...

Tópico 2 (Finanzas):

ventas: 0.095, ingresos: 0.078, margen: 0.062,
balance: 0.055, fiscal: 0.048, ...

Tópico 5 (RRHH):

empleado: 0.088, nomina: 0.072, contrato: 0.065,
vacaciones: 0.052, personal: 0.048, ...

Tópico 8 (Legal):

contrato: 0.092, clausula: 0.075, partes: 0.068,
obligacion: 0.055, demanda: 0.045, ...

8. Persistencia del Modelo

```

def save(self, path: str):
    """Guarda modelo entrenado."""
    import os
    os.makedirs(path, exist_ok=True)

    if self._model:
        self._model.save(os.path.join(path, 'lda.model'))
    if self._dictionary:
        self._dictionary.save(os.path.join(path, 'dictionary.dict'))

def load(self, path: str) -> 'LDATopicModeler':
    """Carga modelo entrenado."""
    import os

    model_path = os.path.join(path, 'lda.model')
    dict_path = os.path.join(path, 'dictionary.dict')

    if os.path.exists(model_path):
        self._model = LdaModel.load(model_path)
    if os.path.exists(dict_path):
        self._dictionary = corpora.Dictionary.load(dict_path)

    self._is_trained = True
    return self

```

9. Re-entrenamiento Incremental

Cuando llegan nuevos documentos:

```

def update(self, new_documents: List[str]):
    """
    Actualiza modelo con nuevos documentos.
    """
    processed = [self._preprocess(doc) for doc in new_documents]
    processed = [doc for doc in processed if doc]

    # Actualizar diccionario
    self._dictionary.add_documents(processed)

    # Crear corpus para nuevos documentos
    new_corpus = [self._dictionary.doc2bow(doc) for doc in processed]

    # Actualizar modelo (online learning)
    self._model.update(new_corpus)

```

10. Inferencia de Tópicos desde Nombre

Para archivos binarios sin contenido extraíble:

```

def infer_topics_from_name(
    self,
    filename_tokens: List[str]
) -> List[float]:
    """
    Infiere tópicos solo del nombre del archivo.

    Útil para archivos binarios (exe, zip, jpg, etc.)
    """
    if not self._is_trained or not filename_tokens:
        return [1.0 / self.num_topics] * self.num_topics

    # Expandir tokens del nombre con sinónimos conocidos
    expanded_tokens = self._expand_tokens(filename_tokens)

    # Obtener distribución
    bow = self._dictionary.doc2bow(expanded_tokens)

    if not bow:
        return [1.0 / self.num_topics] * self.num_topics

    topic_dist = self._model.get_document_topics(bow, minimum_probability=0.0)

    distribution = [0.0] * self.num_topics
    for topic_id, prob in topic_dist:
        distribution[topic_id] = prob

    return distribution

```

11. Flujo Completo

PIPELINE LDA EN DISTRISearch

1. ENTRENAMIENTO (una vez, al iniciar cluster)
 - ├ Recopilar todos los documentos del corpus
 - ├ Preprocesar (tokenizar, filtrar stopwords)
 - ├ Entrenar LDA con K=20 tópicos
 - └ Guardar modelo en disco
2. VECTORIZACIÓN (por documento)
 - ├ Cargar modelo entrenado
 - ├ Preprocesar contenido del documento
 - ├ Convertir a BOW
 - └ Obtener distribución de tópicos $[p_0, p_1, \dots, p_{19}]$
3. BÚSQUEDA
 - ├ Query → distribución de tópicos

- └─ Comparar con docs usando Jensen-Shannon
 - └─ Contribuye 20% a la similaridad total
4. ACTUALIZACIÓN (periódica)
- └─ Nuevos documentos llegan
 - └─ Actualización incremental del modelo

12. Resumen de Parámetros

Parámetro	Valor Default	Descripción
num_topics	20	Número de tópicos latentes
passes	10	Pasadas de entrenamiento
min_df	5	Frecuencia mínima de documento
max_df	0.5	Frecuencia máxima (50% de docs)
alpha	auto	Prior documento-tópico
eta	auto	Prior tópico-palabra

Anterior: [03 Vector Característico Adaptativo.md](#)

Siguiente sección: [../03 Particionamiento/README.md](#)

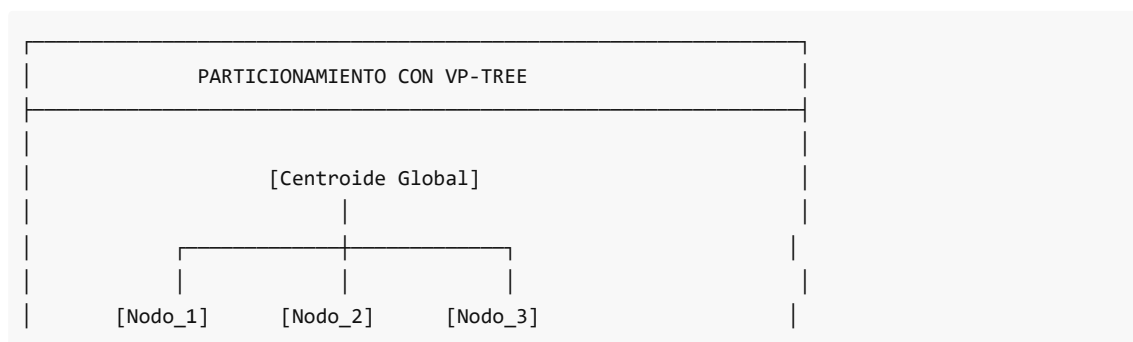
03 Particionamiento

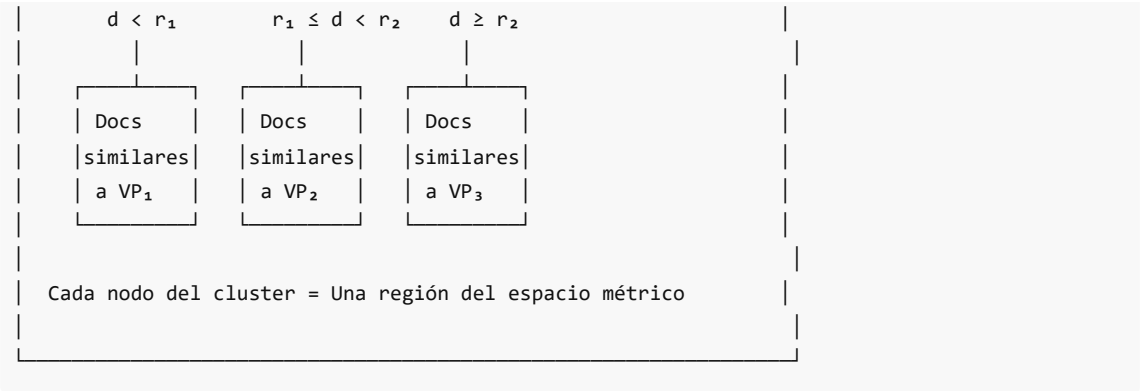
Sistema de Particionamiento

Particionamiento Semántico con VP-Tree Distribuido

1. Visión General

DistriSearch utiliza **VP-Trees (Vantage-Point Trees)** para particionar documentos basándose en **similitud semántica**, no en hash. Esto garantiza que documentos similares residan en el mismo nodo.





2. ¿Por Qué VP-Tree en Lugar de Hash?

Hash Tradicional	VP-Tree Semántico
Distribución aleatoria	Distribución por similitud
Documentos similares dispersos	Documentos similares agrupados
Búsqueda requiere todos los nodos	Búsqueda dirigida a nodos relevantes
No preserva localidad	Preserva localidad semántica

2.1 Ventaja Clave

Con VP-Tree, una búsqueda puede **podar nodos** que no contienen documentos relevantes:

Búsqueda: "reporte ventas Q1"

Hash tradicional:

├─ Consultar Nodo 1 ✓

├─ Consultar Nodo 2 ✓

└─ Consultar Nodo 3 ✓ → 3 consultas

VP-Tree:

├─ Calcular distancia a VPs

├─ Nodo 2 contiene docs similares

└─ Consultar solo Nodo 2 ✓ → 1 consulta

3. Arquitectura de Particionamiento

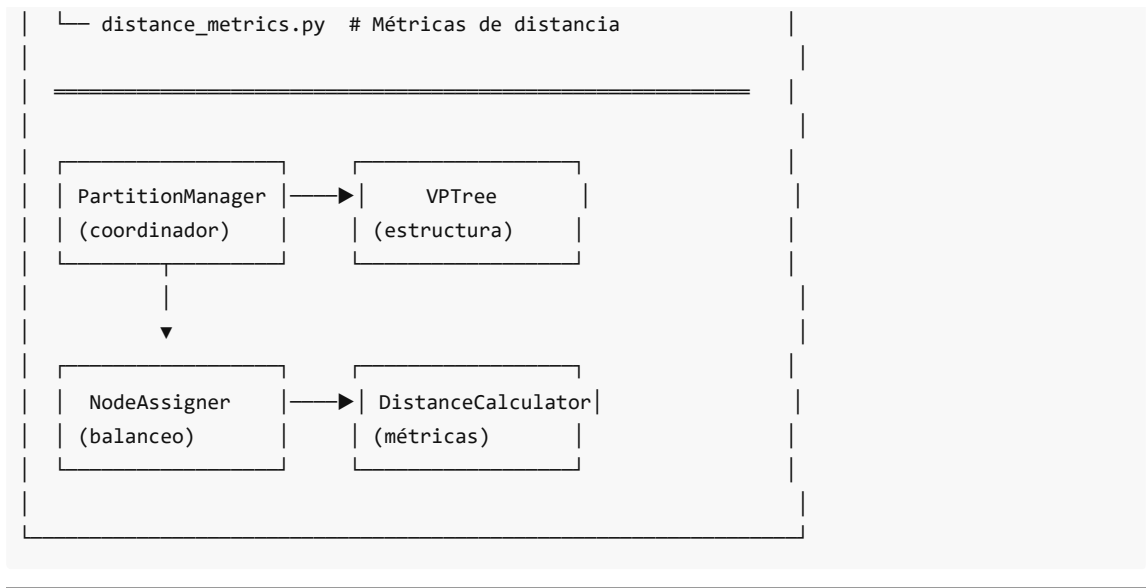
COMPONENTES DE PARTICIONAMIENTO

backend/app/core/partitioning/

├─ vp_tree.py # ● VP-Tree principal

├─ node_assignment.py # Asignación a nodos

└─ partition_manager.py # ● Coordinador de alto nivel



4. Fórmula de Distancia

La distancia entre documentos usa la combinación ponderada:

$$d(A, B) = 0.4 \cdot d_{\text{name}} + 0.4 \cdot d_{\text{content}} + 0.2 \cdot d_{\text{topic}}$$

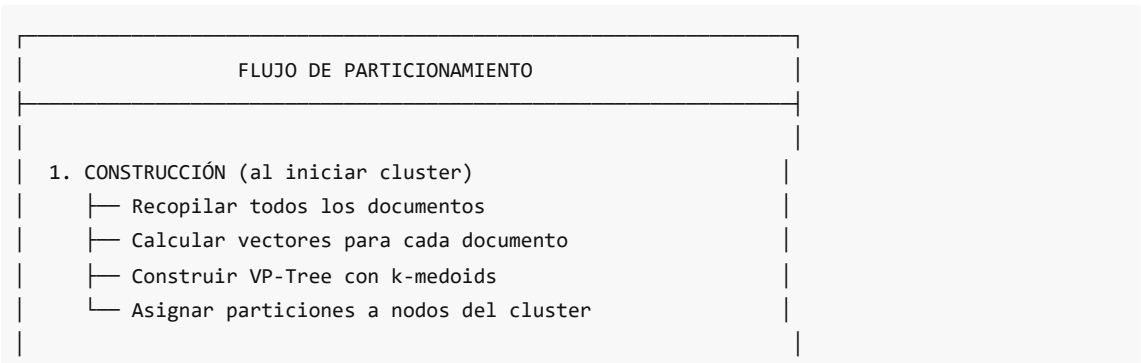
Donde:

- d_{name} = distancia coseno de TF-IDF del nombre
- d_{content} = distancia Jaccard de MinHash
- d_{topic} = divergencia Jensen-Shannon de LDA

```
# backend/app/core/partitioning/distance_metrics.py

@dataclass
class DistanceWeights:
    """Pesos para cálculo de distancia combinada."""
    name_weight: float = 0.4
    content_weight: float = 0.4
    topic_weight: float = 0.2
```

5. Flujo de Particionamiento



- ```

2. INSERCIÓN (nuevo documento)
├─ Calcular vector del documento
├─ Encontrar partición en VP-Tree
├─ Obtener nodo asignado a esa partición
└─ Almacenar documento en ese nodo

3. BÚSQUEDA
├─ Calcular vector de la query
├─ Identificar particiones relevantes
├─ Consultar solo nodos de esas particiones
└─ Agregar y ordenar resultados

```

6. Índice de Contenidos

| Archivo                                    | Descripción                           |
|--------------------------------------------|---------------------------------------|
| <a href="#">01 VP Tree Distribuido.md</a>  | Estructura del VP-Tree distribuido    |
| <a href="#">02 Algoritmo Asignacion.md</a> | Algoritmo de asignación de documentos |
| <a href="#">03 Vantage Points.md</a>       | Selección y gestión de vantage points |

7. Clase Principal: PartitionManager

```
backend/app/core/partitioning/partition_manager.py

class PartitionManager:
 """
 Gestor de particionamiento y enrutamiento de documentos.
 """

 def __init__(
 self,
 leaf_size: int = 50,
 replication_factor: int = 2,
 vp_selection: VantagePointSelection = VantagePointSelection.K_MEDOIDS
):
 self.vp_tree = VPTree(
 leaf_size=leaf_size,
 selection_strategy=vp_selection
)
 self.node_assigner = NodeAssigner(replication_factor=replication_factor)

 def route_document(self, document: Dict) -> RoutingResult:
 """Enruta documento al nodo apropiado."""
 partition_id = self.vp_tree.find_partition(document)
 primary_node = self._partition_assignments[partition_id]
 return RoutingResult(
```

```
document_id=document["id"],
partition_id=partition_id,
primary_node=primary_node
)
```

**Anterior:** [./02 Vectorizacion Busqueda/README.md](#)

**Siguiente:** [01 VP Tree Distribuido.md](#)

## VP-Tree Distribuido

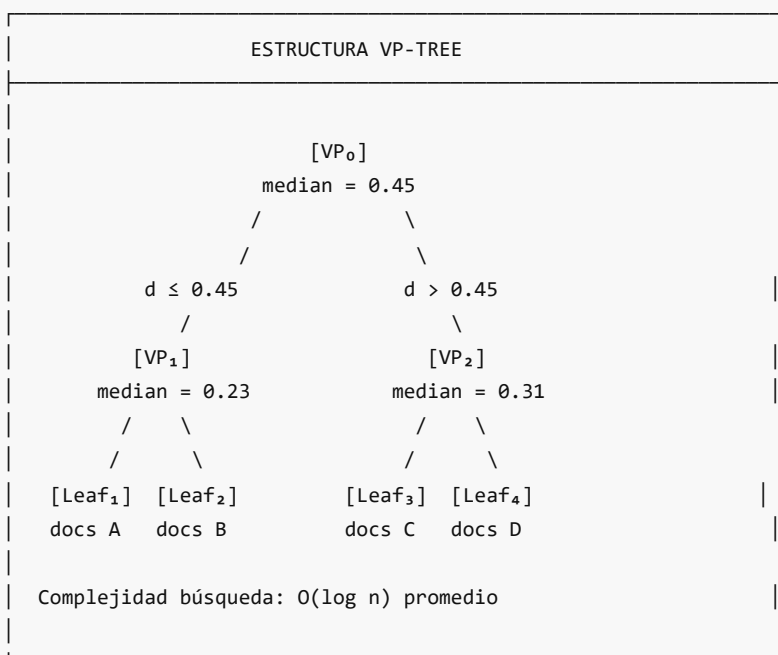
### Árboles de Vantage-Point para Espacios Métricos

#### 1. ¿Qué es un VP-Tree?

Un **Vantage-Point Tree (VP-Tree)** es una estructura de datos para organizar puntos en un espacio métrico, permitiendo búsquedas eficientes de vecinos cercanos.

##### 1.1 Concepto Básico

1. Se elige un punto como **vantage point (VP)** o punto de referencia
2. Se calcula la **distancia mediana** de todos los puntos al VP
3. Los puntos se dividen en dos grupos:
  - **Izquierda:** Puntos más cercanos que la mediana
  - **Derecha:** Puntos más lejanos que la mediana
4. Se repite recursivamente





## 2. ¿Por Qué VP-Tree para DistriSearch?

### 2.1 Requisitos del Sistema

- 1. Sin hash para ubicación: Restricción del proyecto
- 2. Preservar localidad semántica: Docs similares juntos
- 3. Espacios métricos arbitrarios: Nuestra distancia combinada

### 2.2 Propiedades de VP-Tree

| Propiedad                         | Beneficio                                  |
|-----------------------------------|--------------------------------------------|
| Funciona con cualquier métrica    | Compatible con nuestra distancia ponderada |
| Divide por distancia, no posición | Agrupar por similitud                      |
| Búsqueda logarítmica              | Eficiente en grandes corpus                |
| Fácil de distribuir               | Cada hoja → un nodo del cluster            |

## 3. Implementación en DistriSearch

### 3.1 Estructura VPNode

```
backend/app/core/partitioning/vp_tree.py

@dataclass
class VPNode:
 """
 Nodo en la estructura VP-Tree.
 """
 vantage_point: Optional[Dict[str, Any]] = None # Documento elegido como VP
 vantage_id: Optional[str] = None # ID del documento VP
 median_distance: float = 0.0 # Umbral de partición
 left: Optional['VPNode'] = None # Subárbol izquierdo
 right: Optional['VPNode'] = None # Subárbol derecho
 documents: List[Dict[str, Any]] = field(default_factory=list) # Docs en hojas
 node_id: str = "" # ID único del nodo
 depth: int = 0 # Profundidad en el árbol
 assigned_node: Optional[str] = None # Nodo del cluster asignado

 @property
 def is_leaf(self) -> bool:
 """¿Es nodo hoja?"""
 return self.left is None and self.right is None

 @property
 def size(self) -> int:
 """Número de documentos en subárbol."""
 if self.is_leaf:
 return len(self.documents)
```

```

count = 1 if self.vantage_point else 0
if self.left:
 count += self.left.size
if self.right:
 count += self.right.size
return count

```

## 3.2 Clase VPTree

```

class VPTree:
 """
 VP-Tree para particionamiento de documentos.
 """

 def __init__(
 self,
 distance_calculator: Optional[DistanceCalculator] = None,
 leaf_size: int = 50,
 selection_strategy: VantagePointSelection = VantagePointSelection.K_MEDOIDS,
 sample_size: int = 10
):
 """
 Args:
 distance_calculator: Calculador de distancias
 leaf_size: Máximo documentos por hoja
 selection_strategy: Estrategia para elegir VP
 sample_size: Candidatos a muestrear para VP
 """
 self.distance_calc = distance_calculator or DistanceCalculator()
 self.leaf_size = leaf_size
 self.selection_strategy = selection_strategy
 self.sample_size = sample_size
 self.root: Optional[VPNode] = None
 self._all_nodes: Dict[str, VPNode] = {}

```

## 4. Construcción Recursiva

### 4.1 Algoritmo

```

def _build_recursive(
 self,
 documents: List[Dict[str, Any]],
 depth: int = 0
) -> Optional[VPNode]:
 """
 Construye VP-Tree recursivamente.
 """
 if not documents:
 return None

```

```

node_id = self._generate_node_id()

Caso base: pocos documentos → nodo hoja
if len(documents) <= self.leaf_size:
 node = VPNode(
 documents=documents.copy(),
 node_id=node_id,
 depth=depth
)
 self._all_nodes[node_id] = node
 return node

Seleccionar vantage point
vp, vp_idx = self._select_vantage_point(documents)
vp_id = vp.get("id") or vp.get("document_id")

Calcular distancias desde VP
remaining = documents[:vp_idx] + documents[vp_idx + 1:]
distances = []

for doc in remaining:
 d = self._distance(vp, doc)
 distances.append((d, doc))

Ordenar y encontrar mediana
distances.sort(key=lambda x: x[0])
median_idx = len(distances) // 2
median_distance = distances[median_idx][0]

Particionar documentos
left_docs = [doc for d, doc in distances if d <= median_distance]
right_docs = [doc for d, doc in distances if d > median_distance]

Crear nodo
node = VPNode(
 vantage_point=vp,
 vantage_id=vp_id,
 median_distance=median_distance,
 node_id=node_id,
 depth=depth
)

Construir subárboles recursivamente
node.left = self._build_recursive(left_docs, depth + 1)
node.right = self._build_recursive(right_docs, depth + 1)

self._all_nodes[node_id] = node
return node

```

## 4.2 Visualización del Proceso

Documentos iniciales: [D1, D2, D3, D4, D5, D6, D7, D8]

Paso 1: Seleccionar  $VP_0 = D3$  (k-medoids)

Calcular distancias a D3:

D1: 0.2, D2: 0.3, D4: 0.5, D5: 0.4, D6: 0.6, D7: 0.35, D8: 0.55

Mediana = 0.4

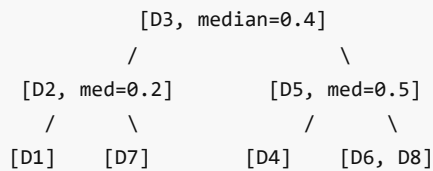
Paso 2: Particionar

Izquierda ( $d \leq 0.4$ ): [D1, D2, D7]

Derecha ( $d > 0.4$ ): [D4, D5, D6, D8]

Paso 3: Recursión en cada subárbol...

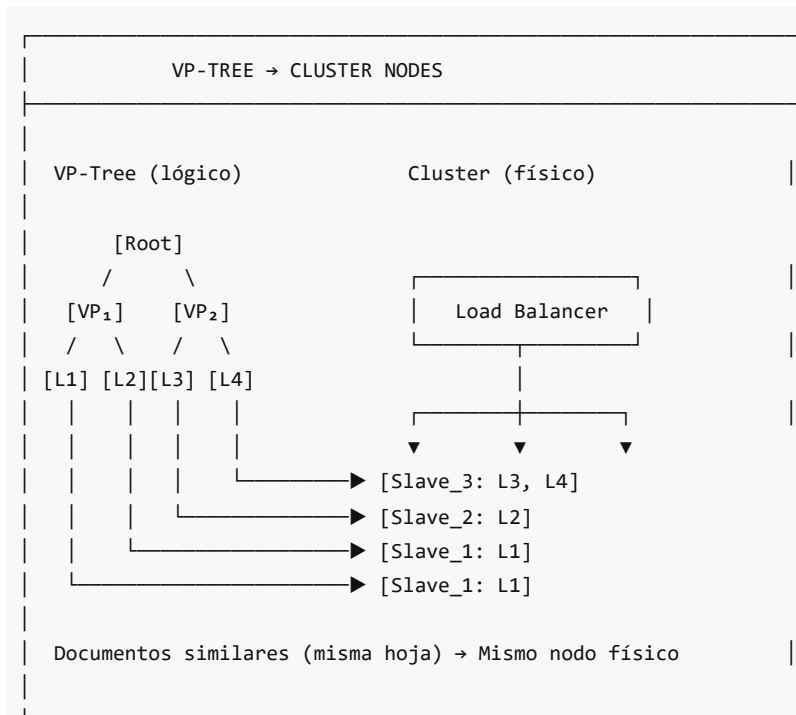
Resultado:



## 5. Distribución: Nodos del Cluster = Hojas del Árbol

### 5.1 Concepto

Cada **nodo hoja** del VP-Tree se asigna a un **nodo del cluster**:



### 5.2 Asignación de Particiones

```

def assign_nodes_to_partitions(
 self,
 cluster_nodes: List[str],
 strategy: str = "balanced"
) -> Dict[str, str]:
 """
 Asigna nodos del cluster a particiones del árbol.

 Returns:
 Mapping: partition_id -> cluster_node_id
 """
 assignments = {}
 leaf_nodes = [n for n in self._all_nodes.values() if n.is_leaf]

 if strategy == "balanced":
 # Ordenar hojas por tamaño (mayor primero)
 leaf_nodes.sort(key=lambda x: len(x.documents), reverse=True)
 node_loads = {n: 0 for n in cluster_nodes}

 for leaf in leaf_nodes:
 # Asignar al nodo menos cargado
 least_loaded = min(node_loads, key=node_loads.get)
 leaf.assigned_node = least_loaded
 assignments[leaf.node_id] = least_loaded
 node_loads[least_loaded] += len(leaf.documents)

 return assignments

```

## 6. Búsqueda en VP-Tree

### 6.1 Búsqueda K-NN (K Nearest Neighbors)

```

def search_knn(
 self,
 query: Dict[str, Any],
 k: int = 10,
 max_distance: Optional[float] = None
) -> List[Tuple[Dict[str, Any], float]]:
 """
 Encuentra los k vecinos más cercanos a la query.
 """
 neighbors: List[Tuple[float, Dict]] = []
 tau = max_distance if max_distance else float('inf') # Radio de búsqueda

 def search_recursive(node: Optional[VPNode]):
 nonlocal tau

 if node is None:
 return

```

```

if node.is_leaf:
 # Verificar todos los documentos en la hoja
 for doc in node.documents:
 d = self._distance(query, doc)
 if d < tau:
 neighbors.append((d, doc))
 neighbors.sort(key=lambda x: x[0])
 if len(neighbors) > k:
 neighbors.pop()
 if len(neighbors) == k:
 tau = neighbors[-1][0] # Actualizar radio
 return

Calcular distancia al vantage point
vp_dist = self._distance(query, node.vantage_point)

Verificar si VP califica
if vp_dist < tau:
 neighbors.append((vp_dist, node.vantage_point))
 # ... actualizar tau

Determinar orden de búsqueda (poda inteligente)
if vp_dist < node.median_distance:
 # Query más cerca de izquierda
 if vp_dist - tau <= node.median_distance:
 search_recursive(node.left)
 if vp_dist + tau >= node.median_distance:
 search_recursive(node.right)
else:
 # Query más cerca de derecha
 if vp_dist + tau >= node.median_distance:
 search_recursive(node.right)
 if vp_dist - tau <= node.median_distance:
 search_recursive(node.left)

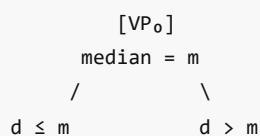
search_recursive(self.root)
return [(doc, dist) for dist, doc in neighbors]

```

## 6.2 Poda del Espacio de Búsqueda

La clave de la eficiencia es la **poda**:

Query Q con radio  $\tau$  (distancia al k-ésimo vecino actual)



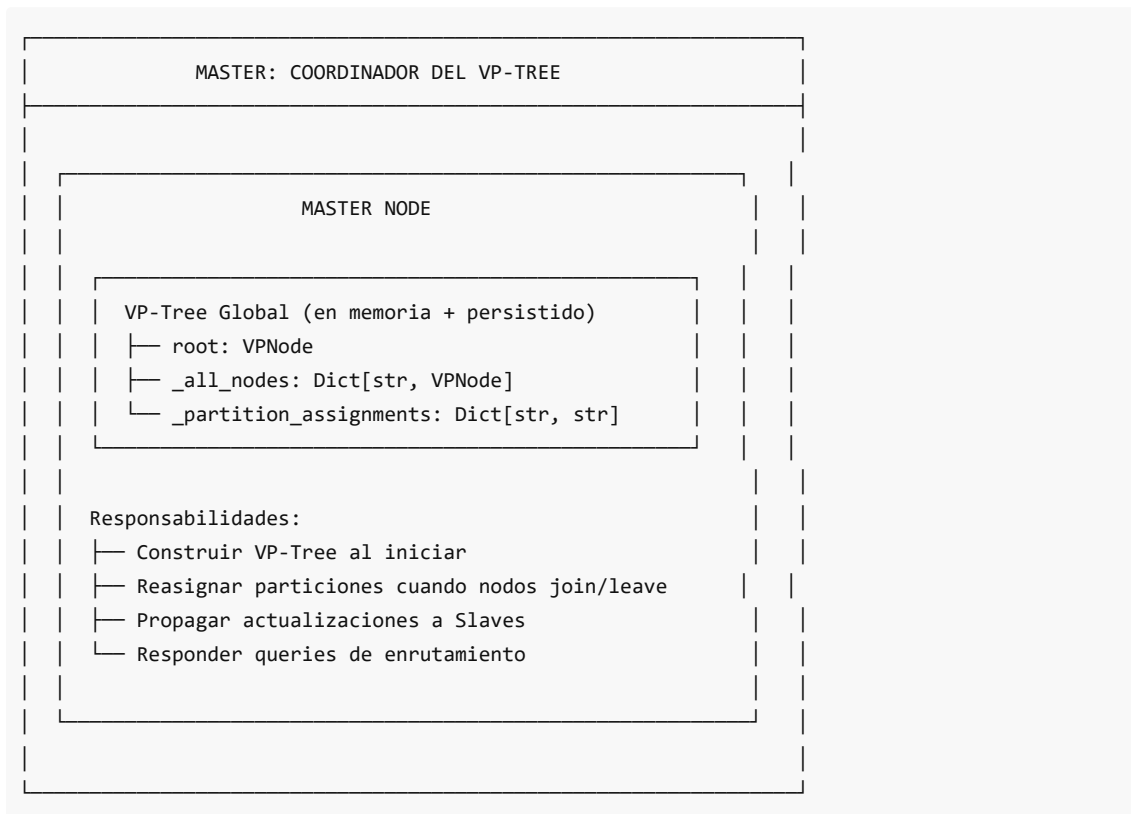
Si  $d(Q, VP_0) = 0.3$  y  $m = 0.5$ :

- Q está en región izquierda

- Si  $\tau = 0.1$ :
  - Izquierda:  $d(Q, VP) - \tau = 0.2 \leq 0.5$  ✓ BUSCAR
  - Derecha:  $d(Q, VP) + \tau = 0.4 < 0.5$  ✗ PODAR

Resultado: Solo buscamos en subárbol izquierdo (50% poda)

## 7. El Master Mantiene el VP-Tree Global



## 8. Complejidad

| Operación     | Complejidad                            |
|---------------|----------------------------------------|
| Construcción  | $O(n \log n)$                          |
| Búsqueda K-NN | $O(\log n)$ promedio, $O(n)$ peor caso |
| Inserción     | $O(\log n)$                            |
| Rango         | $O(\log n + k)$ donde k = resultados   |

## 9. Parámetros de Configuración

```
backend/app/core/partitioning/vp_tree.py

class VPTree:
```

```
def __init__(
 self,
 leaf_size: int = 50, # Docs máximos por hoja
 selection_strategy: VantagePointSelection = VantagePointSelection.K_MEDOIDS,
 sample_size: int = 10 # Candidatos para VP
):
```

| Parámetro          | Default   | Descripción                      |
|--------------------|-----------|----------------------------------|
| leaf_size          | 50        | Más pequeño = árbol más profundo |
| selection_strategy | K_MEDOIDS | Mejor calidad de partición       |
| sample_size        | 10        | Trade-off calidad/velocidad      |

**Anterior:** [README.md](#)

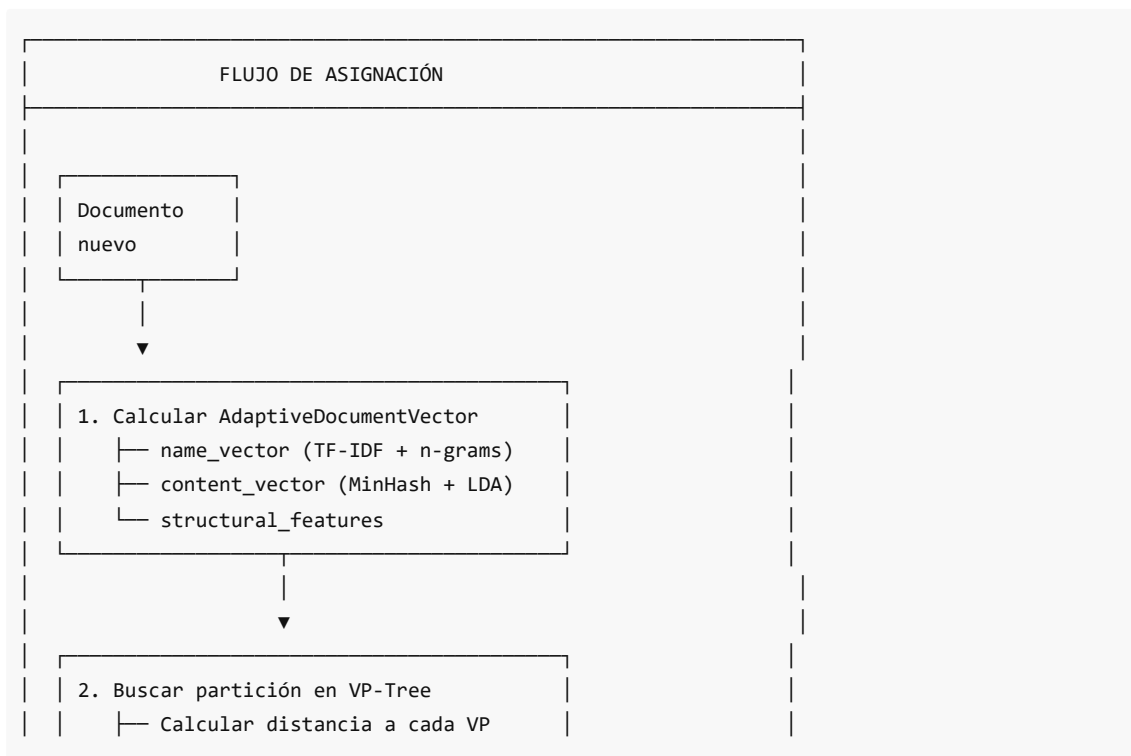
**Siguiente:** [02 Algoritmo Asignacion.md](#)

# Algoritmo de Asignación de Documentos

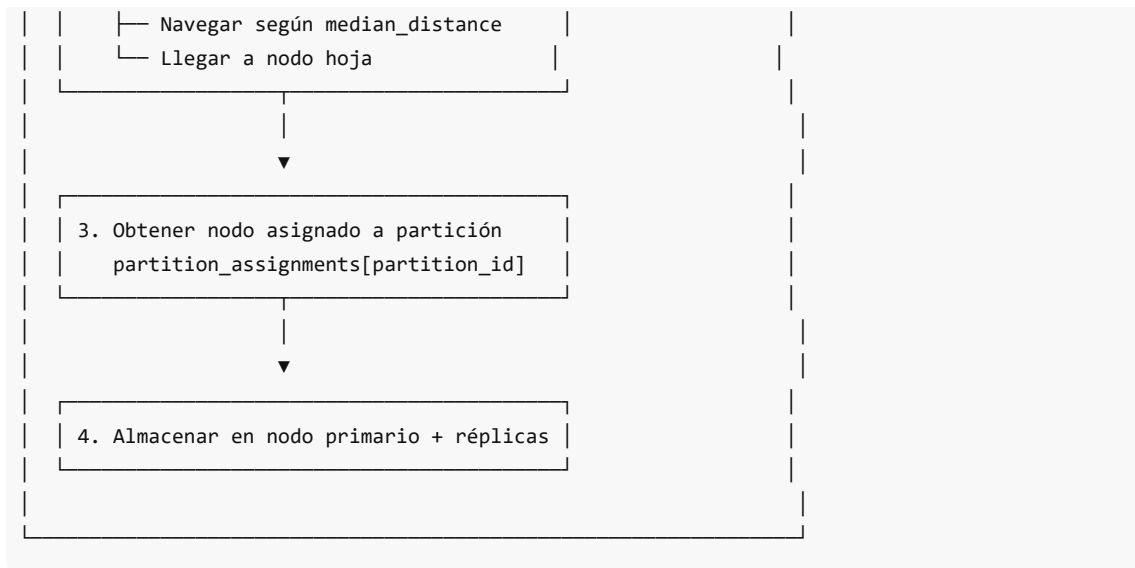
## Enrutamiento Basado en VP-Tree

### 1. Visión General

Cuando llega un **nuevo documento**, el sistema debe decidir en qué **nodo del cluster** almacenarlo. El algoritmo garantiza que documentos similares residan juntos.







## 2. Clase VPTreePartitioner (Conceptual)

El `PartitionManager` actúa como el `VPTreePartitioner`:

```
backend/app/core/partitioning/partition_manager.py

class PartitionManager:
 """
 Gestor de particiones VP-Tree.
 """

 def __init__(
 self,
 leaf_size: int = 50,
 replication_factor: int = 2,
 vp_selection: VantagePointSelection = VantagePointSelection.K_MEDOIDS
):
 self.vp_tree = VPTree(
 leaf_size=leaf_size,
 selection_strategy=vp_selection
)
 self.node_assigner = NodeAssigner(
 replication_factor=replication_factor
)
 self._partition_assignments: Dict[str, str] = {} # partition → node
```

## 3. Algoritmo: Encontrar Partición

### 3.1 find\_partition

```
backend/app/core/partitioning/vp_tree.py
```

```

def find_partition(self, query: Dict[str, Any]) -> Optional[str]:
 """
 Encuentra la partición (nodo hoja) donde pertenece el documento.

 Args:
 query: Documento con vectores

 Returns:
 ID de la partición (nodo hoja del VP-Tree)
 """
 if self.root is None:
 return None

 node = self.root

 while not node.is_leaf:
 # Calcular distancia al vantage point actual
 vp_dist = self._distance(query, node.vantage_point)

 # Decidir rama según distancia vs mediana
 if vp_dist <= node.median_distance:
 # Documento más cercano → rama izquierda
 if node.left:
 node = node.left
 else:
 break
 else:
 # Documento más lejano → rama derecha
 if node.right:
 node = node.right
 else:
 break

 return node.node_id

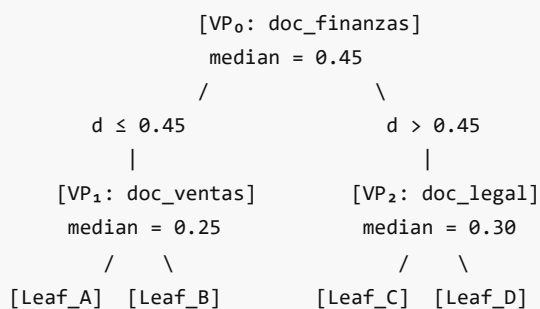
```

## 3.2 Visualización

Documento nuevo: "ReporteVentas\_Q1\_2024.xlsx"

Vector calculado: {name: {...}, content: {...}, topics: [...]}

VP-Tree:



Proceso:

1.  $d(\text{doc}, \text{VP}_0) = 0.32 \rightarrow \leq 0.45 \rightarrow \text{Izquierda}$
2.  $d(\text{doc}, \text{VP}_1) = 0.18 \rightarrow \leq 0.25 \rightarrow \text{Izquierda}$
3. Llegamos a Leaf\_A  $\rightarrow \text{partition\_id} = \text{"vpn\_3"}$

Resultado: Documento va a Leaf\_A (asignada a Slave\_1)

## 4. Cálculo de Distancia

### 4.1 Fórmula Combinada

```
backend/app/core/partitioning/distance_metrics.py

class DistanceCalculator:
 """
 Calcula distancias usando fórmula ponderada:
 $d(A,B) = 0.4 * \text{cosine_distance}(\text{name}) +$
 $0.4 * \text{jaccard_distance}(\text{content}) +$
 $0.2 * \text{jsd}(\text{topics})$
 """

 def weighted_distance(
 self,
 doc_a: Dict[str, Any],
 doc_b: Dict[str, Any]
) -> float:
 """
 Distancia ponderada entre documentos.
 """

 # Distancia del nombre (coseno)
 name_dist = self.cosine_distance(
 doc_a.get("name_vector"),
 doc_b.get("name_vector")
)

 # Distancia del contenido (Jaccard via MinHash)
 content_dist = self.jaccard_distance(
 doc_a.get("minhash_signature"),
 doc_b.get("minhash_signature")
)

 # Distancia de tópicos (Jensen-Shannon)
 topic_dist = self.jensen_shannon_distance(
 doc_a.get("topic_distribution"),
 doc_b.get("topic_distribution")
)

 # Combinación ponderada
 return (
 self.weights.name_weight * name_dist +
```

```

 self.weights.content_weight * content_dist +
 self.weights.topic_weight * topic_dist
)

```

## 4.2 Métricas Individuales

```

def cosine_distance(self, vec_a: np.ndarray, vec_b: np.ndarray) -> float:
 """
 Distancia coseno: $d = (1 - \cos(\theta)) / 2$

 Normalizada a [0, 1]
 """
 similarity = self.cosine_similarity(vec_a, vec_b)
 return (1.0 - similarity) / 2.0

def jaccard_distance(self, sig_a: np.ndarray, sig_b: np.ndarray) -> float:
 """
 Distancia Jaccard estimada desde MinHash:
 $d = 1 - J(A, B)$
 """
 similarity = self.jaccard_similarity(sig_a, sig_b)
 return 1.0 - similarity

def jensen_shannon_distance(self, p: np.ndarray, q: np.ndarray) -> float:
 """
 Distancia Jensen-Shannon:
 $d = \sqrt{\text{JSD}(P || Q)}$

 Donde $\text{JSD} = 0.5 * \text{KL}(P || M) + 0.5 * \text{KL}(Q || M)$, $M = (P+Q)/2$
 """
 m = 0.5 * (p + q)
 jsd = 0.5 * np.sum(p * np.log(p / m)) + 0.5 * np.sum(q * np.log(q / m))
 return np.sqrt(jsd / np.log(2))

```

## 5. Enrutamiento Completo

### 5.1 route\_document

```

backend/app/core/partitioning/partition_manager.py

def route_document(
 self,
 document: Dict[str, Any],
 strategy: AssignmentStrategy = AssignmentStrategy.VP_TREE_PARTITION
) -> RoutingResult:
 """
 Enruta documento al nodo apropiado.
 """
 doc_id = document.get("id")

```

```

1. Encontrar partición en VP-Tree
partition_id = self.vp_tree.find_partition(document)

2. Obtener nodo asignado a esa partición
if partition_id and partition_id in self._partition_assignments:
 primary_node = self._partition_assignments[partition_id]

3. Obtener nodos para réplicas (afinidad semántica)
replica_result = self.node_assigner.assign_document(
 document,
 strategy=AssignmentStrategy.SEMANTIC_AFFINITY,
 partition_id=partition_id
)

return RoutingResult(
 document_id=doc_id,
 partition_id=partition_id,
 primary_node=primary_node,
 replica_nodes=replica_result.replica_nodes
)

Fallback: usar NodeAssigner directamente
result = self.node_assigner.assign_document(document, strategy)

return RoutingResult(
 document_id=doc_id,
 partition_id="unpartitioned",
 primary_node=result.assigned_node,
 replica_nodes=result.replica_nodes
)

```

## 6. Estrategias de Asignación Alternativas

### 6.1 Enum de Estrategias

```

backend/app/core/partitioning/node_assignment.py

class AssignmentStrategy(Enum):
 """Estrategias de asignación de documentos."""
 ROUND_ROBIN = "round_robin" # Rotativo
 LEAST_LOADED = "least_loaded" # Menos cargado
 SEMANTIC_AFFINITY = "semantic_affinity" # Por similaridad
 CONSISTENT_HASH = "consistent_hash" # Hash consistente
 VP_TREE_PARTITION = "vp_tree_partition" # ● Principal

```

### 6.2 Afinidad Semántica (para réplicas)

```

def _semantic_affinity_assignment(
 self,
 document: Dict[str, Any],
 exclude_nodes: Optional[List[str]] = None
) -> Optional[str]:
 """
 Asigna basándose en afinidad semántica.

 Coloca documento en nodo que tiene documentos más similares.
 """
 exclude = set(exclude_nodes or [])
 healthy_nodes = [
 n for n in self._nodes.values()
 if n.is_healthy and n.available_capacity > 0 and n.node_id not in exclude
]

 best_node = None
 best_affinity = float('inf') # Menor distancia = mejor

 for node in healthy_nodes:
 node_docs = self._node_documents.get(node.node_id, [])

 if not node_docs:
 affinity = 0.5 # Nodo vacío: afinidad neutral
 else:
 # Promedio de distancias a muestra de documentos
 sample = node_docs[:10]
 distances = [
 self.distance_calc.weighted_distance(document, doc)
 for doc in sample
]
 affinity = np.mean(distances)

 # Penalizar nodos muy cargados
 load_penalty = node.load_factor * 0.2
 adjusted_affinity = affinity + load_penalty

 if adjusted_affinity < best_affinity:
 best_affinity = adjusted_affinity
 best_node = node.node_id

 return best_node

```

## 7. Ejemplo Completo

```

Nuevo documento llega
document = {
 "id": "doc_12345",
 "filename": "ReporteVentas_Q1_2024.xlsx",

```

```

 "content": "Las ventas del trimestre...",
 "vectors": {...} # Pre-calculados
}

1. Vectorizar (si no está pre-calculado)
vectorizer = DocumentVectorizer()
doc_vector = vectorizer.vectorize(
 filename=document["filename"],
 content=document["content"]
)

2. Agregar vector al documento
document["name_vector"] = doc_vector.name_vector.tokens_tfidf
document["minhash_signature"] = doc_vector.content_vector.minhash_signatures
document["topic_distribution"] = doc_vector.content_vector.topic_distribution

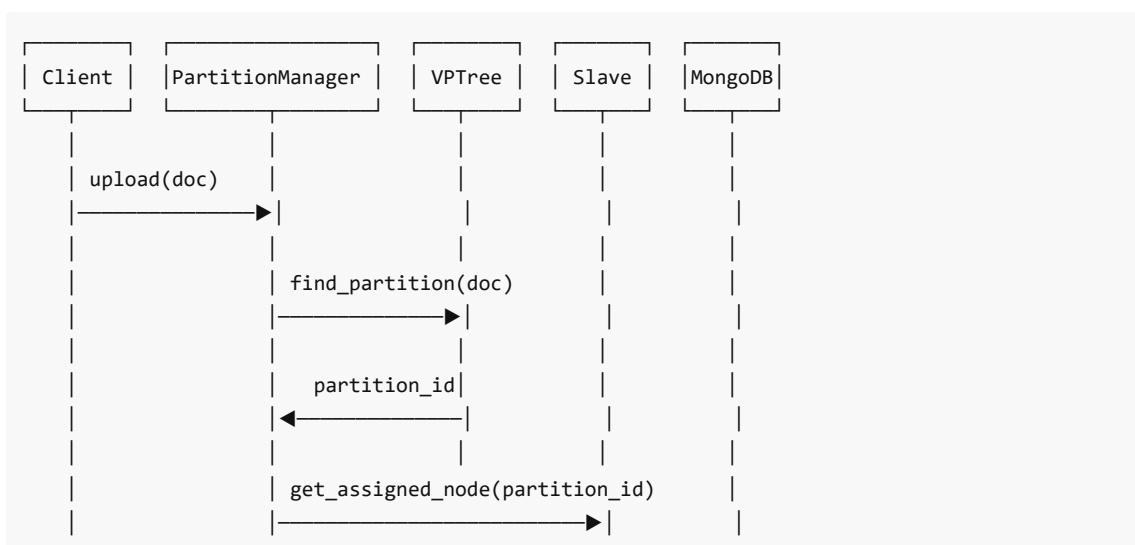
3. Enrutar
partition_manager = PartitionManager()
routing = partition_manager.route_document(document)

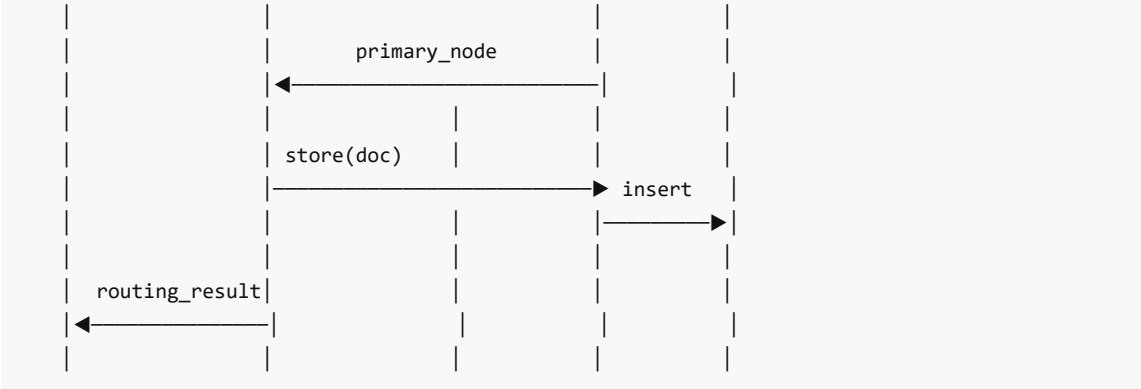
print(routing)
RoutingResult(
document_id="doc_12345",
partition_id="vpn_7",
primary_node="slave_2",
replica_nodes=["slave_1", "slave_3"]
)

4. Almacenar
await store_document(routing.primary_node, document)
for replica_node in routing.replica_nodes:
 await replicate_document(replica_node, document)

```

## 8. Diagrama de Secuencia





## 9. Propiedades del Algoritmo

| Propiedad              | Valor                             |
|------------------------|-----------------------------------|
| Complejidad asignación | $O(\log n)$                       |
| Garantía de localidad  | Sí (docs similares → mismo nodo)  |
| Balanceo de carga      | Sí (estrategia "balanced")        |
| Tolerancia a fallos    | Sí (réplicas en nodos diferentes) |
| Determinístico         | Sí (mismo doc → misma partición)  |

**Anterior:** [01 VP Tree Distribuido.md](#)

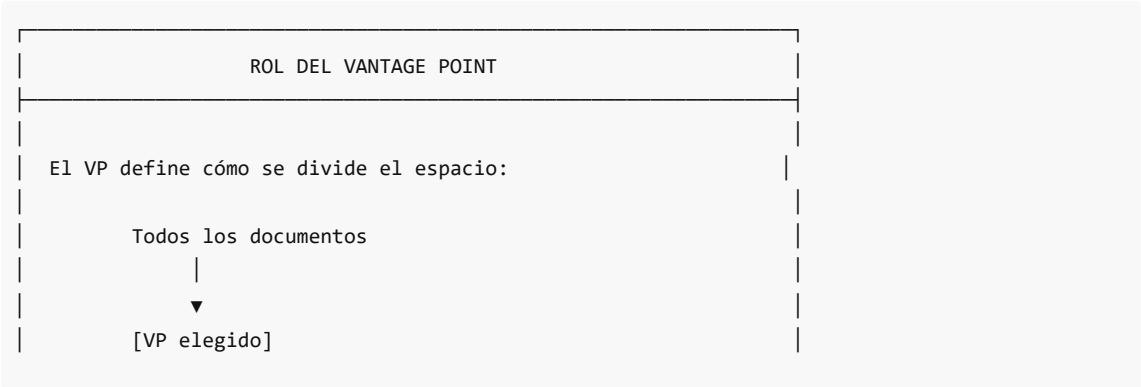
**Siguiente:** [03 Vantage Points.md](#)

# Vantage Points

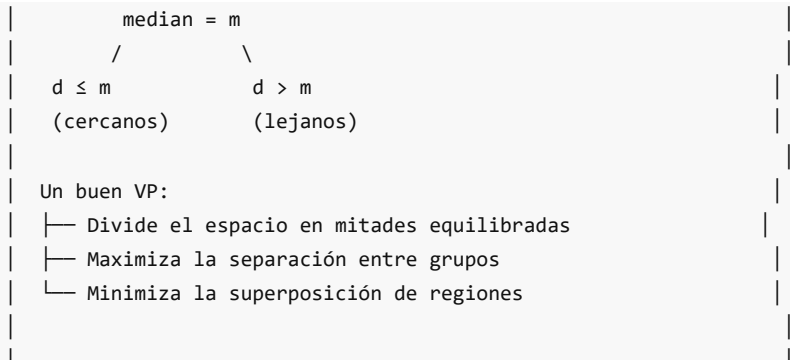
## Selección y Gestión de Puntos de Referencia

### 1. ¿Qué es un Vantage Point?

Un **Vantage Point (VP)** es un documento elegido como **punto de referencia** para dividir el espacio. La calidad del VP afecta directamente la eficiencia del árbol.







## 2. Estrategias de Selección

DistriSearch implementa tres estrategias:

```
backend/app/core/partitioning/vp_tree.py

class VantagePointSelection(Enum):
 """Estrategias para seleccionar vantage points."""
 RANDOM = "random" # Aleatorio (rápido, baja calidad)
 K_MEDOIDS = "k_medoids" # 🟡 Por defecto (óptimo)
 MAX_SPREAD = "max_spread" # Máxima varianza
```

| Estrategia | Complejidad | Calidad    | Uso                 |
|------------|-------------|------------|---------------------|
| RANDOM     | O(1)        | Baja       | Construcción rápida |
| K_MEDOIDS  | O(n-k)      | Alta       | Producción          |
| MAX_SPREAD | O(n-k)      | Media-Alta | Alternativa         |

## 3. Algoritmo K-Medoids

### 3.1 Concepto

El **medoid** es el punto que **minimiza la suma de distancias** a todos los demás puntos del conjunto. Es más robusto que el centroide (k-means) porque:

- No requiere calcular promedios (válido para cualquier métrica)
- Siempre es un punto real del dataset
- Más resistente a outliers

### 3.2 Fórmula

Para un conjunto de puntos  $P$ , el medoid  $m$  es:

$$m = \arg\min_{p \in P} \sum_{q \in P} d(p, q)$$

### 3.3 Implementación

```
backend/app/core/partitioning/vp_tree.py

def _select_vantage_point_kmedoids(
 self,
 documents: List[Dict[str, Any]]
) -> Tuple[Dict[str, Any], int]:
 """
 Selecciona VP usando enfoque k-medoids.

 Encuentra el documento que minimiza la suma de distancias
 a todos los demás (el medoid).
 """
 n = len(documents)

 # Muestrear candidatos para eficiencia
 if n <= self.sample_size:
 candidates = list(range(n))
 else:
 candidates = random.sample(range(n), self.sample_size)

 best_idx = candidates[0]
 best_total_dist = float('inf')

 for idx in candidates:
 # Calcular suma de distancias a todos los documentos
 total_dist = 0.0
 for j in range(n):
 if j != idx:
 total_dist += self._distance(documents[idx], documents[j])

 # Actualizar mejor candidato
 if total_dist < best_total_dist:
 best_total_dist = total_dist
 best_idx = idx

 return documents[best_idx], best_idx
```

### 3.4 Visualización

Documentos: [D1, D2, D3, D4, D5]

Distancias (matriz simétrica):

|    | D1  | D2  | D3  | D4  | D5  |
|----|-----|-----|-----|-----|-----|
| D1 | 0   | 0.3 | 0.5 | 0.7 | 0.4 |
| D2 | 0.3 | 0   | 0.4 | 0.6 | 0.3 |
| D3 | 0.5 | 0.4 | 0   | 0.3 | 0.5 |
| D4 | 0.7 | 0.6 | 0.3 | 0   | 0.6 |
| D5 | 0.4 | 0.3 | 0.5 | 0.6 | 0   |

Suma de distancias:

D1:  $0.3 + 0.5 + 0.7 + 0.4 = 1.9$   
D2:  $0.3 + 0.4 + 0.6 + 0.3 = 1.6 \leftarrow \text{Mínimo (MEDOID)}$   
D3:  $0.5 + 0.4 + 0.3 + 0.5 = 1.7$   
D4:  $0.7 + 0.6 + 0.3 + 0.6 = 2.2$   
D5:  $0.4 + 0.3 + 0.5 + 0.6 = 1.8$

VP elegido: D2 (medoid del conjunto)

## 4. Estrategia MAX\_SPREAD

Alternativa que maximiza la **varianza** de distancias:

```
def _select_vantage_point_spread(
 self,
 documents: List[Dict[str, Any]]
) -> Tuple[Dict[str, Any], int]:
 """
 Selecciona VP que maximiza la dispersión de distancias.

 Un VP con alta varianza en distancias divide mejor el espacio.
 """
 n = len(documents)
 candidates = random.sample(range(n), min(self.sample_size, n))

 best_idx = candidates[0]
 best_spread = -1.0

 for idx in candidates:
 # Calcular distancias a todos
 distances = []
 for j in range(n):
 if j != idx:
 d = self._distance(documents[idx], documents[j])
 distances.append(d)

 # Spread = varianza de distancias
 if distances:
 spread = np.var(distances)
 if spread > best_spread:
 best_spread = spread
 best_idx = idx

 return documents[best_idx], best_idx
```

### 4.1 ¿Por qué varianza?

Candidato A (baja varianza):  
Distancias: [0.4, 0.42, 0.38, 0.41, 0.39]  
Varianza: 0.0002  
Problema: Todos los docs equidistantes → mala partición

Candidato B (alta varianza):

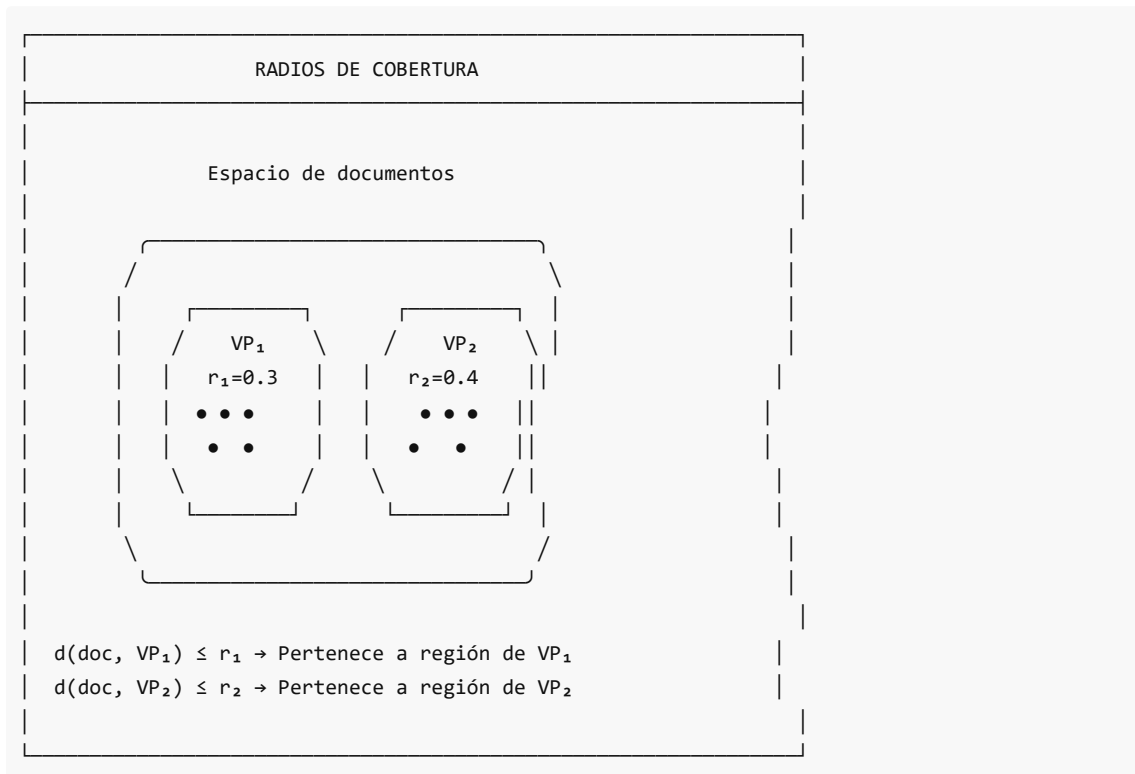
Distancias: [0.1, 0.15, 0.8, 0.85, 0.9]

Varianza: 0.12

Ventaja: Clara separación entre cercanos y lejanos

## 5. Radio de Cobertura

Cada VP tiene un **radio** (median\_distance) que define su región:



### 5.1 Cálculo del Radio

```
def _build_recursive(self, documents: List[Dict], depth: int = 0):
 # ... seleccionar VP ...

 # Calcular distancias desde VP
 distances = [(self._distance(vp, doc), doc) for doc in remaining]
 distances.sort(key=lambda x: x[0])

 # Mediana como radio
 median_idx = len(distances) // 2
 median_distance = distances[median_idx][0] # Este es el RADIO

 # Particionar
 left_docs = [doc for d, doc in distances if d <= median_distance]
 right_docs = [doc for d, doc in distances if d > median_distance]
```

## 6. Recomputación al Cambiar Topología

Cuando un nodo se une o abandona el cluster:

```
backend/app/core/partitioning/partition_manager.py

async def on_node_join(self, new_node_id: str):
 """
 Maneja la incorporación de un nuevo nodo.
 """
 # 1. Registrar nuevo nodo
 self.node_assigner.register_node(new_node_id)

 # 2. Recalcular asignaciones de particiones
 cluster_nodes = [n.node_id for n in self.node_assigner.get_healthy_nodes()]

 self._partition_assignments = self.vp_tree.assign_nodes_to_partitions(
 cluster_nodes,
 strategy="balanced"
)

 # 3. Identificar documentos a migrar
 migrations = self._identify_migrations(new_node_id)

 # 4. Migrar gradualmente
 for batch in chunks(migrations, 50):
 await self._migrate_batch(batch, new_node_id)
 await asyncio.sleep(1) # Rate limiting

async def on_node_leave(self, failed_node_id: str):
 """
 Maneja la salida/fallo de un nodo.
 """
 # 1. Obtener documentos huérfanos
 orphaned = self.node_assigner.unregister_node(failed_node_id)

 # 2. Reasignar particiones
 cluster_nodes = [n.node_id for n in self.node_assigner.get_healthy_nodes()]

 self._partition_assignments = self.vp_tree.assign_nodes_to_partitions(
 cluster_nodes,
 strategy="balanced"
)

 # 3. Recuperar documentos desde réplicas
 for doc_id in orphaned:
 await self._recover_from_replica(doc_id)
```

---

## 7. Impacto en Eficiencia de Búsqueda

### 7.1 VP Bien Elegido

```
Query: doc_ventas

VP bien elegido (central):
├─ Distancia a VP = 0.3
├─ median = 0.4
├─ τ (radio de búsqueda) = 0.1
├─
├─ Condición izquierda: $0.3 - 0.1 = 0.2 \leq 0.4$ ✓ BUSCAR
├─ Condición derecha: $0.3 + 0.1 = 0.4 = 0.4$ ✗ PODAR

Resultado: Solo buscamos 50% del árbol
```

### 7.2 VP Mal Elegido

```
VP mal elegido (extremo):
 Todos los documentos a distancia similar del VP
 └─ Distancias: [0.5, 0.51, 0.49, 0.52, ...]
 └─ median ≈ 0.5
 └─
 └─ Cualquier query cercana al borde:
 └─ Condición izquierda: ✓ BUSCAR
 └─ Condición derecha: ✓ BUSCAR

Resultado: Buscamos 100% del árbol (sin poda)
```

## 8. Trade-offs en Selección

| Aspecto             | K_MEDOIDS  | MAX_SPREAD | RANDOM     |
|---------------------|------------|------------|------------|
| Tiempo construcción | Medio      | Medio      | Muy rápido |
| Calidad partición   | Óptima     | Buena      | Variable   |
| Balanceo            | Excelente  | Bueno      | Aleatorio  |
| Robustez outliers   | Alta       | Media      | Baja       |
| Uso recomendado     | Producción | Testing    | Prototipo  |

## 9. Ejemplo: Recomputación de VPs

```
Escenario: Nuevo nodo se une

Estado inicial
vp_tree.root = VPNode(
 vantage_id="doc_central",
 median_distance=0.45,
```

```

 left=VPNode(assigned_node="slave_1", ...),
 right=VPNode(assigned_node="slave_2", ...)
)

Nuevo nodo: slave_3 se une

1. No reconstruimos el árbol completo (costoso)
2. Solo reasignamos particiones

new_assignments = {
 "vpn_1": "slave_1", # Sin cambio
 "vpn_2": "slave_3", # Movido de slave_2 a slave_3
 "vpn_3": "slave_2",
 "vpn_4": "slave_3" # Nueva asignación
}

3. Migrar documentos de particiones reasignadas
migrate(from="slave_2", to="slave_3", partition="vpn_2")

```

## 10. Persistencia de Vantage Points

```

def save_vp_tree(self, path: str):
 """Guarda VP-Tree para recuperación."""
 state = {
 "root": self._serialize_node(self.root),
 "partition_assignments": self._partition_assignments,
 "metadata": {
 "document_count": self._document_count,
 "leaf_size": self.leaf_size,
 "created_at": datetime.utcnow().isoformat()
 }
 }

 with open(path, 'w') as f:
 json.dump(state, f)

def load_vp_tree(self, path: str):
 """Recupera VP-Tree desde disco."""
 with open(path, 'r') as f:
 state = json.load(f)

 self.root = self._deserialize_node(state["root"])
 self._partition_assignments = state["partition_assignments"]

```

## 11. Resumen de Propiedades

| Propiedad | Descripción |
|-----------|-------------|
|-----------|-------------|

|                  |                                                |
|------------------|------------------------------------------------|
| <b>Medoid</b>    | Punto que minimiza distancia total             |
| <b>Radio</b>     | Distancia mediana al VP                        |
| <b>Región</b>    | Todos los puntos a distancia $\leq$ radio      |
| <b>Poda</b>      | Descartar regiones fuera del radio de búsqueda |
| <b>Recomputo</b> | Solo asignaciones, no estructura (eficiente)   |

*Anterior:* [02 Algoritmo Asignacion.md](#)

*Siguiente sección:* [../04 Rebalanceo/README.md](#)

# 04 Rebalanceo

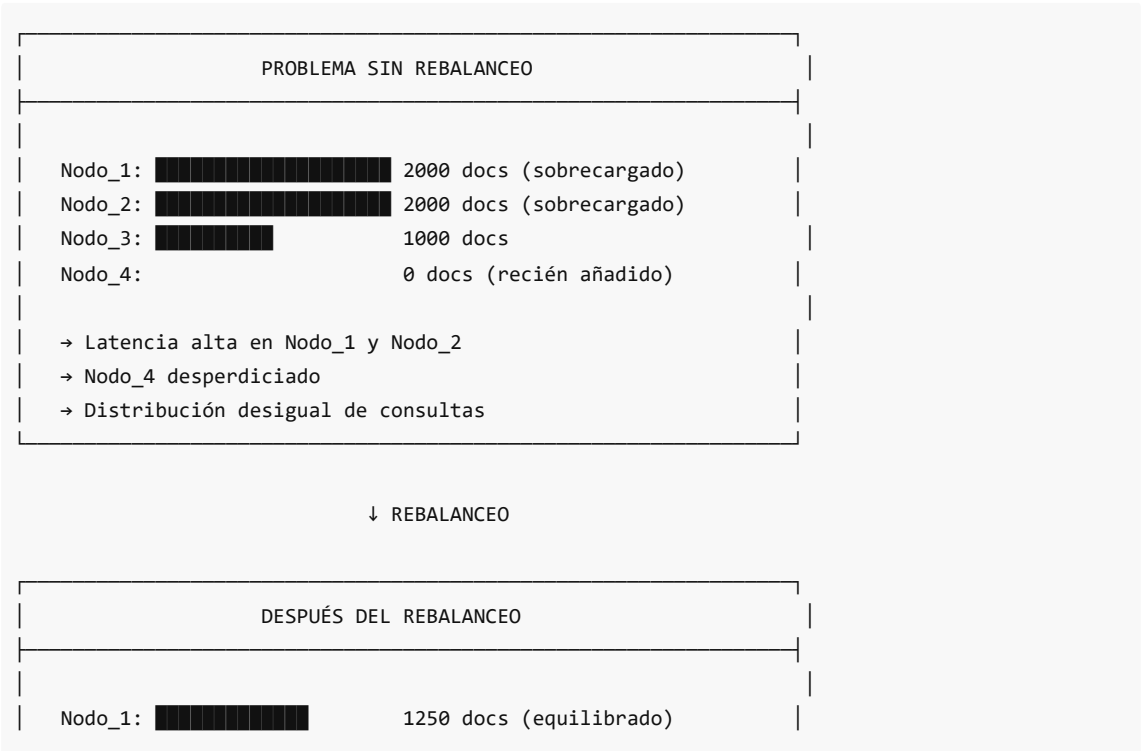
## 04. Rebalanceo Activo

### Visión General

El **rebalanceo activo** es uno de los mecanismos más importantes en DistriSearch para mantener una distribución equilibrada de documentos cuando la topología del clúster cambia. Este módulo se activa automáticamente cuando:

- Un nuevo nodo se une al clúster
- Un nodo existente abandona el clúster
- Se detecta un desbalance de carga significativo

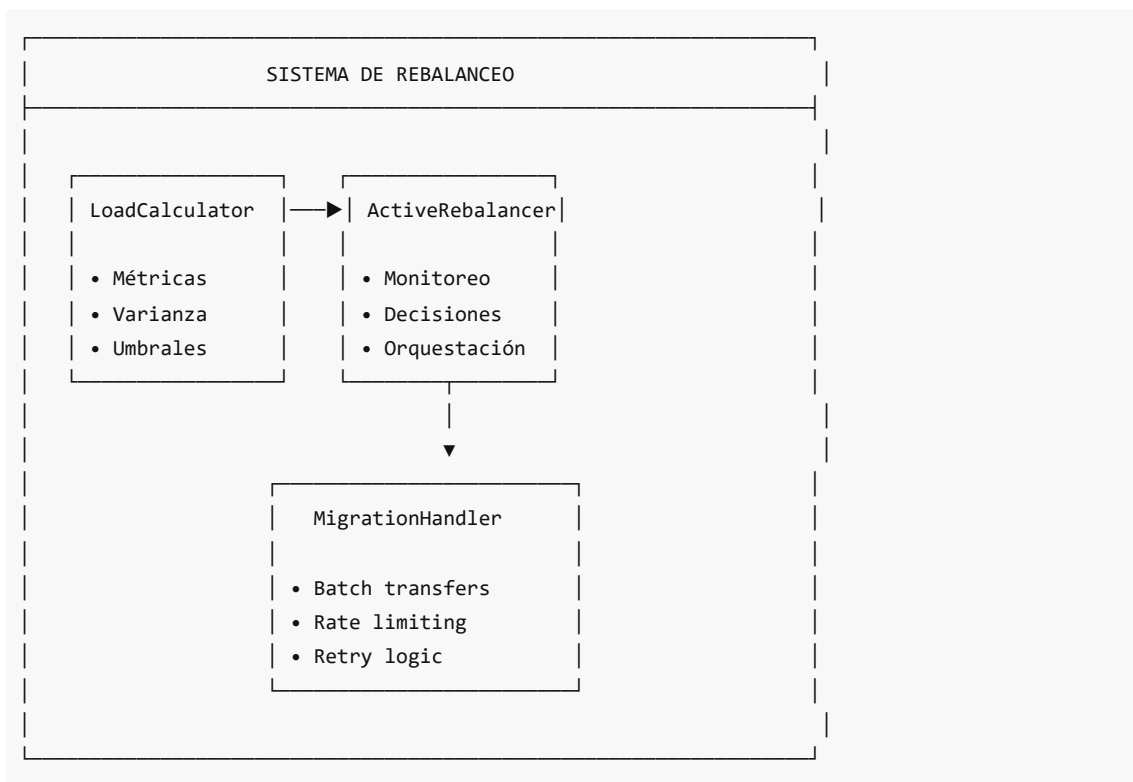
### ¿Por Qué Rebalancear?





```
Nodo_2: ██████████ 1250 docs (equilibrado)
Nodo_3: ██████████ 1250 docs (equilibrado)
Nodo_4: ██████████ 1250 docs (equilibrado)

→ Carga distribuida uniformemente
→ Mejor tiempo de respuesta
→ Utilización óptima de recursos
```



| Componente       | Archivo              | Responsabilidad                                       |
|------------------|----------------------|-------------------------------------------------------|
| ActiveRebalancer | active_rebalancer.py | Orquesta todo el proceso de rebalanceo                |
| LoadCalculator   | load_calculator.py   | Calcula métricas de carga y decide cuándo rebalancear |
| MigrationHandler | migration_handler.py | Ejecuta migraciones con batching y rate limiting      |

### Factor de Carga

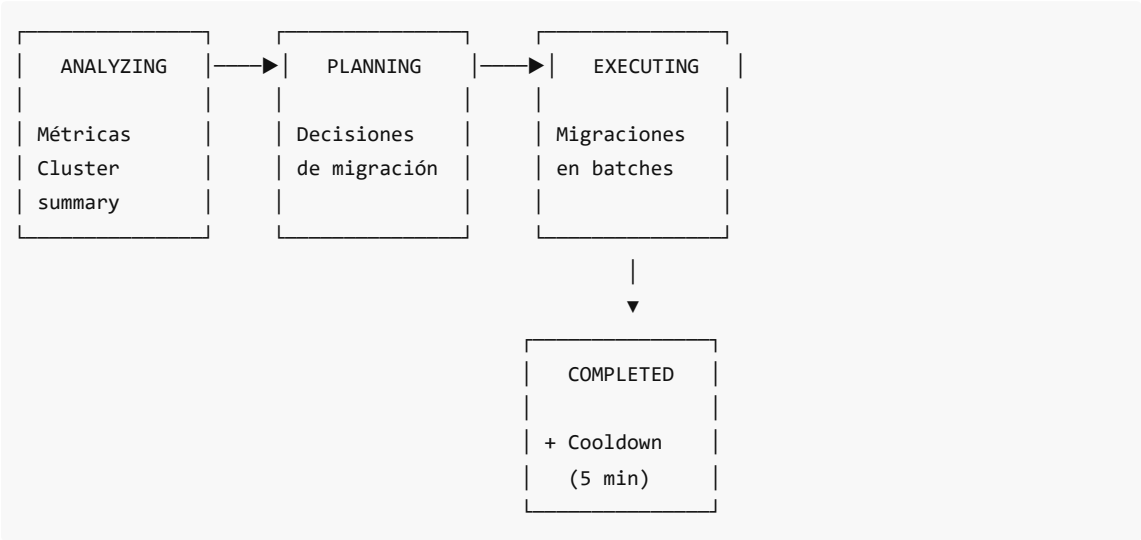
Carga Combinada (ponderada)

$$combined\_load = 0.4 \cdot load\_factor + 0.2 \cdot storage\_factor + 0.2 \cdot cpu\_usage + 0.2 \cdot memory\_usage$$

Umbral de Desbalance

$$needs\_rebalance = load\_std\_dev > 0.2 \vee imbalance\_ratio > 0.5$$

Flujo de Rebalanceo



Contenido de Esta Sección

- 1. [Rebalanceo Activo](#): La clase `ActiveRebalancer` y su proceso completo
- 2. [Power of Two Choices](#): Algoritmo de selección de nodo destino
- 3. [Migración de Documentos](#): Proceso detallado de transferencia

Navegación:

- [← Anterior: Particionamiento](#)
- [→ Siguiente: Replicación](#)

Rebalanceo Activo

Concepto

El **rebalanceo activo** es un mecanismo proactivo que redistribuye documentos entre nodos del clúster para mantener una carga equilibrada. A diferencia del rebalanceo reactivo (que solo actúa ante fallos), el activo monitorea continuamente y actúa preventivamente.

Clase `ActiveRebalancer`

Ubicación: `backend/app/core/rebalancing/active_rebalancer.py`

```

class ActiveRebalancer:
 """
 Actively monitors and rebalances cluster load.

 Features:
 - Continuous load monitoring
 - Automatic rebalance triggering
 - Coordinated migrations with rate limiting
 - Operation history tracking
 """

 def __init__(
 self,
 config: Optional[RebalanceConfig] = None,
 document_selector: Callable[[str, int], Awaitable[List[str]]] = None,
 transfer_func: Callable[[str, str, List[str]], Awaitable[Dict]] = None
):
 self.config = config or RebalanceConfig()
 self.load_calculator = LoadCalculator(
 imbalance_threshold=self.config.imbalance_threshold,
 critical_threshold=self.config.critical_threshold,
 min_transfer_size=self.config.min_documents_to_move
)
 self.migration_handler = MigrationHandler(...)

```

## Configuración de Rebalanceo

```

@dataclass
class RebalanceConfig:
 """Configuration for rebalancing."""
 # Umbrales
 imbalance_threshold: float = 0.2 # Desviación estándar máxima
 critical_threshold: float = 0.9 # Factor de carga crítico
 min_documents_to_move: int = 10 # Mínimo para justificar migración

 # Timing
 check_interval_sec: float = 60.0 # Verificar cada 60 segundos
 cooldown_after_rebalance_sec: float = 300.0 # 5 min de cooldown

 # Configuración de migración (según arquitectura)
 batch_size: int = 50 # 50 docs por batch
 batch_delay_sec: float = 1.0 # 1 segundo entre batches
 max_concurrent_migrations: int = 2 # Máximo 2 migraciones paralelas

 # Límites
 max_documents_per_rebalance: int = 1000 # Máximo por operación
 max_duration_sec: float = 3600.0 # Máximo 1 hora

```

## Eventos Disparadores

### 1. Nuevo Nodo se Une ( `on_node_join` )

| PROCESO DE REBALANCEO (NODE_JOIN)                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Nuevo nodo $N_4$ se une al cluster                                                                                                                                                                                          |
| 2. Master calcula nuevo VP-Tree con $N_4$ <ul style="list-style-type: none"><li>- <math>N_4</math> recibe un "vantage point" inicial (centroide vacío)</li></ul>                                                               |
| 3. Identificar documentos candidatos a migrar: <ul style="list-style-type: none"><li>- Docs en nodos sobrecargados (&gt;120% promedio)</li><li>- Docs cuyo VP más cercano ahora es <math>N_4</math></li></ul>                  |
| 4. Migración gradual (no disruptiva): <ul style="list-style-type: none"><li>- Priorizar docs más cercanos al nuevo VP</li><li>- Transferir en batches de 50 docs</li><li>- Mantener réplica temporal hasta confirmar</li></ul> |
| 5. Actualizar índices y vantage points                                                                                                                                                                                         |

### 2. Nodo Abandona ( `on_node_leave` )

Cuando un nodo abandona voluntariamente (shutdown graceful):

```
async def on_node_leave(self, leaving_node_id: str):
 # 1. Obtener documentos del nodo que se va
 docs_to_redistribute = self._get_documents_in_node(leaving_node_id)

 # 2. Para cada documento, encontrar mejor nodo destino
 for doc in docs_to_redistribute:
 target = self._find_best_node(doc, exclude=[leaving_node_id])
 await self._migrate_document(doc, leaving_node_id, target)

 # 3. Actualizar VP-Tree sin el nodo
 self._recompute_vantage_points()
```

### 3. Desbalance de Carga

Detectado automáticamente por el monitor:

```
async def _check_and_rebalance(self) -> None:
 """Check load and trigger rebalance if needed."""
 if self._status != RebalanceStatus.IDLE:
 return

 if self.is_in_cooldown: # Evitar rebalanceos consecutivos
```

```

 return

 needs_rebalance, reason = self.load_calculator.needs_rebalancing()

 if needs_rebalance:
 logger.info(f"Rebalance triggered: {reason}")
 await self.execute_rebalance()

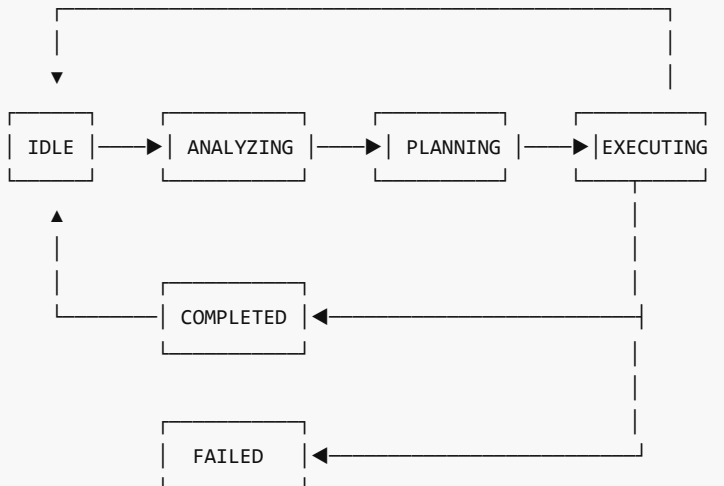
```

## Estados del Rebalanceo

```

class RebalanceStatus(Enum):
 IDLE = "idle" # Sin actividad
 ANALYZING = "analyzing" # Analizando métricas
 PLANNING = "planning" # Generando plan
 EXECUTING = "executing" # Ejecutando migraciones
 COMPLETED = "completed" # Completado
 FAILED = "failed" # Falló
 PAUSED = "paused" # Pausado manualmente

```



## Proceso de Ejecución Completo

```

async def execute_rebalance(self) -> RebalanceOperation:
 """Execute a rebalance operation."""
 self._operation_counter += 1
 operation = RebalanceOperation(
 operation_id=f"rebal_{self._operation_counter}",
 status=RebalanceStatus.ANALYZING,
 started_at=datetime.utcnow()
)

 try:
 # FASE 1: Analizar cluster

```

```

logger.info(f"Operation {operation.operation_id}: Analyzing cluster")
summary = self.load_calculator.calculate_cluster_summary()

FASE 2: Generar plan
self._status = RebalanceStatus.PLANNING
decisions = self.load_calculator.generate_rebalance_plan()

if not decisions:
 logger.info("No rebalance needed after analysis")
 operation.status = RebalanceStatus.COMPLETED
 return operation

FASE 3: Ejecutar migraciones
self._status = RebalanceStatus.EXECUTING

for decision in decisions:
 result = await self._execute_decision(decision)
 operation.documents_moved += result.documents_migrated

FASE 4: Completar
operation.status = RebalanceStatus.COMPLETED
self._last_rebalance = datetime.utcnow() # Inicia cooldown

except Exception as e:
 operation.status = RebalanceStatus.FAILED
 operation.error_message = str(e)

return operation

```

## Métricas de Carga ( LoadCalculator )

```

@dataclass
class LoadMetrics:
 """Load metrics for a single node."""
 node_id: str
 document_count: int
 capacity: int
 storage_used_bytes: int = 0
 cpu_usage: float = 0.0
 memory_usage: float = 0.0
 query_rate: float = 0.0 # queries/segundo
 avg_latency_ms: float = 0.0
 is_healthy: bool = True

 @property
 def load_factor(self) -> float:
 """Document load factor (0-1)."""
 return self.document_count / self.capacity if self.capacity > 0 else 1.0

 @property

```

```
def load_level(self) -> LoadLevel:
 """Categorize load level."""
 load = self.load_factor
 if load <= 0:
 return LoadLevel.EMPTY
 elif load < 0.4:
 return LoadLevel.LOW
 elif load < 0.75:
 return LoadLevel.NORMAL
 elif load < 0.9:
 return LoadLevel.HIGH
 else:
 return LoadLevel.CRITICAL
```

## Niveles de Carga

| Nivel    | Rango  | Acción                             |
|----------|--------|------------------------------------|
| EMPTY    | 0%     | Candidato a recibir documentos     |
| LOW      | <40%   | Puede recibir más documentos       |
| NORMAL   | 40-75% | Estado ideal                       |
| HIGH     | 75-90% | Considerar migrar documentos       |
| CRITICAL | >90%   | Migración urgente (prioridad alta) |

## Decisión de Rebalanceo

```
def needs_rebalancing(self) -> Tuple[bool, str]:
 """Determine if cluster needs rebalancing."""
 summary = self.calculate_cluster_summary()

 if summary.node_count < 2:
 return False, "Insufficient nodes for rebalancing"

 if summary.healthy_nodes < 2:
 return False, "Insufficient healthy nodes"

 # Nodos en estado crítico → rebalanceo inmediato
 if summary.critical_nodes:
 return True, f"Critical load on nodes: {summary.critical_nodes}"

 # Alta desviación estándar → desbalance
 if summary.load_std_dev > self.imbalance_threshold: # 0.2
 return True, f"Load imbalance detected (std_dev={summary.load_std_dev:.3f})"

 # Alta ratio de desbalance
 if summary.imbalance_ratio > 0.5:
 return True, f"High imbalance ratio: {summary.imbalance_ratio:.2f}"
```

```
return False, "Cluster is balanced"
```

## Ejemplo Práctico

### Situación Inicial:

Nodo\_1: 3000 docs / 5000 cap = 60% (NORMAL)  
Nodo\_2: 4500 docs / 5000 cap = 90% (CRITICAL)  
Nodo\_3: 1500 docs / 5000 cap = 30% (LOW)  
Nodo\_4: 0 docs / 5000 cap = 0% (EMPTY) ← Nuevo

### Análisis:

Total docs: 9000  
Promedio ideal:  $9000 / 4 = 2250$  docs/nodo  
 $\text{std\_dev} = 1.56 > 0.2 \rightarrow$  NECESITA REBALANCEO

### Plan de Migración:

1. Nodo\_2 → Nodo\_4: 2250 docs (prioridad ALTA - crítico)
2. Nodo\_1 → Nodo\_4: 750 docs (prioridad NORMAL)
3. Nodo\_1 → Nodo\_3: 0 docs (ya equilibrado)

### Resultado:

Nodo\_1: 2250 docs (45% - NORMAL)  
Nodo\_2: 2250 docs (45% - NORMAL)  
Nodo\_3: 2250 docs (45% - NORMAL)  
Nodo\_4: 2250 docs (45% - NORMAL)

### Navegación:

- [← README](#)
- [→ Siguiente: Power of Two Choices](#)

## Power of Two Choices

### Fundamento Teórico

"**The Power of Two Choices**" es un algoritmo de balanceo de carga que reduce drásticamente la varianza en comparación con la asignación aleatoria simple. Fue introducido por Azar, Broder, Karlin y Upfal en 1994.

### Intuición

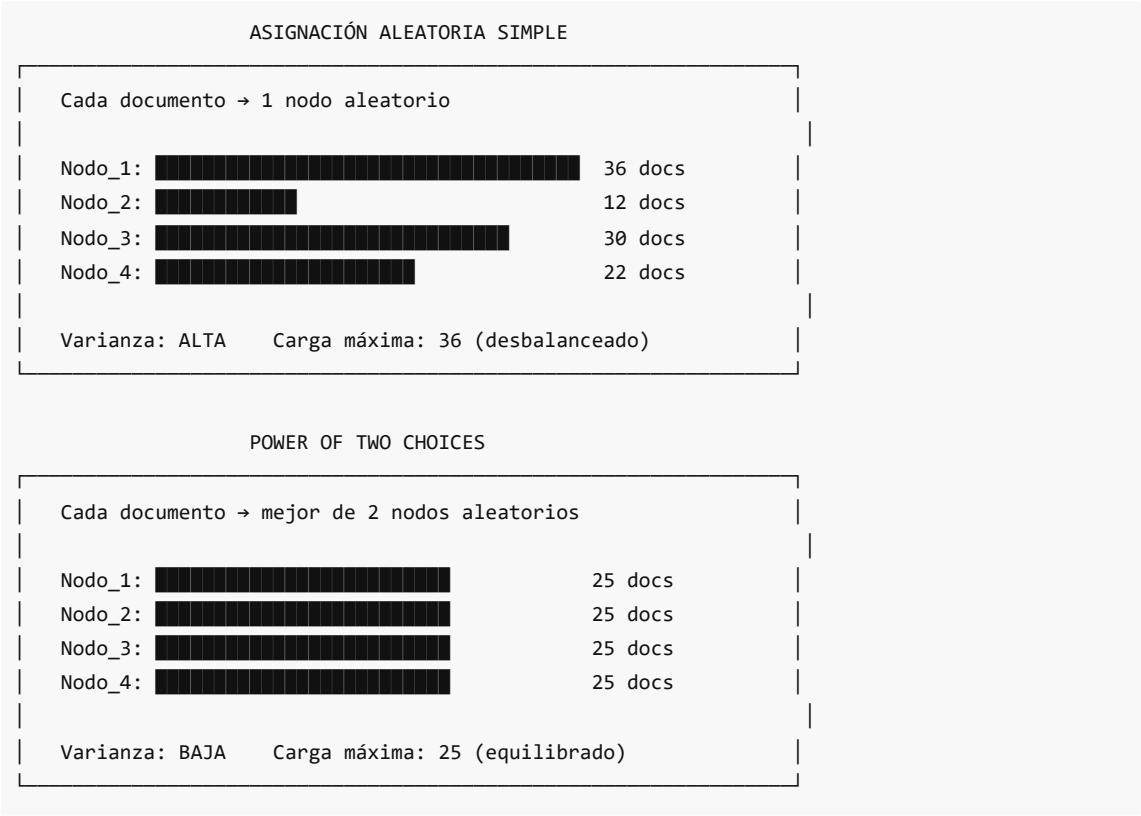
En lugar de elegir un nodo al azar para colocar un documento:

1. Seleccionar **2 nodos candidatos** al azar
2. Elegir el **menos cargado** de los dos



Este simple cambio reduce la carga máxima de  $O(\log n / \log \log n)$  a  $O(\log \log n)$ .

### Comparación: Random vs Power of Two



### Complejidad Matemática

| Estrategia | Carga Máxima Esperada                | Varianza |
|------------|--------------------------------------|----------|
| Random     | $\Theta(\frac{\log n}{\log \log n})$ | Alta     |
| Power of 2 | $\Theta(\log \log n)$                | Baja     |
| Power of d | $\Theta(\frac{\log \log n}{\log d})$ | Muy baja |

Para  $n = 10^6$  nodos:

- Random: ~5-6 veces el promedio
- Power of 2: ~2-3 veces el promedio

### Implementación en DistriSearch

DistriSearch combina Power of Two Choices con **afinidad semántica**:

```
Del reporte técnico (report.tex)
def select_migration_target(doc, cluster):
 """
 Power of Two Choices + Afinidad Semántica
```

```

"""
1. Seleccionar 2 nodos candidatos al azar
choice1, choice2 = random.sample(cluster.nodes, 2)

2. Calcular score combinado para cada candidato
score1 = calculate_combined_score(doc, choice1)
score2 = calculate_combined_score(doc, choice2)

3. Elegir el mejor
return choice1 if score1 > score2 else choice2

def calculate_combined_score(doc, node):
 """
 Score combinado: afinidad semántica + carga inversa

 α = 0.6 (peso afinidad)
 """
 alpha = 0.6

 # Afinidad semántica: coseno entre doc y centroide del nodo
 affinity = cosine_similarity(doc.vector, node.centroid)

 # Carga inversa (menor carga = mejor score)
 load = node.document_count / node.capacity
 load_score = 1 - load

 return alpha * affinity + (1 - alpha) * load_score

```

## Fórmulas del Score

### Score de Afinidad

$$\text{affinity}(\text{doc}, \text{node}) = \cos(\mathbf{v}(\text{doc}), \text{centroid}(\text{node})) = \frac{\mathbf{v}(\text{doc}) \cdot \text{centroid}(\text{node})}{\|\mathbf{v}(\text{doc})\| \cdot \|\text{centroid}(\text{node})\|}$$

### Score de Carga

$$\text{load\_score}(\text{node}) = 1 - \frac{\text{document\_count}}{\text{capacity}}$$

### Score Combinado

$$\text{score}(\text{doc}, \text{node}) = \alpha \cdot \text{affinity} + (1 - \alpha) \cdot \text{load\_score}$$

Donde  $\alpha = 0.6$  prioriza la afinidad semántica sobre la carga.

## Algoritmo de Rebalanceo con Power of Two Choices

Algorithm: Rebalance with Power of Two Choices

---

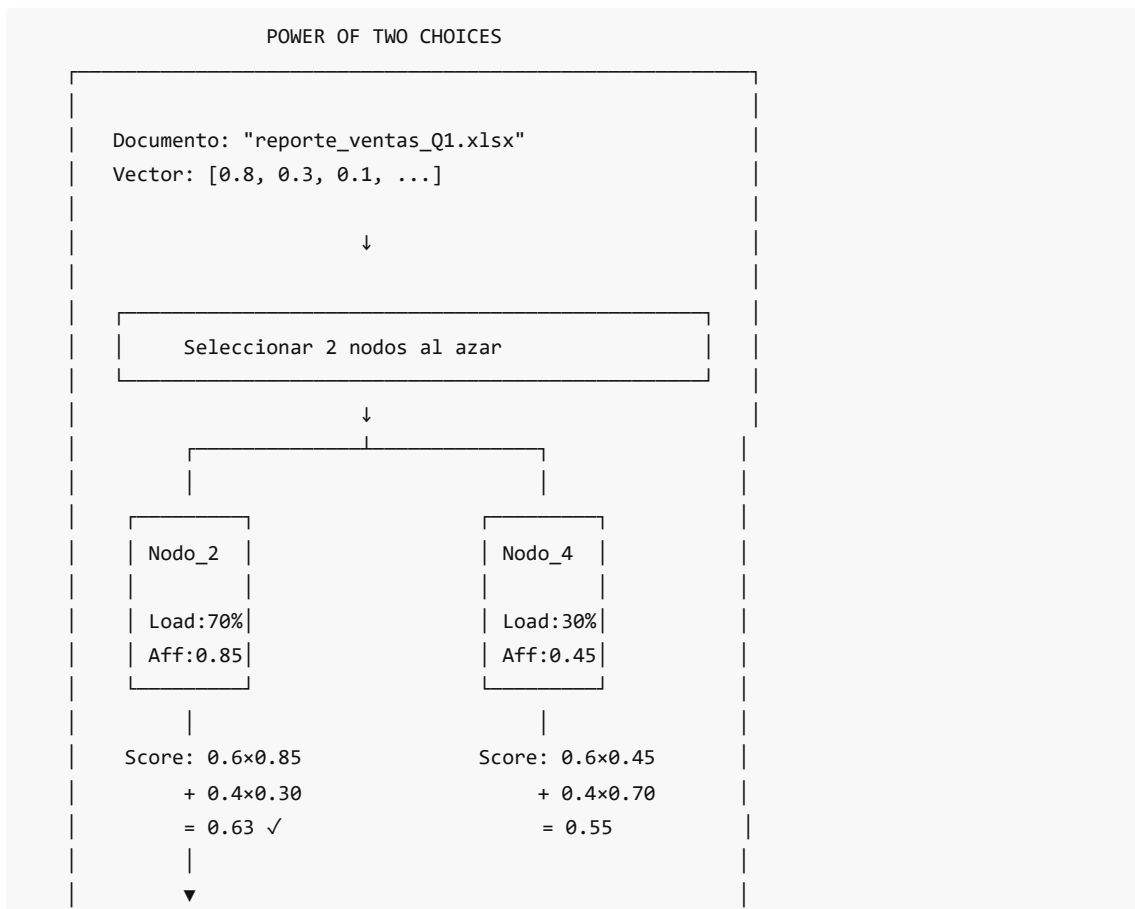
INPUT: cluster, event (NODE\_JOIN | NODE\_LEAVE | IMBALANCE)

OUTPUT: cluster rebalanceado

```
1. IF event.type = NODE_JOIN:
2. overloaded ← GetOverloadedNodes(cluster) // >120% promedio
3. FOR EACH node IN overloaded:
4. docs_to_migrate ← SelectMigrationCandidates(node)
5. FOR EACH doc IN docs_to_migrate:
6. choice1, choice2 ← RandomSample(cluster.nodes, 2)
7. score1 ← $\alpha \cdot \text{affinity}(\text{doc}, \text{choice1}) + (1-\alpha) \cdot (1-\text{load}(\text{choice1}))$
8. score2 ← $\alpha \cdot \text{affinity}(\text{doc}, \text{choice2}) + (1-\alpha) \cdot (1-\text{load}(\text{choice2}))$
9. best ← choice1 IF score1 > score2 ELSE choice2
10. Migrate(doc, node, best)
11. END FOR
12. END FOR

13. ELSE IF event.type = NODE_LEAVE:
14. orphan_docs ← event.node.documents
15. FOR EACH doc IN orphan_docs:
16. target ← FindBestNode(doc, cluster) // Por afinidad
17. RestoreFromReplica(doc, target)
18. END FOR
19. END IF
```

## Ejemplo Visual



Documento asignado a Nodo\_2 (mayor afinidad)

## Beneficios en DistriSearch

### 1. Reducción de Varianza

- Distribución más uniforme de documentos
- Menos "hotspots" (nodos sobrecargados)

### 2. Preservación de Localidad Semántica

- Documentos similares tienden a quedar juntos
- Búsquedas más eficientes (menos nodos a consultar)

### 3. Balanceo Adaptativo

- Considera carga actual, no solo capacidad
- Se adapta a patrones de uso

### 4. Simplicidad

- Solo 2 comparaciones por documento
- Overhead mínimo vs random simple

## Implementación en LoadCalculator

```
backend/app/core/rebalancing/load_calculator.py

def get_migration_candidates(
 self,
 source_node: str,
 count: int
) -> Dict[str, Any]:
 """Get information for selecting migration candidates."""
 return {
 "source_node": source_node,
 "count": count,
 "criteria": {
 # Prefer documents that:
 # 1. Have low access frequency
 # 2. Are semantically distant from node's centroid
 # 3. Have recently completed replication
 "prefer_low_access": True,
 "prefer_semantic_outliers": True, # Docs en el "borde"
 "require_replicated": True # Seguridad
 }
 }
```

## Selección de Candidatos a Migrar

Para Power of Two Choices, no todos los documentos son candidatos. Se priorizan:

1. **Documentos "frontera"**: Los más lejanos del centroide del nodo actual
2. **Documentos poco accedidos**: Migrarlos tiene menor impacto
3. **Documentos ya replicados**: Menor riesgo de pérdida

```
def _get_documents_to_migrate(
 self,
 node_id: str,
 target_count: int
) -> List[Document]:
 """Selecciona documentos para migrar usando criterio de frontera."""
 docs = self.partition_index.get_documents_in_node(node_id)
 vp = self.vantage_points[node_id]

 # Ordenar por distancia al VP (los más lejanos son candidatos)
 docs.sort(key=lambda d: d.compute_distance(vp), reverse=True)

 return docs[:target_count]
```

---

### Navegación:

- [← Anterior: Rebalanceo Activo](#)
  - [→ Siguiente: Migración de Documentos](#)
- 

## Migración de Documentos

### Concepto

La **migración de documentos** es el proceso de transferir documentos de un nodo a otro durante el rebalanceo. DistriSearch implementa un sistema de migración robusto con:

- **Transferencias en batch** (50 docs/batch)
- **Rate limiting** (1 segundo entre batches)
- **Reintentos automáticos**
- **Seguimiento de progreso**

### Clase MigrationHandler

**Ubicación:** backend/app/core/rebalancing/migration\_handler.py

```
class MigrationHandler:
 """
 Handles document migrations between cluster nodes.

 Implements:
 - Batch transfers (50 docs/batch per architecture spec)
```

```

- Rate limiting (1s sleep between batches)
- Progress tracking
- Retry logic
- Cancellation support
"""

def __init__(
 self,
 config: Optional[MigrationConfig] = None,
 transfer_func: Callable[[str, str, List[str]], Awaitable[Dict]] = None
):
 self.config = config or MigrationConfig()
 self._transfer_func = transfer_func
 self._tasks: Dict[str, MigrationTask] = {}

```

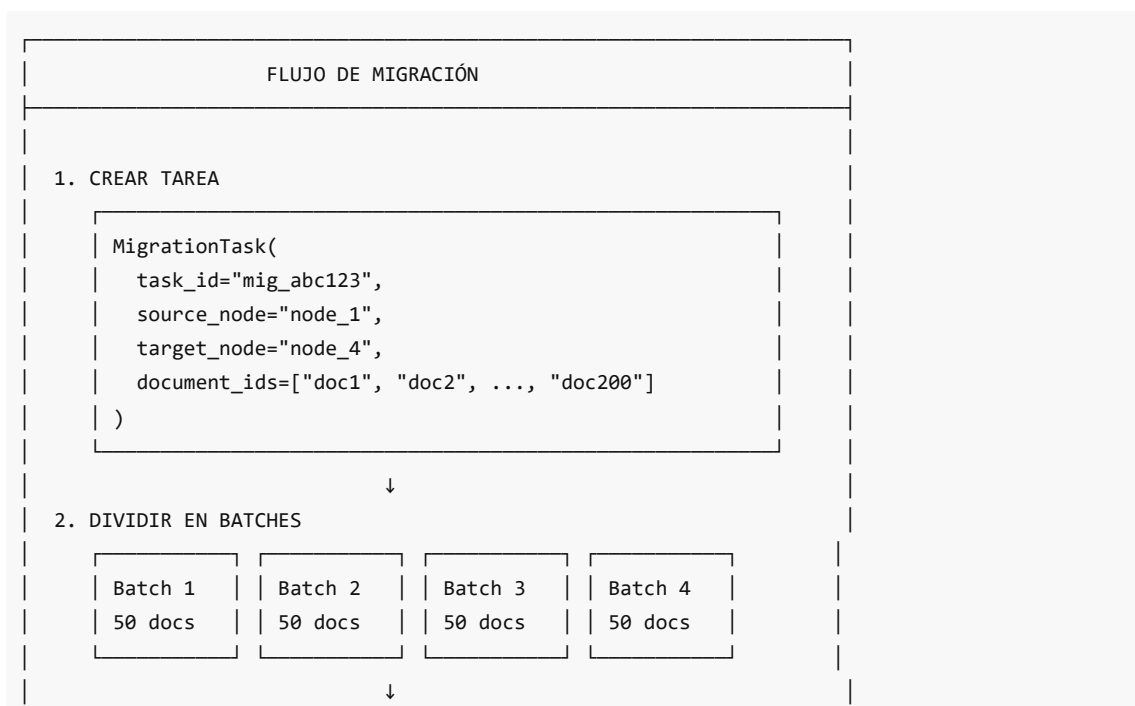
## Configuración de Migración

```

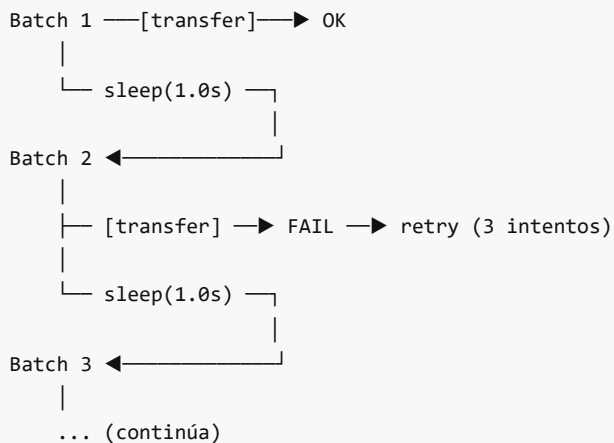
@dataclass
class MigrationConfig:
 """Configuration for migration operations."""
 batch_size: int = 50 # docs por batch (según especificación)
 batch_delay_sec: float = 1.0 # sleep entre batches
 max_concurrent_batches: int = 1 # una migración a la vez
 max_retries: int = 3 # reintentos por batch fallido
 retry_delay_sec: float = 5.0 # espera antes de reintentar
 transfer_timeout_sec: float = 30.0 # timeout por transferencia

```

## Proceso de Migración Paso a Paso



### 3. EJECUTAR BATCH + RATE LIMITING



### 4. RESULTADO FINAL

```
MigrationResult(
 success=True,
 documents_migrated=195,
 documents_failed=5,
 duration_sec=8.5
)
```

## Estructura de la Tarea de Migración

```
@dataclass
class MigrationTask:
 """Represents a document migration task."""
 task_id: str
 source_node: str
 target_node: str
 document_ids: List[str]
 status: MigrationStatus = MigrationStatus.PENDING
 created_at: datetime = field(default_factory=datetime.utcnow)
 started_at: Optional[datetime] = None
 completed_at: Optional[datetime] = None
 progress: float = 0.0 # 0.0 a 1.0
 documents_migrated: int = 0
 documents_failed: int = 0
 retry_count: int = 0
 max_retries: int = 3

 @property
 def total_documents(self) -> int:
 return len(self.document_ids)
```

```

@property
def is_complete(self) -> bool:
 return self.status in (
 MigrationStatus.COMPLETED,
 MigrationStatus.FAILED,
 MigrationStatus.CANCELLED
)

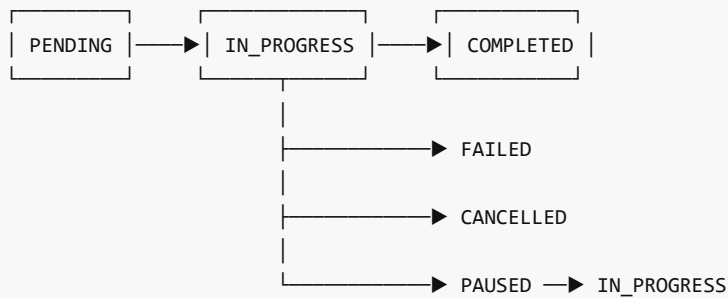
```

## Estados de Migración

```

class MigrationStatus(Enum):
 PENDING = "pending" # Creada, esperando ejecución
 IN_PROGRESS = "in_progress" # Ejecutándose
 COMPLETED = "completed" # Terminada exitosamente
 FAILED = "failed" # Falló (agotó reintentos)
 CANCELLED = "cancelled" # Cancelada manualmente
 PAUSED = "paused" # Pausada

```



## Ejecución con Rate Limiting

```

async def _execute_migration(self, task: MigrationTask) -> MigrationResult:
 """Execute the actual migration with batching and rate limiting."""
 failed_docs = []

 # Dividir en batches de 50
 batches = [
 task.document_ids[i:i + self.config.batch_size]
 for i in range(0, len(task.document_ids), self.config.batch_size)
]

 total_batches = len(batches)
 logger.info(f"Task {task.task_id}: {total_batches} batches")

 for batch_idx, batch in enumerate(batches):
 # Verificar cancelación
 if task.task_id in self._cancelled:
 task.status = MigrationStatus.CANCELLED

```



```

 break

Ejecutar transferencia del batch
try:
 batch_result = await self._transfer_batch(
 task.source_node,
 task.target_node,
 batch
)

 task.documents_migrated += len(batch_result.get("migrated", []))
 task.documents_failed += len(batch_result.get("failed", []))

except Exception as e:
 # Lógica de reintentos
 if task.retry_count < task.max_retries:
 task.retry_count += 1
 await asyncio.sleep(self.config.retry_delay_sec) # 5s
 # Reintentar...
 else:
 task.documents_failed += len(batch)

Actualizar progreso
task.progress = (batch_idx + 1) / total_batches

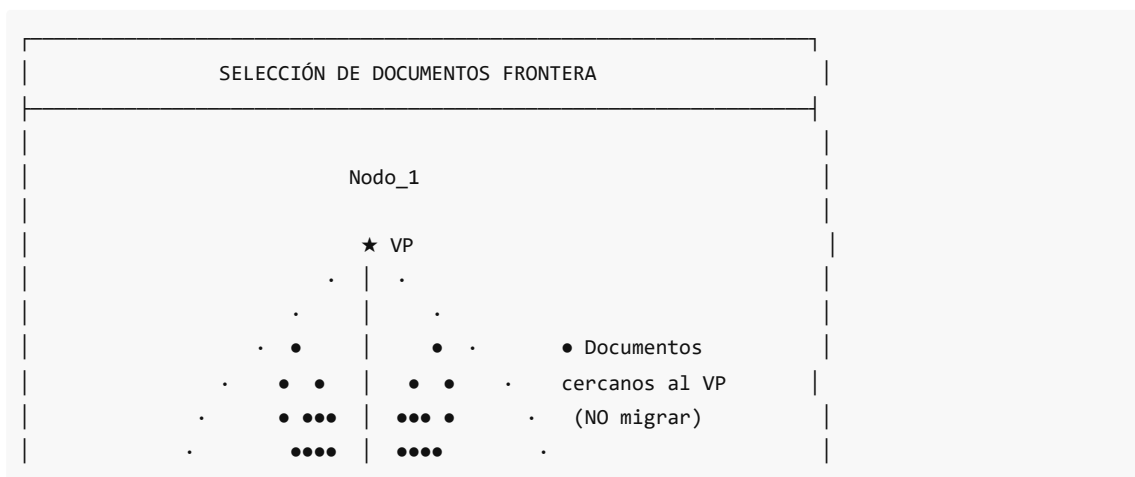
RATE LIMITING: sleep entre batches
if batch_idx < total_batches - 1:
 await asyncio.sleep(self.config.batch_delay_sec) # 1s

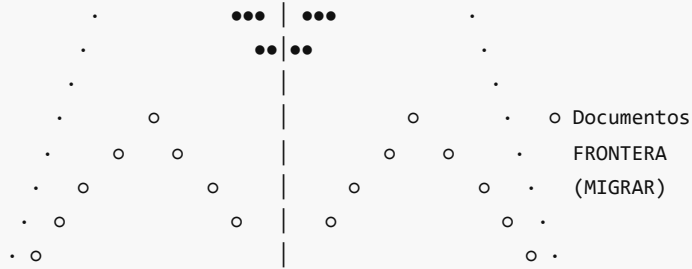
Finalizar
task.completed_at = datetime.utcnow()
return MigrationResult(...)

```

## Selección de Documentos "Frontera"

Los documentos candidatos a migración son aquellos en el **borde** de la partición: los más alejados del vantage point actual.





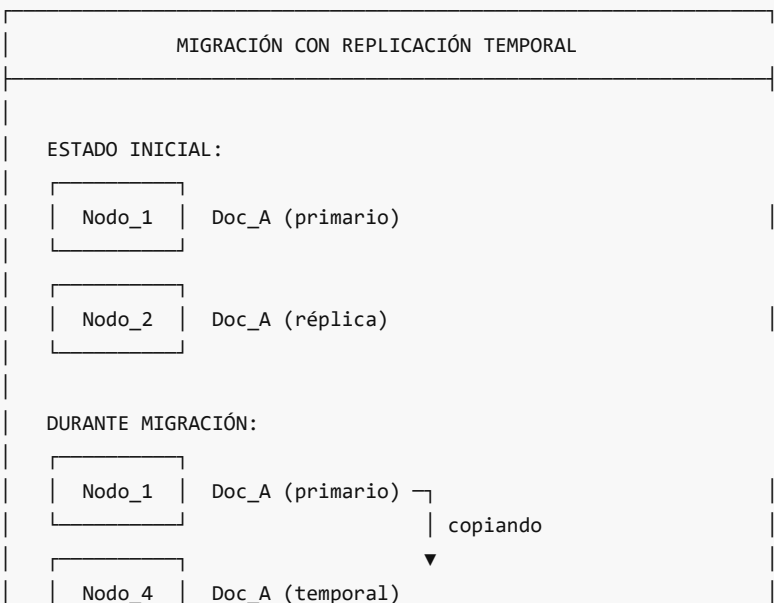
```
def _get_documents_to_migrate(
 self,
 node_id: str,
 target_count: int,
 prefer_similar_to_new_vp: bool = True
) -> List[Document]:
 """Selecciona documentos para migrar usando criterio de frontera."""
 docs = self.partition_index.get_documents_in_node(node_id)
 vp = self.vantage_points[node_id]

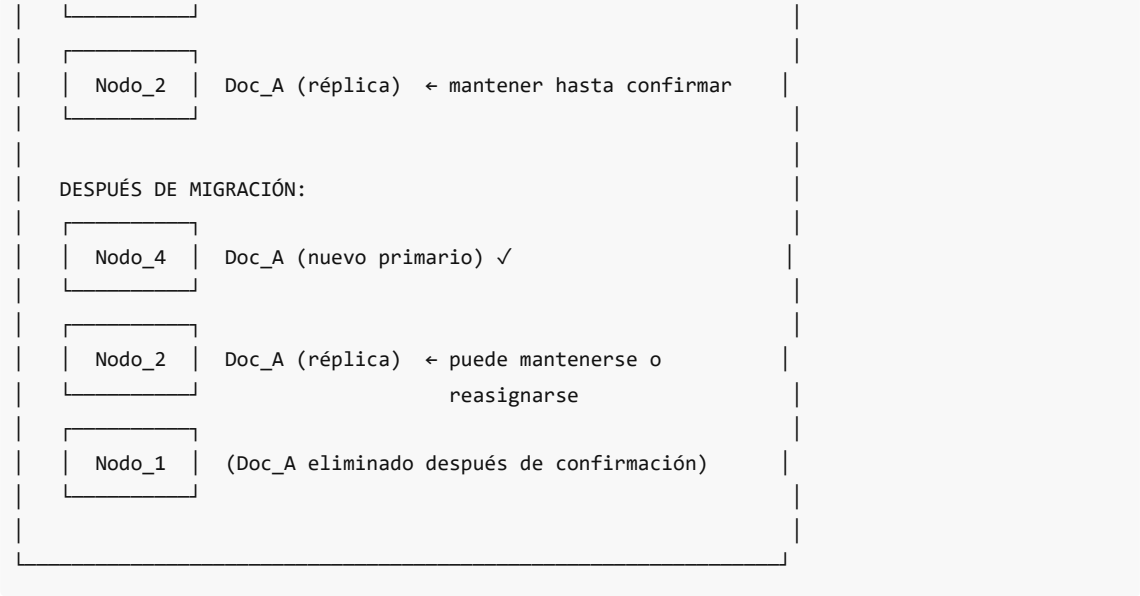
 # Ordenar por distancia al VP (los más lejanos son candidatos)
 docs.sort(key=lambda d: d.compute_distance(vp), reverse=True)

 return docs[:target_count]
```

## Replicación Temporal Durante Transferencia

Para garantizar disponibilidad durante la migración:





## Actualización de Índices

Después de cada migración exitosa:

1. **Actualizar VP-Tree:** El documento ahora pertenece a otra partición
2. **Actualizar routing table:** Las consultas deben ir al nuevo nodo
3. **Invalidar caché:** Si hay búsquedas cacheadas que incluían el documento
4. **Notificar réplicas:** Las réplicas deben saber la nueva ubicación

## Impacto en Disponibilidad de Búsqueda

| Fase               | Disponibilidad | Notas                    |
|--------------------|----------------|--------------------------|
| Pre-migración      | 100%           | Normal                   |
| Durante copia      | 100%           | Doc disponible en origen |
| Verificación       | 100%           | Doc en ambos nodos       |
| Eliminación origen | 100%           | Doc solo en destino      |

La búsqueda nunca se interrumpe gracias a la replicación temporal.

## Estadísticas de Migración

```
def get_statistics(self) -> Dict[str, Any]:
 """Get migration statistics."""
 active = self.get_active_tasks()
 completed = [t for t in self._tasks.values()
 if t.status == MigrationStatus.COMPLETED]
 failed = [t for t in self._tasks.values()
 if t.status == MigrationStatus.FAILED]

 return {
```

```

 "total_tasks": len(self._tasks),
 "active_tasks": len(active),
 "completed_tasks": len(completed),
 "failed_tasks": len(failed),
 "total_documents_migrated": self._total_migrated,
 "total_documents_failed": self._total_failed,
 "success_rate": self._total_migrated /
 (self._total_migrated + self._total_failed)
 }

```

## Ejemplo de Migración Completa

Ejemplo: Migrar 200 documentos de Nodo\_1 a Nodo\_4

---

Configuración:

```

batch_size = 50
batch_delay = 1.0s
max_retries = 3

```

Ejecución:

```

Batch 1: docs[0:50] → OK (0.5s)
[sleep 1.0s]
Batch 2: docs[50:100] → FAIL → retry → OK (5.5s + 0.5s)
[sleep 1.0s]
Batch 3: docs[100:150] → OK (0.5s)
[sleep 1.0s]
Batch 4: docs[150:200] → OK (0.5s)

```

Tiempo total: 0.5 + 1 + 6 + 1 + 0.5 + 1 + 0.5 = ~10.5 segundos

Resultado:

```

✓ 200 documentos migrados
✓ 0 documentos fallidos
✓ 1 reintento realizado

```

---

Navegación:

- [← Anterior: Power of Two Choices](#)
- [→ Siguiente: Replicación](#)

## 05 Replicacion

### 05. Replicación con Afinidad Semántica

#### Visión General

DistriSearch implementa un sistema de **replicación inteligente** que no solo garantiza tolerancia a fallos, sino que también optimiza el rendimiento de búsquedas colocando réplicas en nodos que contienen documentos

semánticamente similares.

## ¿Por Qué Replicar?

SIN REPLICACIÓN

Nodo\_1: [Doc\_A] [Doc\_B] [Doc\_C]  
Nodo\_2: [Doc\_D] [Doc\_E] [Doc\_F]  
Nodo\_3: [Doc\_G] [Doc\_H] [Doc\_I]  
  
⚠ Si Nodo\_1 falla → Doc\_A, Doc\_B, Doc\_C PERDIDOS

CON REPLICACIÓN (factor=2)

Nodo\_1: [Doc\_A●] [Doc\_B●] [Doc\_C●] [Doc\_Do] [Doc\_Eo]  
Nodo\_2: [Doc\_D●] [Doc\_E●] [Doc\_F●] [Doc\_Ao] [Doc\_Bo]  
Nodo\_3: [Doc\_G●] [Doc\_H●] [Doc\_I●] [Doc\_Co] [Doc\_Fo]  
  
● = Primario    ○ = Réplica  
  
✓ Si Nodo\_1 falla → Doc\_A, Doc\_B, Doc\_C disponibles en réplicas

## Replicación con Afinidad Semántica

La innovación de DistriSearch es colocar réplicas **estratégicamente**:

REPLICACIÓN TRADICIONAL (Random/Round-Robin)

ventas\_q1.xlsx (primario: Nodo\_1, réplica: Nodo\_3)  
ventas\_q2.xlsx (primario: Nodo\_2, réplica: Nodo\_1)  
ventas\_q3.xlsx (primario: Nodo\_3, réplica: Nodo\_2)  
  
Búsqueda "ventas" → consultar 3 nodos

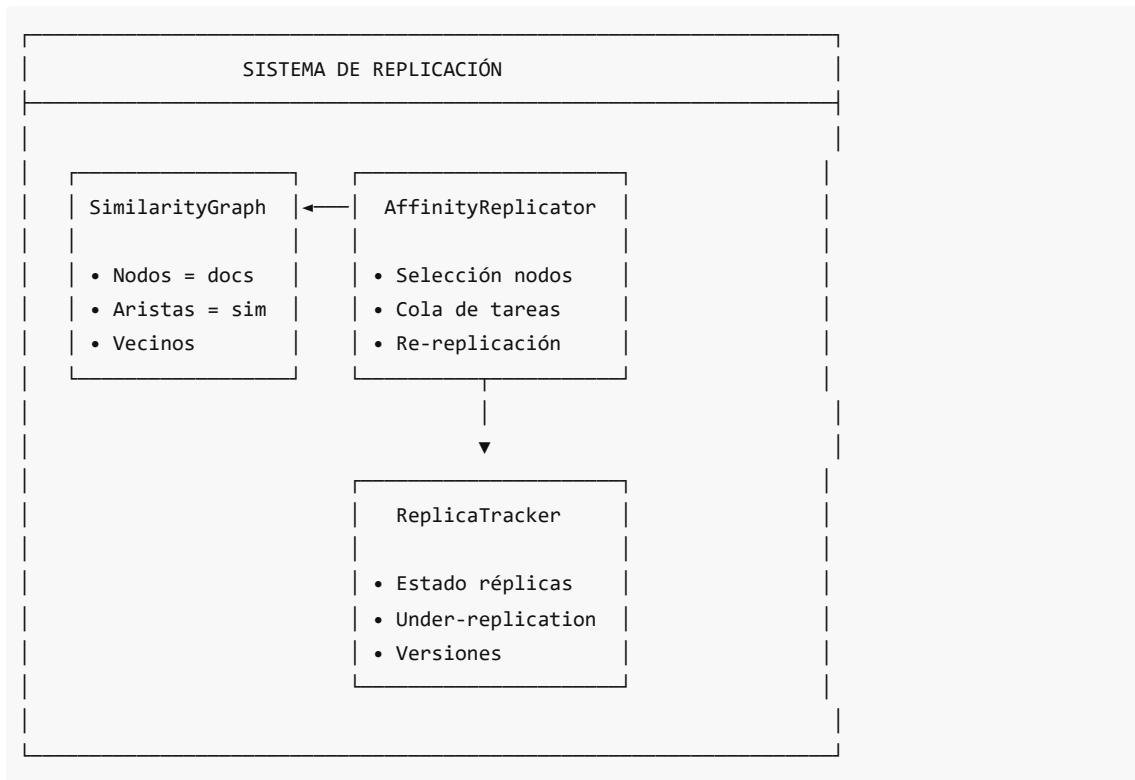
REPLICACIÓN CON AFINIDAD SEMÁNTICA

Nodo\_1: ventas\_q1● ventas\_q2○ ventas\_q3○ ← Cluster semántico

```
Nodo_2: marketing_q1• marketing_q2o
Nodo_3: rrhh_contratos• rrhh_nominaso

Búsqueda "ventas" → consultar solo Nodo_1 (tiene todo)
```

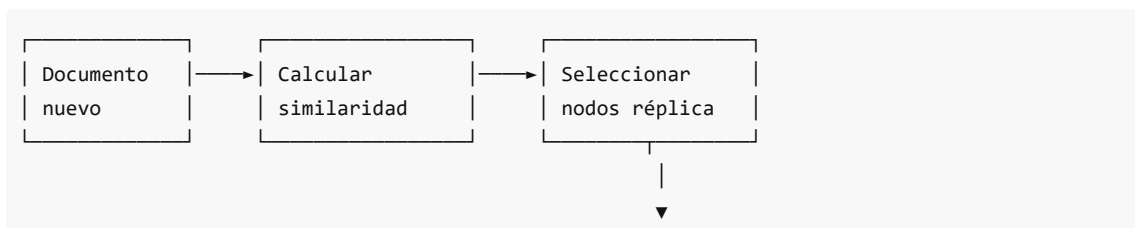
## Arquitectura del Sistema de Replicación

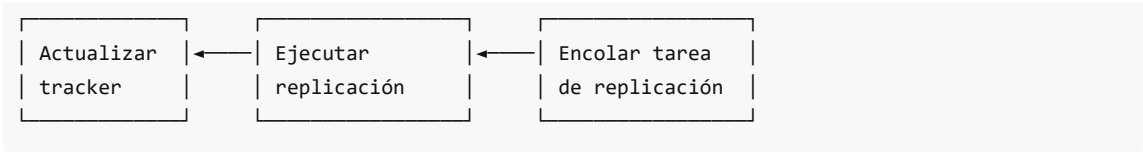


## Componentes Principales

| Componente         | Archivo                | Responsabilidad                      |
|--------------------|------------------------|--------------------------------------|
| AffinityReplicator | affinity_replicator.py | Orquesta la replicación con afinidad |
| SimilarityGraph    | similarity_graph.py    | Grafo de similitud entre documentos  |
| ReplicaTracker     | replica_tracker.py     | Seguimiento del estado de réplicas   |

## Flujo de Replicación





## Fórmulas Clave

### Score de Afinidad de Nodo

$$\text{affinity}(\text{doc}, \text{node}) = \sum_{\text{neighbor} \in \text{node.docs}} \text{similarity}(\text{doc}, \text{neighbor})$$

### Selección de Nodo Réplica

$$\text{best\_node} = \arg\max_{\text{node} \in \text{candidates}} \text{affinity}(\text{doc}, \text{node})$$

## Contenido de Esta Sección

- [Afinidad Semántica](#): Por qué y cómo usar afinidad
- [Grafo de Similitud](#): Estructura del grafo
- [Factor de Replicación](#): Configuración y trade-offs

### Navegación:

- [← Anterior: Rebalanceo](#)
- [→ Siguiente: Tolerancia a Fallos](#)

# Afinidad Semántica en Replicación

## Concepto

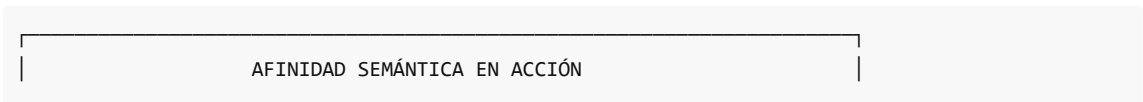
La **afinidad semántica** es el principio de colocar réplicas de documentos en nodos que ya contienen documentos similares. Esto crea "clusters semánticos" naturales que mejoran el rendimiento de búsqueda.

## ¿Por Qué Afinidad Semántica?

### Beneficios

| Beneficio                 | Descripción                                    |
|---------------------------|------------------------------------------------|
| Mejor localidad de caché  | Documentos relacionados comparten caché        |
| Búsquedas más rápidas     | Menos nodos a consultar para queries similares |
| Menor tráfico de red      | Resultados encontrados localmente              |
| Mejor tolerancia a fallos | Documentos relacionados sobreviven juntos      |

### Ejemplo Visual



#### DOCUMENTOS:

- ventas\_q1.xlsx (tema: ventas, finanzas)
- ventas\_q2.xlsx (tema: ventas, finanzas) ← Similar a q1
- marketing\_2024.pdf (tema: marketing)
- rrhh\_nominas.xlsx (tema: recursos humanos)

#### SIN AFINIDAD:

| Nodo_1     | Nodo_2     | Nodo_3     |
|------------|------------|------------|
| ventas_q1● | ventas_q2● | marketing● |
| rrhh       | marketingo | ventas_q1o |

Búsqueda "ventas trimestre" → consultar Nodo\_1 + Nodo\_2 + Nodo\_3

#### CON AFINIDAD:

| Nodo_1     | Nodo_2     | Nodo_3 |
|------------|------------|--------|
| ventas_q1● | marketing● | rrhh●  |
| ventas_q2o |            |        |

Búsqueda "ventas trimestre" → consultar solo Nodo\_1

## Clase AffinityReplicator

**Ubicación:** backend/app/core/replication/affinity\_replicator.py

```
class AffinityReplicator:
 """
 Manages document replication with semantic affinity placement.

 Features:
 - Automatic replica placement based on document similarity
 - Background replication monitoring
 - Re-replication on node failure
 - Replica synchronization
 """

 def __init__(
 self,
 config: Optional[ReplicationConfig] = None,
 replicate_func: Optional[Callable] = None,
 get_document_vectors: Optional[Callable] = None
):
 self.config = config or ReplicationConfig()

 # Componentes
 self.similarity_graph = SimilarityGraph()
```



```

self.replica_tracker = ReplicaTracker(
 default_replication_factor=self.config.replication_factor
)

Cola de tareas
self._pending_tasks: Dict[str, ReplicationTask] = {}

```

## Configuración

```

@dataclass
class ReplicationConfig:
 """Configuration for replication."""
 replication_factor: int = 2 # Número de copias totales
 max_concurrent_replications: int = 5 # Máximo paralelo
 replication_timeout_sec: float = 60.0 # Timeout por réplica
 retry_count: int = 3 # Reintentos
 retry_delay_sec: float = 5.0 # Espera entre reintentos
 sync_batch_size: int = 10 # Tamaño de batch
 check_interval_sec: float = 30.0 # Intervalo de verificación

```

## Proceso de Selección de Nodos Réplica

```

async def _select_replica_nodes(
 self,
 document_id: str,
 primary_node: str,
 document_vectors: Optional[Dict[str, Any]]
) -> List[str]:
 """
 Select nodes for replicas using semantic affinity.
 """
 num_replicas = self.config.replication_factor - 1

 if num_replicas <= 0:
 return []

 # Excluir nodo primario
 available_nodes = [n for n in self._cluster_nodes if n != primary_node]

 if not available_nodes:
 logger.warning("No available nodes for replicas")
 return []

 # Si tenemos vectores y documentos existentes, usar afinidad
 if document_vectors and len(self.similarity_graph._nodes) > 0:
 # Encontrar nodos con documentos similares
 affinity_scores = self.similarity_graph.find_best_replica_nodes(
 document_id=document_id,

```

```

 candidate_nodes=available_nodes,
 num_replicas=num_replicas
)

 if affinity_scores:
 return [node for node, _ in affinity_scores[:num_replicas]]

 # Fallback: selección round-robin
 return available_nodes[:num_replicas]

```

## Algoritmo de Selección

Algorithm: Select Replica Nodes with Affinity

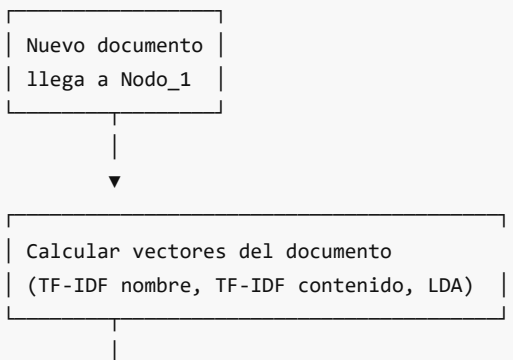
---

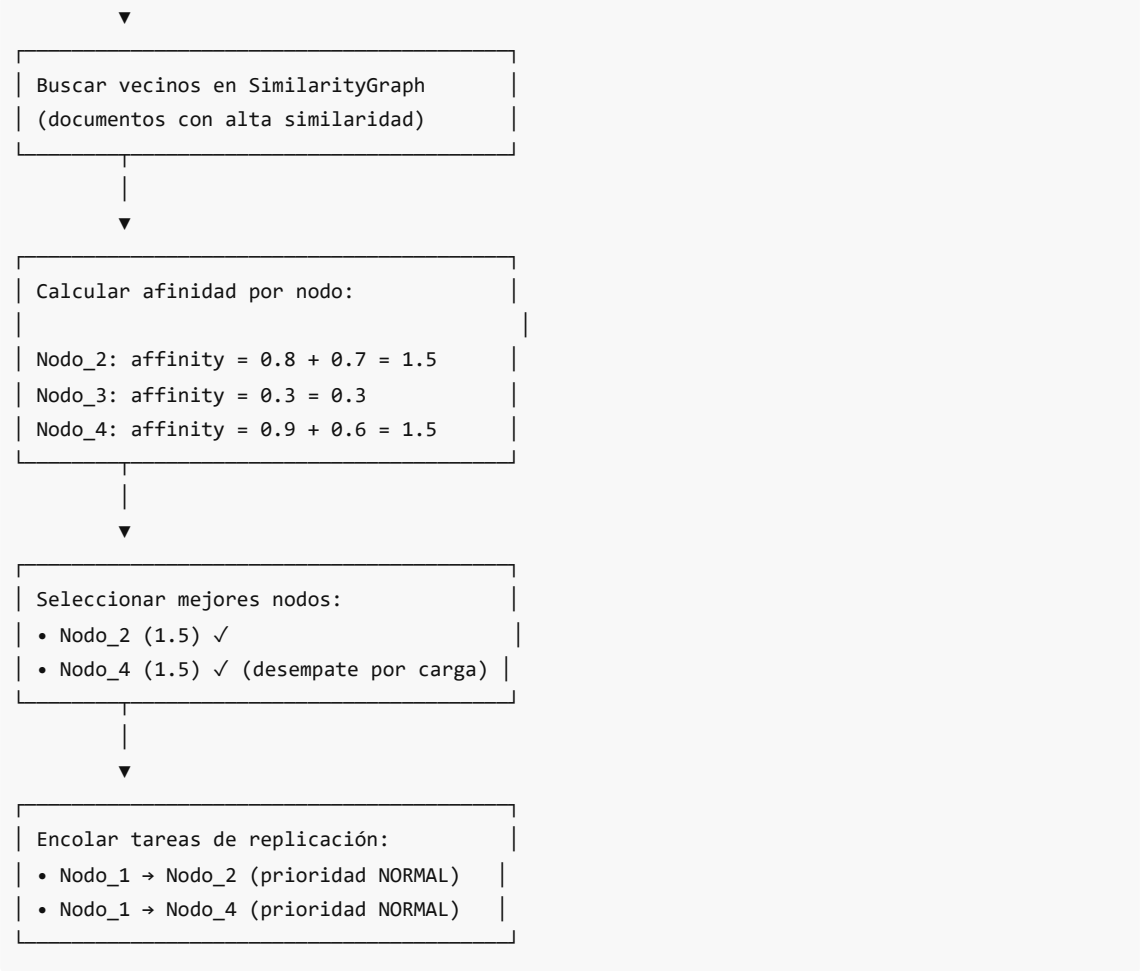
INPUT: document, primary\_node, replication\_factor

OUTPUT: lista de nodos para réplicas

1. num\_replicas  $\leftarrow$  replication\_factor - 1
2. available  $\leftarrow$  cluster\_nodes - {primary\_node}
3. IF document tiene vectores AND existe grafo de similaridad:
  4. neighbors  $\leftarrow$  get\_neighbors(document, limit=50)
  5. FOR EACH candidate IN available:
    6. affinity  $\leftarrow$  0
    7. FOR EACH (neighbor\_id, similarity) IN neighbors:
      8. IF neighbor\_id está en candidate:
        9. affinity  $\leftarrow$  affinity + similarity
    10. END FOR
    11. scores[candidate]  $\leftarrow$  affinity
    12. END FOR
    13. RETURN top num\_replicas nodos por score
  14. ELSE:
    15. RETURN available[:num\_replicas] // Fallback

## Diagrama de Flujo





## Prioridades de Replicación

```
class ReplicationPriority(Enum):
 """Priority levels for replication tasks."""
 CRITICAL = 4 # Primario perdido, sin réplicas
 HIGH = 3 # Sub-replicado (under-replicated)
 NORMAL = 2 # Documento nuevo
 LOW = 1 # Optimización en background
```

| Prioridad | Situación    | Ejemplo                    |
|-----------|--------------|----------------------------|
| CRITICAL  | Sin copias   | Nodo único falló           |
| HIGH      | Bajo factor  | 1 copia cuando necesita 2  |
| NORMAL    | Nuevo doc    | Documento recién subido    |
| LOW       | Optimización | Mover réplica a mejor nodo |

## Registro de Documento

```

async def register_document(
 self,
 document_id: str,
 primary_node: str,
 document_vectors: Optional[Dict[str, Any]] = None,
 size_bytes: int = 0,
 checksum: Optional[str] = None
) -> DocumentReplicas:
 """
 Register a new document and initiate replication.
 """
 # 1. Seleccionar nodos réplica usando afinidad
 replica_nodes = await self._select_replica_nodes(
 document_id,
 primary_node,
 document_vectors
)

 # 2. Registrar en tracker
 doc_replicas = self.replica_tracker.register_document(
 document_id=document_id,
 primary_node=primary_node,
 replica_nodes=replica_nodes,
 replication_factor=self.config.replication_factor
)

 # 3. Añadir al grafo de similaridad
 self.similarity_graph.add_document(
 document_id=document_id,
 primary_node=primary_node,
 document_vectors=document_vectors,
 replica_nodes=replica_nodes
)

 # 4. Encolar tareas de replicación
 for replica_node in replica_nodes:
 task = self._create_task(
 document_id=document_id,
 source_node=primary_node,
 target_node=replica_node,
 priority=ReplicationPriority.NORMAL
)
 self._pending_tasks[task.task_id] = task

 return doc_replicas

```

## Ventajas de la Afinidad Semántica

### 1. Localidad de Búsqueda

Consultas sobre temas específicos encuentran resultados en menos nodos.

## 2. Mejor Uso de Caché

Documentos relacionados comparten estructuras de índice.

## 3. Tolerancia a Fallos Coherente

Si un nodo falla, los documentos relacionados en otros nodos permiten respuestas parciales coherentes.

## 4. Eficiencia en Red

Menos comunicación inter-nodo para queries relacionadas.

### Navegación:

- [← README](#)
- [→ Siguiente: Grafo de Similaridad](#)

# Grafo de Similaridad

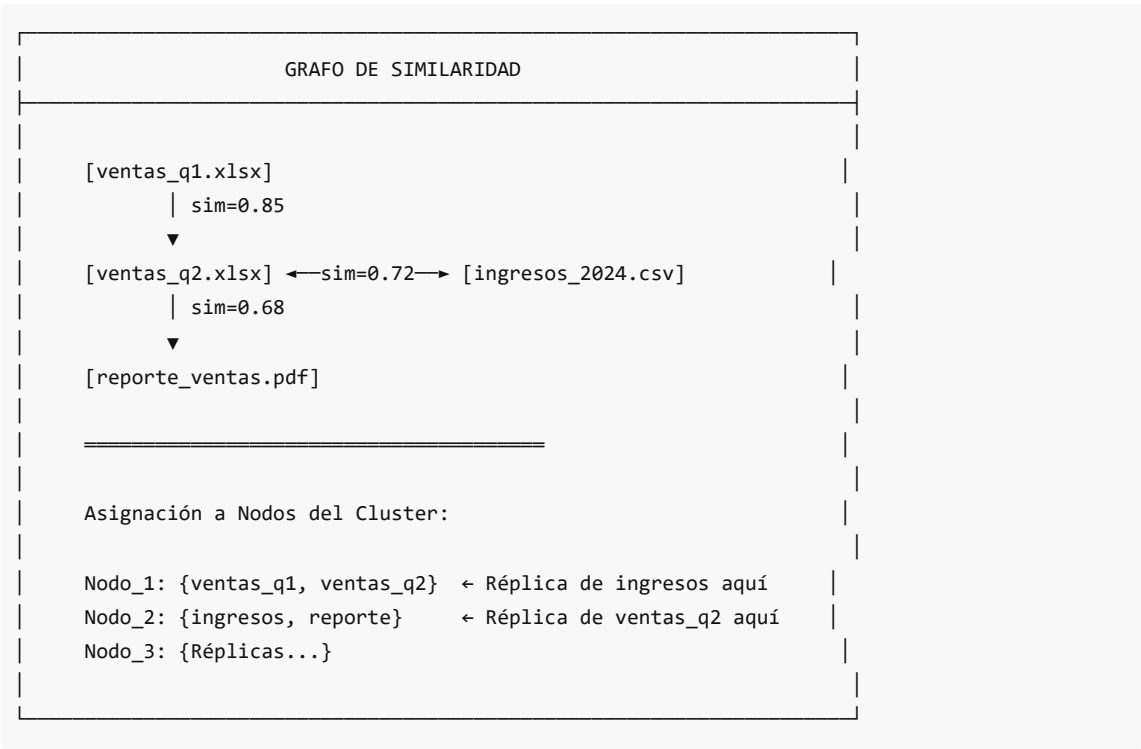
## Concepto

El **grafo de similitud** es una estructura de datos donde:

- **Nodos** = Documentos
- **Aristas** = Relaciones de similitud (peso = score de similitud)

Este grafo permite encontrar rápidamente documentos relacionados y determinar la mejor ubicación para réplicas.

## Estructura Visual



## Clase SimilarityGraph

**Ubicación:** backend/app/core/replication/similarity\_graph.py

```
class SimilarityGraph:
 """
 Graph structure tracking document similarities.

 Used to:
 - Find semantically similar documents
 - Determine optimal replica placement
 - Support nearest-neighbor queries
 """

 def __init__(
 self,
 similarity_threshold: float = 0.3,
 max_neighbors: int = 20,
 distance_func: Optional[Callable] = None
):
 self.similarity_threshold = similarity_threshold
 self.max_neighbors = max_neighbors
 self._distance_func = distance_func

 # Almacenamiento del grafo
 self._nodes: Dict[str, DocumentNode] = {}
 self._edges: Dict[str, Set[str]] = defaultdict(set)
 self._similarity_cache: Dict[Tuple[str, str], float] = {}

 # Índice para búsquedas rápidas
 self._node_to_docs: Dict[str, Set[str]] = defaultdict(set)
```

## Estructura de Datos

### Nodo del Documento

```
@dataclass
class DocumentNode:
 """Node in the similarity graph representing a document."""
 document_id: str
 primary_node: str # Nodo primario
 replica_nodes: List[str] = field(default_factory=list)
 neighbors: Dict[str, float] = field(default_factory=dict) # doc_id → sim
 created_at: datetime = field(default_factory=datetime.utcnow)
 vector_hash: Optional[str] = None # Para detectar cambios

 @property
 def all_nodes(self) -> List[str]:
```

```

 """All nodes storing this document."""
 return [self.primary_node] + self.replica_nodes

```

## Arista de Similitud

```

@dataclass
class SimilarityEdge:
 """Edge in similarity graph between two documents."""
 doc_a: str
 doc_b: str
 similarity: float = 0.0 # 0.0 a 1.0
 created_at: datetime = field(default_factory=datetime.utcnow)

```

## Operaciones del Grafo

### Añadir Documento

```

def add_document(
 self,
 document_id: str,
 primary_node: str,
 document_vectors: Optional[Dict[str, Any]] = None,
 replica_nodes: Optional[List[str]] = None
) -> DocumentNode:
 """Add a document to the graph."""
 node = DocumentNode(
 document_id=document_id,
 primary_node=primary_node,
 replica_nodes=replica_nodes or []
)

 self._nodes[document_id] = node
 self._node_to_docs[primary_node].add(document_id)

 for replica in node.replica_nodes:
 self._node_to_docs[replica].add(document_id)

 # Calcular similitudes si hay vectores
 if document_vectors and self._distance_func:
 self._update_similarities(document_id, document_vectors)

 return node

```

### Añadir Similitud

```

def add_similarity(
 self,
 doc_a: str,

```

```

 doc_b: str,
 similarity: float
) -> bool:
 """Add or update similarity between two documents."""
 # Solo crear arista si supera umbral
 if similarity < self.similarity_threshold: # 0.3
 return False

 if doc_a not in self._nodes or doc_b not in self._nodes:
 return False

 # Almacenar arista bidireccional
 self._edges[doc_a].add(doc_b)
 self._edges[doc_b].add(doc_a)

 # Cachear similaridad
 key = tuple(sorted([doc_a, doc_b]))
 self._similarity_cache[key] = similarity

 # Actualizar listas de vecinos
 self._nodes[doc_a].neighbors[doc_b] = similarity
 self._nodes[doc_b].neighbors[doc_a] = similarity

 # Podar si hay demasiados vecinos
 self._prune_neighbors(doc_a)
 self._prune_neighbors(doc_b)

 return True

```

## Obtener Vecinos

```

def get_neighbors(
 self,
 document_id: str,
 limit: int = 10
) -> List[Tuple[str, float]]:
 """Get most similar neighbors of a document."""
 node = self._nodes.get(document_id)
 if not node:
 return []

 # Ordenar por similaridad descendente
 sorted_neighbors = sorted(
 node.neighbors.items(),
 key=lambda x: x[1],
 reverse=True
)

 return sorted_neighbors[:limit]

```



## Encontrar Mejores Nodos para Réplicas

```
def find_best_replica_nodes(
 self,
 document_id: str,
 candidate_nodes: List[str],
 num_replicas: int = 1,
 exclude_primary: bool = True
) -> List[Tuple[str, float]]:
 """
 Find best nodes for replicas based on semantic affinity.

 Args:
 document_id: Document to replicate
 candidate_nodes: Available cluster nodes
 num_replicas: Number of replicas needed

 Returns:
 List of (node_id, affinity_score) tuples
 """
 node = self._nodes.get(document_id)
 if not node:
 return []

 # Obtener vecinos del documento
 neighbors = self.get_neighbors(document_id, limit=50)

 # Puntuar cada nodo candidato
 node_scores: Dict[str, float] = {}

 for candidate in candidate_nodes:
 if exclude_primary and candidate == node.primary_node:
 continue
 if candidate in node.replica_nodes:
 continue

 # Calcular afinidad: suma de similaridades con docs en este nodo
 node_docs = self.get_documents_on_node(candidate)

 affinity = 0.0
 for neighbor_id, similarity in neighbors:
 if neighbor_id in node_docs:
 affinity += similarity

 node_scores[candidate] = affinity

 # Ordenar por afinidad descendente
 sorted_nodes = sorted(
 node_scores.items(),
 key=lambda x: x[1],
```

```

 reverse=True
)

 return sorted_nodes[:num_replicas]

```

## Ejemplo de Cálculo de Afinidad

Ejemplo: Buscar mejor nodo para réplica de "reporte\_ventas.pdf"

Vecinos de reporte\_ventas.pdf:

- ventas\_q1.xlsx: sim=0.75
- ventas\_q2.xlsx: sim=0.70
- ingresos\_2024.csv: sim=0.60
- marketing.pdf: sim=0.35

Documentos por nodo:

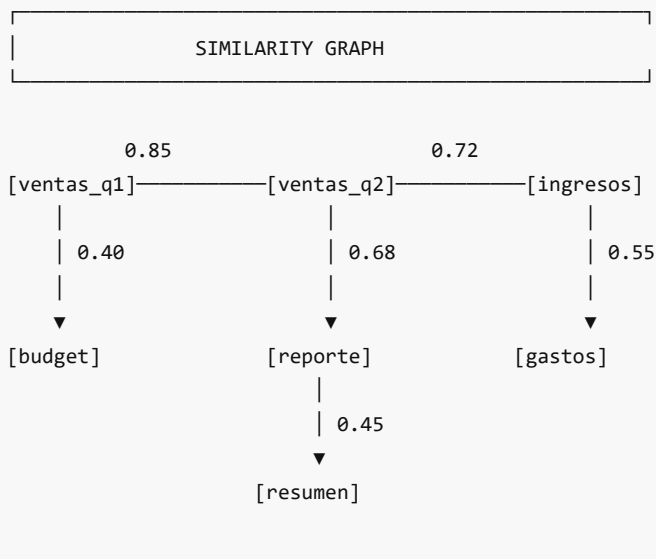
- Nodo\_1: {ventas\_q1, presupuesto}
- Nodo\_2: {marketing, comunicados}
- Nodo\_3: {ventas\_q2, ingresos\_2024}

Cálculo de afinidad:

- Nodo\_1: affinity = 0.75 (ventas\_q1) = 0.75
- Nodo\_2: affinity = 0.35 (marketing) = 0.35
- Nodo\_3: affinity = 0.70 + 0.60 (q2 + ingresos) = 1.30 ← MEJOR

Resultado: Réplica de reporte\_ventas.pdf → Nodo\_3

## Diagrama del Grafo



Threshold = 0.3 → Solo aristas con sim  $\geq$  0.3  
Max neighbors = 20 → Máximo 20 conexiones por nodo

## Poda de Vecinos

Para mantener el grafo eficiente, se limita el número de vecinos:

```
def _prune_neighbors(self, document_id: str) -> None:
 """Keep only top-k neighbors."""
 node = self._nodes.get(document_id)
 if not node or len(node.neighbors) <= self.max_neighbors:
 return

 # Mantener vecinos con mayor similaridad
 sorted_neighbors = sorted(
 node.neighbors.items(),
 key=lambda x: x[1],
 reverse=True
)

 keep = dict(sorted_neighbors[:self.max_neighbors])
 removed = set(node.neighbors.keys()) - set(keep.keys())

 node.neighbors = keep

 # Limpiar aristas y caché
 for doc_id in removed:
 self._edges[document_id].discard(doc_id)
 key = tuple(sorted([document_id, doc_id]))
 self._similarity_cache.pop(key, None)
```

## Actualización de Ubicación

```
def update_document_location(
 self,
 document_id: str,
 primary_node: Optional[str] = None,
 replica_nodes: Optional[List[str]] = None
) -> None:
 """Update the storage location of a document."""
 node = self._nodes.get(document_id)
 if not node:
 return

 # Actualizar índice de nodo primario
 if primary_node and primary_node != node.primary_node:
 self._node_to_docs[node.primary_node].discard(document_id)
 self._node_to_docs[primary_node].add(document_id)
 node.primary_node = primary_node
```

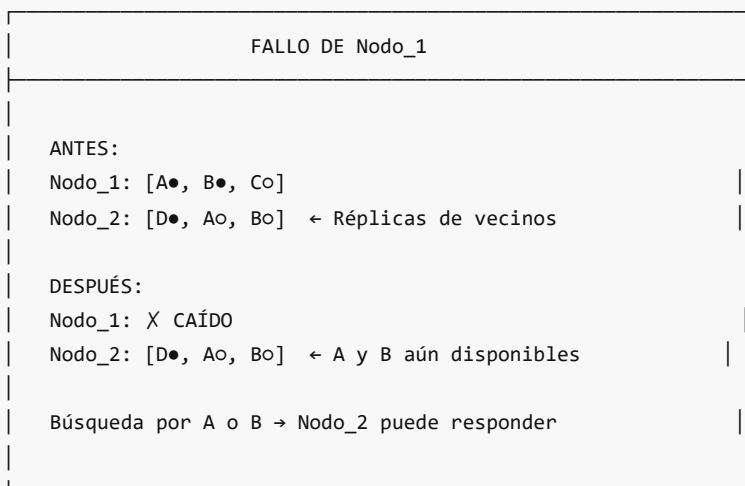
```
Actualizar réplicas
if replica_nodes is not None:
 for old_replica in node.replica_nodes:
 if old_replica not in replica_nodes:
 self._node_to_docs[old_replica].discard(document_id)

 for new_replica in replica_nodes:
 self._node_to_docs[new_replica].add(document_id)

node.replica_nodes = replica_nodes
```

## Impacto en Tolerancia a Fallos

El grafo influye en cómo sobrevive el sistema a fallos:



### Navegación:

- [← Anterior: Afinidad Semántica](#)
- [→ Siguiente: Factor de Replicación](#)

## Factor de Replicación

### Concepto

El **factor de replicación** define cuántas copias de cada documento existen en el cluster. En DistriSearch, el valor por defecto es 2 (1 primario + 1 réplica).

### Configuración

```
@dataclass
class ReplicationConfig:
```

```
replication_factor: int = 2 # Total de copias
...
```

$\text{copies} = \text{replication\_factor} = \text{primary} + \text{replicas}$

| Factor | Primario | Réplicas | Tolerancia |
|--------|----------|----------|------------|
| 1      | 1        | 0        | 0 fallos   |
| 2      | 1        | 1        | 1 fallo    |
| 3      | 1        | 2        | 2 fallos   |

## Trade-offs

| TRADE-OFFS DEL FACTOR DE REPLICACIÓN                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>FACTOR BAJO (1-2):</p> <ul style="list-style-type: none"> <li>✓ Menor uso de almacenamiento</li> <li>✓ Escrituras más rápidas (menos réplicas a sincronizar)</li> <li>X Menor tolerancia a fallos</li> <li>X Mayor riesgo de pérdida de datos</li> </ul> <p>FACTOR ALTO (3+):</p> <ul style="list-style-type: none"> <li>✓ Alta tolerancia a fallos (sobrevive a N-1 fallos)</li> <li>✓ Mejor disponibilidad de lectura</li> <li>X Mayor uso de almacenamiento (factor × tamaño)</li> <li>X Escrituras más lentas (sincronizar más réplicas)</li> <li>X Mayor complejidad de consistencia</li> </ul> |

## Cálculo de Almacenamiento

$\text{storage\_total} = \sum(\text{doc}) \text{ size}(\text{doc}) \times \text{replication\_factor}$

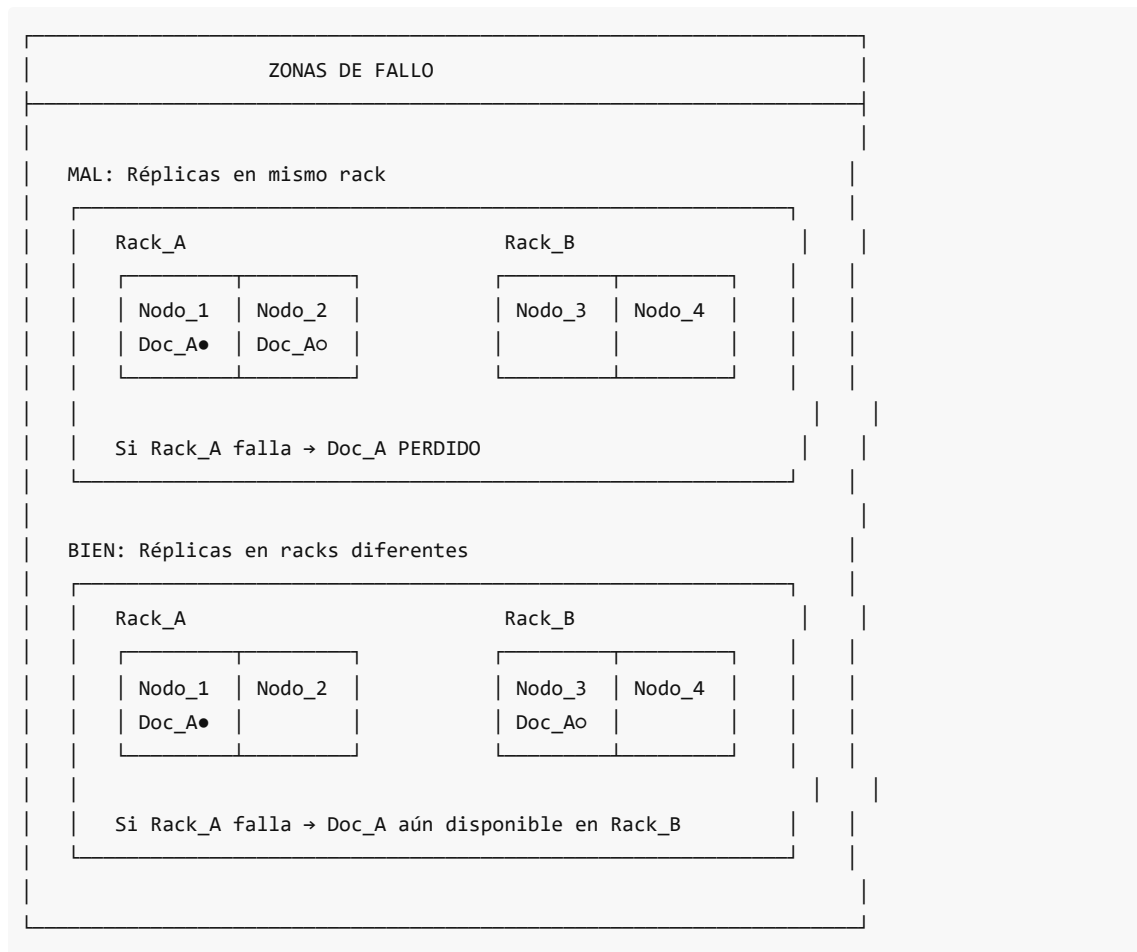
### Ejemplo:

- 10,000 documentos
- Tamaño promedio: 100 KB
- Factor = 2

$\text{storage} = 10,000 \times 100 \text{ KB} \times 2 = 2 \text{ GB}$

## Diversidad de Zonas de Fallo

No basta con tener réplicas; deben estar en **zonas de fallo diferentes**:



## Verificación de Tolerancia a Fallos

```
def _ensures_fault_tolerance(
 self,
 current_selection: List[str],
 candidate: str
) -> bool:
 """Verifica que el candidato no esté en el mismo rack/zona."""
 candidate_zone = self._get_failure_zone(candidate)
 selected_zones = {self._get_failure_zone(n) for n in current_selection}
 return candidate_zone not in selected_zones
```

## Selección de Nodos para Réplicas

El algoritmo completo considera:

1. **Afinidad semántica** (documentos similares)
2. **Diversidad de zonas** (tolerancia a fallos)
3. **Carga del nodo** (balanceo)

```

def select_replica_nodes(
 self,
 doc_id: str,
 source_node: str,
 doc_vector: AdaptiveDocumentVector
) -> List[str]:
 """
 Selecciona nodos para réplicas basándose en:
 1. Nodos que tienen documentos similares
 2. Diversidad geográfica/de fallos
 3. Carga actual
 """
 candidates = []

 # 1. Encontrar documentos similares y sus nodos
 similar_docs = self._find_similar_documents(doc_vector, top_k=10)

 node_affinity_scores = defaultdict(float)
 for sim_doc_id, similarity in similar_docs:
 sim_doc_node = self.partition_index.get_document(sim_doc_id).node_id
 if sim_doc_node != source_node:
 node_affinity_scores[sim_doc_node] += similarity

 # 2. Ordenar nodos por afinidad
 sorted_nodes = sorted(
 node_affinity_scores.items(),
 key=lambda x: x[1],
 reverse=True
)

 # 3. Seleccionar top-k asegurando diversidad
 selected = []
 for node_id, affinity in sorted_nodes:
 if len(selected) >= self.replication_factor - 1:
 break
 if self._ensures_fault_tolerance(selected, node_id):
 selected.append(node_id)

 # 4. Completar con nodos menos cargados si faltan
 if len(selected) < self.replication_factor - 1:
 remaining = self._get_least_loaded_nodes(
 exclude=selected + [source_node],
 count=self.replication_factor - 1 - len(selected)
)
 selected.extend(remaining)

 return selected

```

## Clase ReplicaTracker

Rastrea el estado de todas las réplicas:

```
class ReplicaTracker:
 """
 Tracks all document replicas in the cluster.

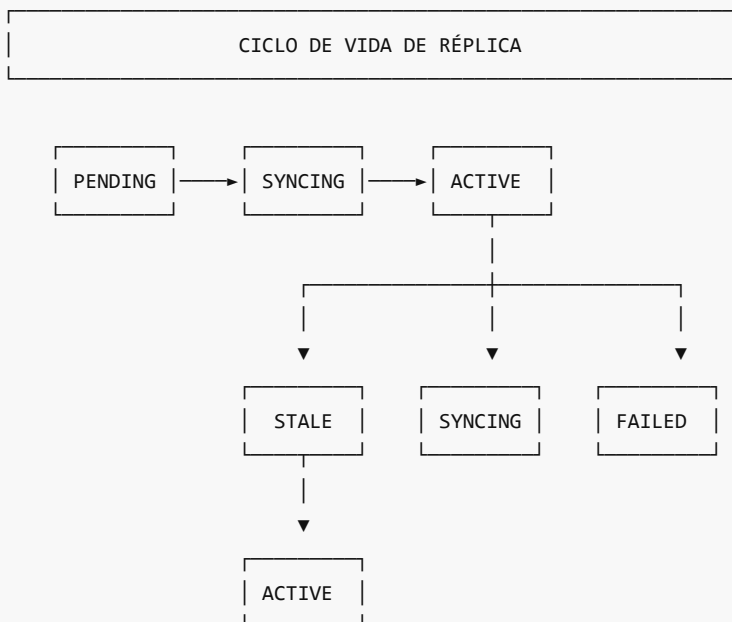
 Provides:
 - Replica state management
 - Under-replication detection
 - Node-to-replica mapping
 - Version tracking for consistency
 """

 def __init__(self, default_replication_factor: int = 2):
 self.default_replication_factor = default_replication_factor

 self._documents: Dict[str, DocumentReplicas] = {}
 self._node_replicas: Dict[str, Set[str]] = defaultdict(set)
 self._under_replicated: Set[str] = set() # Docs bajo factor
```

## Estados de Réplica

```
class ReplicaStatus(Enum):
 ACTIVE = "active" # Funcionando correctamente
 SYNCING = "syncing" # Sincronizándose
 STALE = "stale" # Desactualizada (versión vieja)
 FAILED = "failed" # Nodo falló
 PENDING = "pending" # Esperando creación
```





## Detección de Sub-replicación

```
@dataclass
class DocumentReplicas:
 document_id: str
 primary: Optional[ReplicaInfo] = None
 replicas: List[ReplicaInfo] = field(default_factory=list)
 replication_factor: int = 2

 @property
 def healthy_count(self) -> int:
 return sum(1 for r in self.all_replicas if r.is_healthy)

 @property
 def is_under_replicated(self) -> bool:
 return self.healthy_count < self.replication_factor
```

## Proceso de Re-replicación

Cuando se detecta sub-replicación:

```
def _queue_replication(
 self,
 document_id: str,
 priority: ReplicationPriority = ReplicationPriority.NORMAL
) -> None:
 """Queue replication for an under-replicated document."""
 doc = self.replica_tracker.get_document_replicas(document_id)
 if not doc or not doc.is_under_replicated:
 return

 # Encontrar fuente (réplica sana)
 source_node = None
 for replica in doc.all_replicas:
 if replica.is_healthy:
 source_node = replica.node_id
 break

 if not source_node:
 logger.error(f"No healthy replica for {document_id}")
 return

 # Seleccionar nuevos nodos para réplicas
 current_nodes = set(doc.all_nodes)
 available = [n for n in self._cluster_nodes if n not in current_nodes]

 needed = doc.replication_factor - doc.healthy_count

 for target in available[:needed]:
```

```
task = self._create_task(
 document_id=document_id,
 source_node=source_node,
 target_node=target,
 priority=priority
)
self._pending_tasks[task.task_id] = task
```

## Ejemplo Completo

Escenario: Documento nuevo "informe\_anual.pdf" llega al cluster

Configuracion:

```
replication_factor = 2
cluster_nodes = [Nodo_1, Nodo_2, Nodo_3, Nodo_4]
zones = {Nodo_1: Rack_A, Nodo_2: Rack_A, Nodo_3: Rack_B, Nodo_4: Rack_B}
```

Paso 1: Documento llega a Nodo\_1 (primario)

```
primary_node = Nodo_1
needed_replicas = 2 - 1 = 1
```

Paso 2: Calcular afinidad

Vecinos similares:

- informe\_q1.pdf (sim=0.8) → Nodo\_2
- informe\_q2.pdf (sim=0.75) → Nodo\_3
- resumen\_anual.pdf (sim=0.6) → Nodo\_2

Afinidad:

```
Nodo_2: 0.8 + 0.6 = 1.4
Nodo_3: 0.75 = 0.75
Nodo_4: 0.0 = 0.0
```

Paso 3: Verificar diversidad de zonas

```
Nodo_1 (primario) → Rack_A
Nodo_2 (mejor afinidad) → Rack_A ← MISMO RACK, RECHAZAR
Nodo_3 (segunda afinidad) → Rack_B ✓ DIFERENTE RACK
```

Paso 4: Resultado

```
Primario: Nodo_1 (Rack_A)
Réplica: Nodo_3 (Rack_B) ← Balanceado entre racks
```

Paso 5: Encolar tarea de replicación

```
Task: Nodo_1 → Nodo_3 (informe_anual.pdf)
Priority: NORMAL
```

## Métricas de Replicación

| Métrica | Descripción | Umbral |
|---------|-------------|--------|
|---------|-------------|--------|

|                        |                           |               |
|------------------------|---------------------------|---------------|
| under_replicated_count | Docs con menos réplicas   | 0 ideal       |
| total_replicas         | Total de réplicas         | docs × factor |
| sync_lag               | Retraso de sincronización | < 5 segundos  |
| failed_replicas        | Réplicas fallidas         | 0 ideal       |

#### Navegación:

- [← Anterior: Grafo de Similaridad](#)
- [→ Siguiente: Tolerancia a Fallos](#)

## 06 Tolerancia Fallos

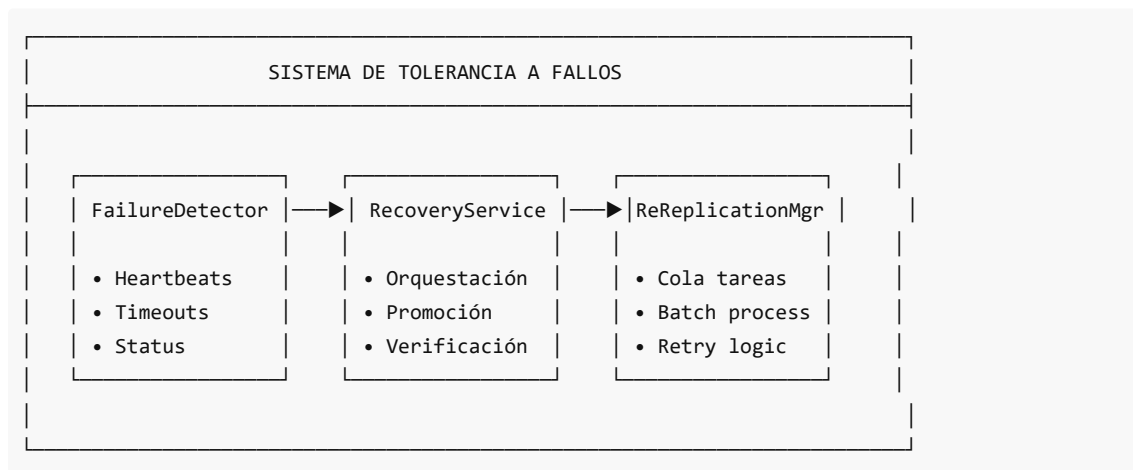
### 06. Tolerancia a Fallos

#### Visión General

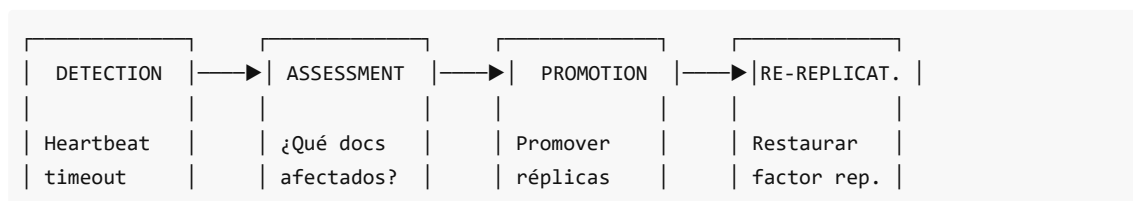
La **tolerancia a fallos** es la capacidad del sistema de continuar operando correctamente cuando uno o más componentes fallan. DistriSearch implementa múltiples capas de protección:

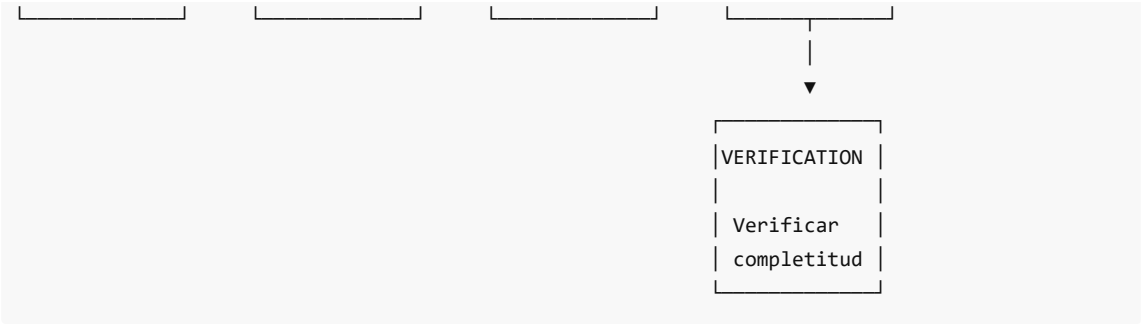
1. **Detección de fallos** via heartbeats
2. **Re-replicación automática** de datos
3. **Recuperación de nodos** al reincorporarse

#### Arquitectura de Tolerancia a Fallos



#### Flujo de Recuperación



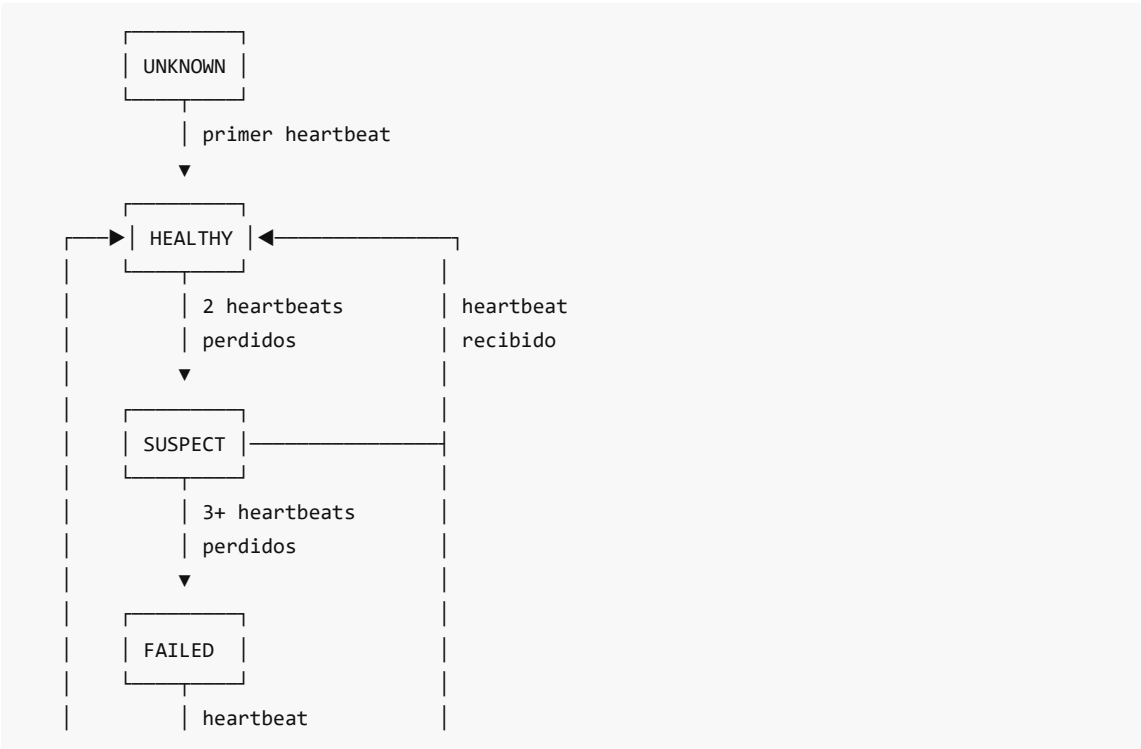


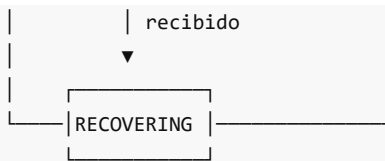
## Componentes Principales

| Componente           | Archivo             | Responsabilidad                     |
|----------------------|---------------------|-------------------------------------|
| FailureDetector      | failure_detector.py | Detecta fallos via heartbeats       |
| RecoveryService      | recovery_service.py | Orquesta el proceso de recuperación |
| ReReplicationManager | re_replication.py   | Gestiona la re-replicación          |

## Estados de Nodo

```
class NodeStatus(Enum):
 HEALTHY = "healthy" # Funcionando correctamente
 SUSPECT = "suspect" # Posible fallo (2 heartbeats perdidos)
 FAILED = "failed" # Fallo confirmado (3+ heartbeats perdidos)
 RECOVERING = "recovering" # Recuperándose después de fallo
 UNKNOWN = "unknown" # Estado inicial
```





## Fases de Recuperación

```
class RecoveryPhase(Enum):
 DETECTION = "detection" # Fallo detectado
 ASSESSMENT = "assessment" # Evaluando impacto
 PROMOTION = "promotion" # Promoviendo réplicas
 RE_REPLICATION = "re_replication" # Re-replicando datos
 VERIFICATION = "verification" # Verificando completitud
 COMPLETED = "completed" # Recuperación exitosa
 FAILED = "failed" # Recuperación fallida
```

## Configuración

```
@dataclass
class RecoveryConfig:
 # Detección de fallos
 heartbeat_interval_sec: float = 5.0 # Verificar cada 5s
 failure_timeout_sec: float = 15.0 # Timeout de 15s
 suspect_threshold: int = 2 # 2 fallos → suspect
 failure_threshold: int = 3 # 3 fallos → failed

 # Re-replicación
 max_concurrent_rereplications: int = 5
 re_replication_batch_size: int = 20
 re_replication_retry_limit: int = 3

 # Timing de recuperación
 assessment_delay_sec: float = 2.0 # Esperar estabilización
 verification_timeout_sec: float = 60.0 # Timeout verificación
```

## Contenido de Esta Sección

1. [Detección de Fallos](#): Heartbeats y timeouts
2. [Re-Replicación](#): Restaurar factor de replicación
3. [Recuperación de Nodos](#): Reintegración al cluster

---

### Navegación:

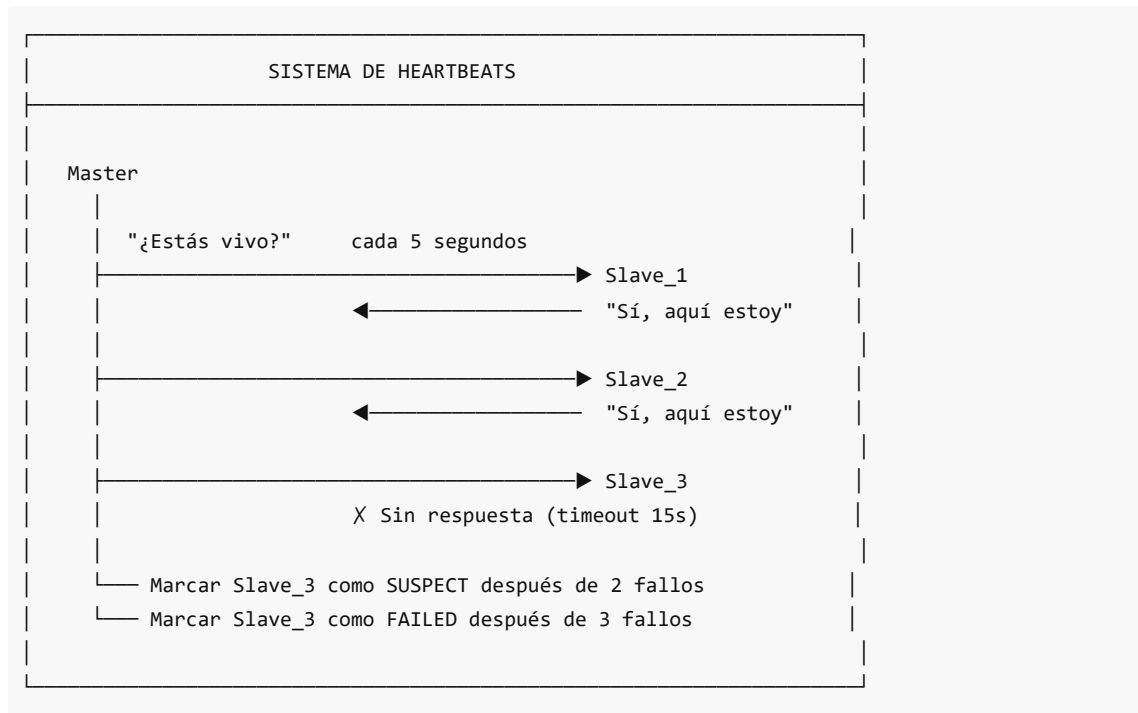
- [← Anterior: Replicación](#)
  - [→ Siguiente: Consenso Raft](#)
-

# Detección de Fallos

## Concepto

La **detección de fallos** es el primer paso en la tolerancia a fallos. DistriSearch usa un sistema de **heartbeats** (latidos) para determinar si un nodo está funcionando.

## Mecanismo de Heartbeat



## Clase FailureDetector

**Ubicación:** backend/app/core/recovery/failure\_detector.py

```
class FailureDetector:
 """
 Detects node failures using heartbeat monitoring.

 Features:
 - Configurable heartbeat interval and timeout
 - Suspect state before declaring failure
 - Callback on failure detection
 - Recovery detection
 """

 def __init__(
 self,
 heartbeat_interval_sec: float = 5.0,
 failure_timeout_sec: float = 15.0,
```

```

 suspect_threshold: int = 2,
 failure_threshold: int = 3,
 on_failure: Optional[Callable[[FailureEvent], Awaitable[None]]] = None,
 on_recovery: Optional[Callable[[str], Awaitable[None]]] = None
):
 self.heartbeat_interval = heartbeat_interval_sec
 self.failure_timeout = failure_timeout_sec
 self.suspect_threshold = suspect_threshold
 self.failure_threshold = failure_threshold
 self._on_failure = on_failure
 self._on_recovery = on_recovery

 self._nodes: Dict[str, NodeHealth] = {}
 self._failed_nodes: Set[str] = set()

```

## Estructura de Salud del Nodo

```

@dataclass
class NodeHealth:
 """Health information for a node."""
 node_id: str
 status: NodeStatus = NodeStatus.UNKNOWN
 last_heartbeat: Optional[datetime] = None
 consecutive_failures: int = 0
 last_failure: Optional[datetime] = None
 recovery_attempts: int = 0
 latency_ms: float = 0.0
 metadata: Dict = field(default_factory=dict)

 @property
 def time_since_heartbeat(self) -> Optional[timedelta]:
 if self.last_heartbeat:
 return datetime.utcnow() - self.last_heartbeat
 return None

 @property
 def is_healthy(self) -> bool:
 return self.status == NodeStatus.HEALTHY

```

## Registro de Heartbeat

```

def record_heartbeat(
 self,
 node_id: str,
 latency_ms: float = 0.0,
 metadata: Optional[Dict] = None
) -> None:
 """Record a heartbeat from a node."""

```

```

if node_id not in self._nodes:
 self.register_node(node_id, metadata)

health = self._nodes[node_id]
was_failed = health.status == NodeStatus.FAILED

Actualizar estado
health.last_heartbeat = datetime.utcnow()
health.consecutive_failures = 0
health.latency_ms = latency_ms
health.status = NodeStatus.HEALTHY

Detectar recuperación
if was_failed:
 self._failed_nodes.discard(node_id)
 health.status = NodeStatus.RECOVERING
 logger.info(f"Node {node_id} recovered")

 if self._on_recovery:
 asyncio.create_task(self._on_recovery(node_id))

```

## Registro de Fallo

```

def record_failure(self, node_id: str, error: str = "") -> None:
 """Record a failed health check for a node."""
 if node_id not in self._nodes:
 return

 health = self._nodes[node_id]
 health.consecutive_failures += 1
 health.last_failure = datetime.utcnow()

 # Actualizar estado basado en fallos consecutivos
 if health.consecutive_failures >= self.failure_threshold: # 3
 if health.status != NodeStatus.FAILED:
 self._mark_failed(health, "threshold", error)
 elif health.consecutive_failures >= self.suspect_threshold: # 2
 health.status = NodeStatus.SUSPECT
 logger.warning(f"Node {node_id} is suspect")

```

## Línea de Tiempo de Detección

|            |         |         |         |         |        |          |
|------------|---------|---------|---------|---------|--------|----------|
| Tiempo:    | 0s      | 5s      | 10s     | 15s     | 20s    | 25s      |
|            |         |         |         |         |        |          |
|            | ▼       | ▼       | ▼       | ▼       | ▼      | ▼        |
| Heartbeat: | ✓       | ✓       | X       | X       | X      |          |
| Status:    | HEALTHY | HEALTHY | HEALTHY | SUSPECT | FAILED | Recovery |
|            |         |         |         |         |        |          |



```
| ↳ on_failure() llamado
|
|_ 2 fallos consecutivos
```

Configuración:

```
heartbeat_interval = 5s
suspect_threshold = 2 (fallos)
failure_threshold = 3 (fallos)
```

## Evento de Fallo

```
@dataclass
class FailureEvent:
 """Represents a node failure event."""
 node_id: str
 detected_at: datetime
 last_healthy: Optional[datetime]
 failure_type: str # "timeout", "error", "explicit"
 details: str = ""

 @property
 def downtime(self) -> Optional[timedelta]:
 if self.last_healthy:
 return self.detected_at - self.last_healthy
 return None
```

## Tipos de Fallos

| Tipo     | Descripción           | Ejemplo            |
|----------|-----------------------|--------------------|
| timeout  | Heartbeat no recibido | Nodo sin respuesta |
| error    | Error en health check | Conexión rechazada |
| explicit | Nodo reportó fallo    | Shutdown graceful  |

## Health Checks en Docker

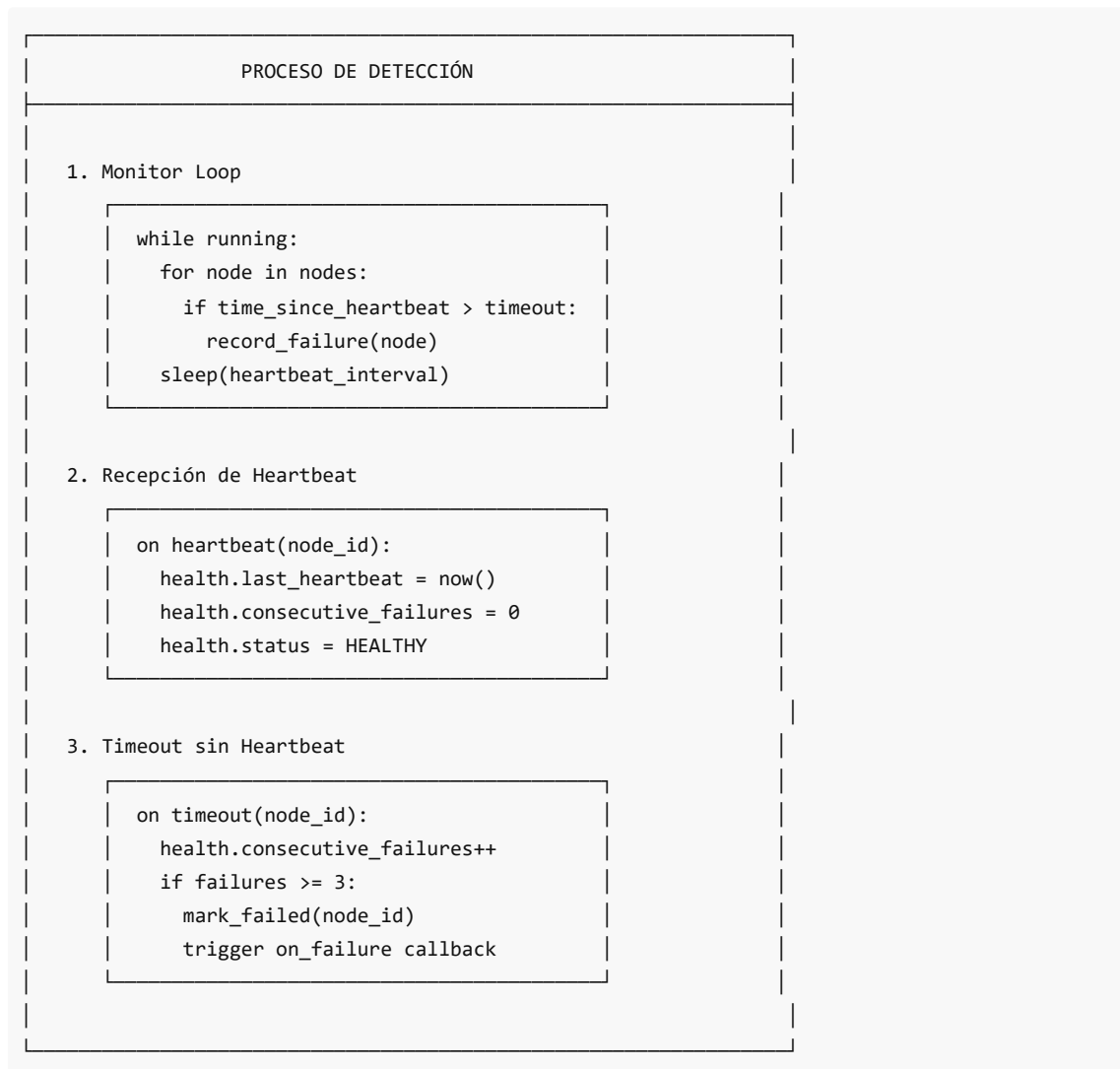
Docker Swarm también realiza health checks:

```
docker-compose.yml
services:
 slave:
 healthcheck:
 test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
 interval: 30s
 timeout: 10s
 retries: 3
 start_period: 40s
```

## Endpoint /health

```
backend/app/api/endpoints.py
@router.get("/health")
async def health_check():
 return {
 "status": "healthy",
 "timestamp": datetime.utcnow().isoformat(),
 "node_id": settings.NODE_ID,
 "version": settings.VERSION
 }
```

## Diagrama de Detección



## Ventajas del Estado SUSPECT

El estado intermedio SUSPECT evita falsos positivos:

- **Red temporal:** Un paquete perdido no dispara recuperación
- **Carga alta:** Nodo lento responde en siguiente ciclo
- **GC pause:** Pausa de garbage collection no es fallo real

#### Navegación:

- [← README](#)
- [→ Siguiente: Re-Replicación](#)

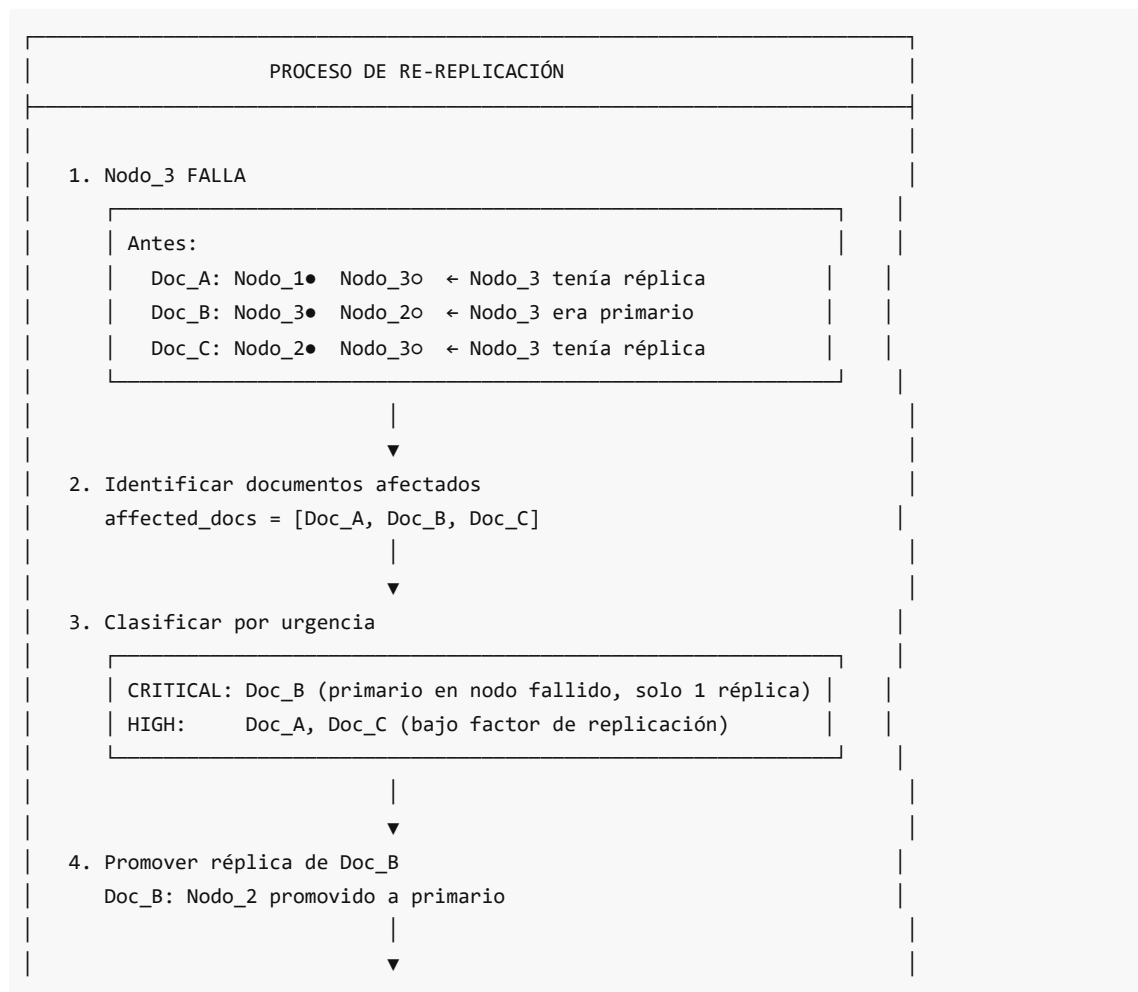
## Re-Replicación

### Concepto

La **re-replicación** es el proceso de crear nuevas réplicas de documentos cuando el factor de replicación cae por debajo del objetivo (típicamente 2). Esto ocurre cuando:

- Un nodo falla permanentemente
- Se detectan réplicas corruptas
- Se aumenta el factor de replicación

### Flujo de Re-Replicación



#### 5. Re-replicar a nuevos nodos

Después:

Doc\_A: Nodo\_1• Nodo\_4o ← Nueva réplica en Nodo\_4  
Doc\_B: Nodo\_2• Nodo\_1o ← Promovido + nueva réplica  
Doc\_C: Nodo\_2• Nodo\_4o ← Nueva réplica en Nodo\_4

## Clase ReReplicationManager

**Ubicación:** backend/app/core/recovery/re\_replication.py

```
class ReReplicationManager:
 """
 Manages re-replication of data after failures.

 When a node fails:
 1. Identifies all documents that were stored on that node
 2. For each document, finds a healthy replica
 3. Replicates to a new node to maintain replication factor
 """

 def __init__(
 self,
 replicate_func: Optional[Callable] = None,
 get_document_replicas: Optional[Callable] = None,
 select_target_node: Optional[Callable] = None,
 max_concurrent: int = 5,
 batch_size: int = 20,
 retry_limit: int = 3
):
 self.max_concurrent = max_concurrent
 self.batch_size = batch_size
 self.retry_limit = retry_limit

 self._pending_tasks: Dict[str, ReReplicationTask] = {}
 self._active_tasks: Dict[str, asyncio.Task] = {}
```

## Tarea de Re-Replicación

```
@dataclass
class ReReplicationTask:
 """Task to re-replicate a document."""
 task_id: str
 document_id: str
 failed_node: str
 source_node: Optional[str] = None # Réplica sana origen
```

```

target_node: Optional[str] = None # Nuevo destino
status: ReReplicationStatus = ReReplicationStatus.PENDING
created_at: datetime = field(default_factory=datetime.utcnow)
started_at: Optional[datetime] = None
completed_at: Optional[datetime] = None
error: Optional[str] = None
retry_count: int = 0
priority: int = 1 # Mayor = más urgente

```

## Estados de Re-Replicación

```

class ReReplicationStatus(Enum):
 PENDING = "pending" # En cola
 IN_PROGRESS = "in_progress" # Ejecutándose
 COMPLETED = "completed" # Éxito
 FAILED = "failed" # Falló
 CANCELLED = "cancelled" # Cancelado

```

## Encolar Re-Replicación

```

def queue_re_replication(
 self,
 document_id: str,
 failed_node: str,
 priority: int = 1
) -> ReReplicationTask:
 """Queue a document for re-replication."""
 task_id = f"rerep_{document_id}_{datetime.utcnow().timestamp()}"

 task = ReReplicationTask(
 task_id=task_id,
 document_id=document_id,
 failed_node=failed_node,
 priority=priority
)

 self._pending_tasks[task_id] = task
 logger.debug(f"Queued re-replication for {document_id}")

 return task

def queue_bulk_re_replication(
 self,
 document_ids: List[str],
 failed_node: str,
 priority: int = 1
) -> List[ReReplicationTask]:

```

```
"""Queue multiple documents for re-replication."""
tasks = []
for doc_id in document_ids:
 task = self.queue_re_replication(doc_id, failed_node, priority)
 tasks.append(task)

logger.info(f"Queued {len(tasks)} documents for re-replication")
return tasks
```

Proceso de Re-Replicación

Algorithm: Re-Replication Process

FOR EACH document IN pending\_tasks (ordenado por prioridad):

1. ENCONTRAR fuente (réplica sana)

source = find\_healthy\_replica(document)

IF source == None:

mark\_task\_failed("No healthy replica")

CONTINUE

2. SELECCIONAR destino

current\_nodes = get\_current\_replica\_nodes(document)

available = cluster\_nodes - current\_nodes - failed\_nodes

# Usar afinidad semántica si es posible

target = select\_best\_node(document, available)

IF target == None:

mark\_task\_failed("No available target")

CONTINUE

3. EJECUTAR replicación

TRY:

success = await replicate(document, source, target)

IF success:

mark\_task\_completed()

ELSE:

retry\_or\_fail()

CATCH error:

retry\_or\_fail()

4. ACTUALIZAR tracker de réplicas

replica\_tracker.add\_replica(document, target)

Priorización

| Prioridad | Valor | Situación                          |
|-----------|-------|------------------------------------|
| CRITICAL  | 3     | Sin réplicas (única copia perdida) |

|        |   |                            |
|--------|---|----------------------------|
| HIGH   | 2 | Bajo factor de replicación |
| NORMAL | 1 | Optimización de ubicación  |

## Servicio de Recuperación

**Ubicación:** backend/app/core/recovery/recovery\_service.py

```
class RecoveryService:
 """
 Coordinates failure detection and recovery.

 Workflow on node failure:
 1. Detect failure via missed heartbeats
 2. Assess impact (which documents affected)
 3. Promote replicas to primary where needed
 4. Re-replicate to maintain replication factor
 5. Verify recovery complete
 """

 async def _execute_recovery(self, task: RecoveryTask) -> None:
 # Phase 1: Assessment
 task.phase = RecoveryPhase.ASSESSMENT
 await asyncio.sleep(self.config.assessment_delay_sec)

 affected_docs = await self._assess_impact(task.failed_node)
 task.affected_documents = affected_docs

 if not affected_docs:
 task.phase = RecoveryPhase.COMPLETED
 return

 # Phase 2: Promotion
 task.phase = RecoveryPhase.PROMOTION
 await self._promote_replicas(task)

 # Phase 3: Re-replication
 task.phase = RecoveryPhase.RE_REPLICATION
 await self._queue_re_replications(task)

 # Phase 4: Verification
 task.phase = RecoveryPhase.VERIFICATION
 await self._verify_recovery(task)

 task.phase = RecoveryPhase.COMPLETED
```

## Promoción de Réplicas

Cuando el nodo fallido era primario:

```

async def _promote_replicas(self, task: RecoveryTask) -> None:
 """
 Promote replicas to primary where the failed node was primary.
 """
 for doc_id in task.affected_documents:
 try:
 replica_info = await self._get_replica_info(doc_id)
 if not replica_info:
 continue

 # Verificar si el nodo fallido era primario
 primary = replica_info.get("primary")
 if primary != task.failed_node:
 continue

 # Encontrar réplica sana para promover
 replicas = replica_info.get("replicas", [])
 healthy_replicas = [
 r for r in replicas
 if r.get("status") == "active" and r.get("node") != task.failed_node
]

 if healthy_replicas:
 new_primary = healthy_replicas[0]["node"]

 if self._promote_func:
 success = await self._promote_func(doc_id, new_primary)
 if success:
 task.promoted primaries[doc_id] = new_primary
 logger.info(f"Promoted {new_primary} to primary for {doc_id}")

 except Exception as e:
 logger.error(f"Failed to promote replica for {doc_id}: {e}")

```

## Verificación de Recuperación

```

async def _verify_recovery(self, task: RecoveryTask) -> None:
 """
 Verify that recovery is complete.
 """
 timeout = self.config.verification_timeout_sec
 start = datetime.utcnow()

 # Esperar a que se completen las re-replicaciones
 while (datetime.utcnow() - start).total_seconds() < timeout:
 pending = self.re_replication_mgr.get_pending_count()
 active = self.re_replication_mgr.get_active_count()

 if pending == 0 and active == 0:

```



```
 break

 await asyncio.sleep(2.0)

Obtener estadísticas
stats = self.re_replication_mgr.get_statistics()
task.documents_recovered = stats["total_completed"]
task.documents_failed = stats["total_failed"]
```

## Ejemplo de Recuperación Completa

Escenario: Nodo\_3 falla con 500 documentos

---

T=0s: Heartbeat de Nodo\_3 no recibido  
T=5s: Segundo heartbeat perdido → SUSPECT  
T=10s: Tercer heartbeat perdido → FAILED

T=10s: RecoveryService.on\_failure() llamado

T=12s: Assessment completado  
affected\_documents = 500  
primaries\_on\_failed = 180  
replicas\_on\_failed = 320

T=12s-15s: Promoción de 180 réplicas a primario

T=15s-45s: Re-replicación de 500 documentos  
batch\_size = 20  
max\_concurrent = 5  
~25 batches, ~6 segundos/batch

T=45s: Verificación completada  
documents\_recovered = 498  
documents\_failed = 2

T=45s: Recovery COMPLETED

---

### Navegación:

- [← Anterior: Detección de Fallos](#)
  - [→ Siguiente: Recuperación de Nodos](#)
- 

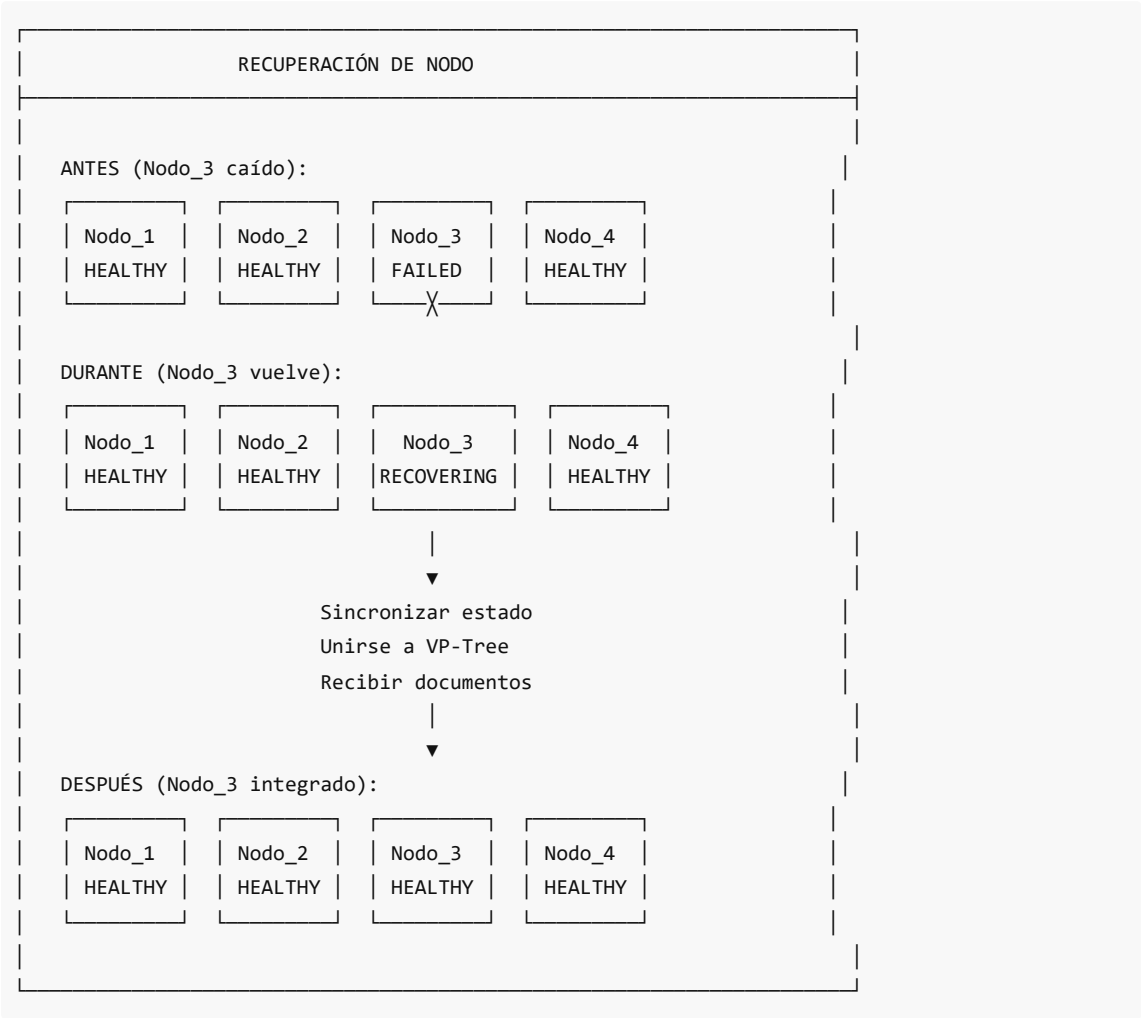
## Recuperación de Nodos

### Concepto

Cuando un nodo que había fallado vuelve a estar disponible, debe **reintegrarse** al cluster de forma ordenada. Este proceso incluye:

- 1. Detección de recuperación
- 2. Sincronización de estado
- 3. Reintegración en VP-Tree
- 4. Posible redistribución de documentos

## Detección de Recuperación



```

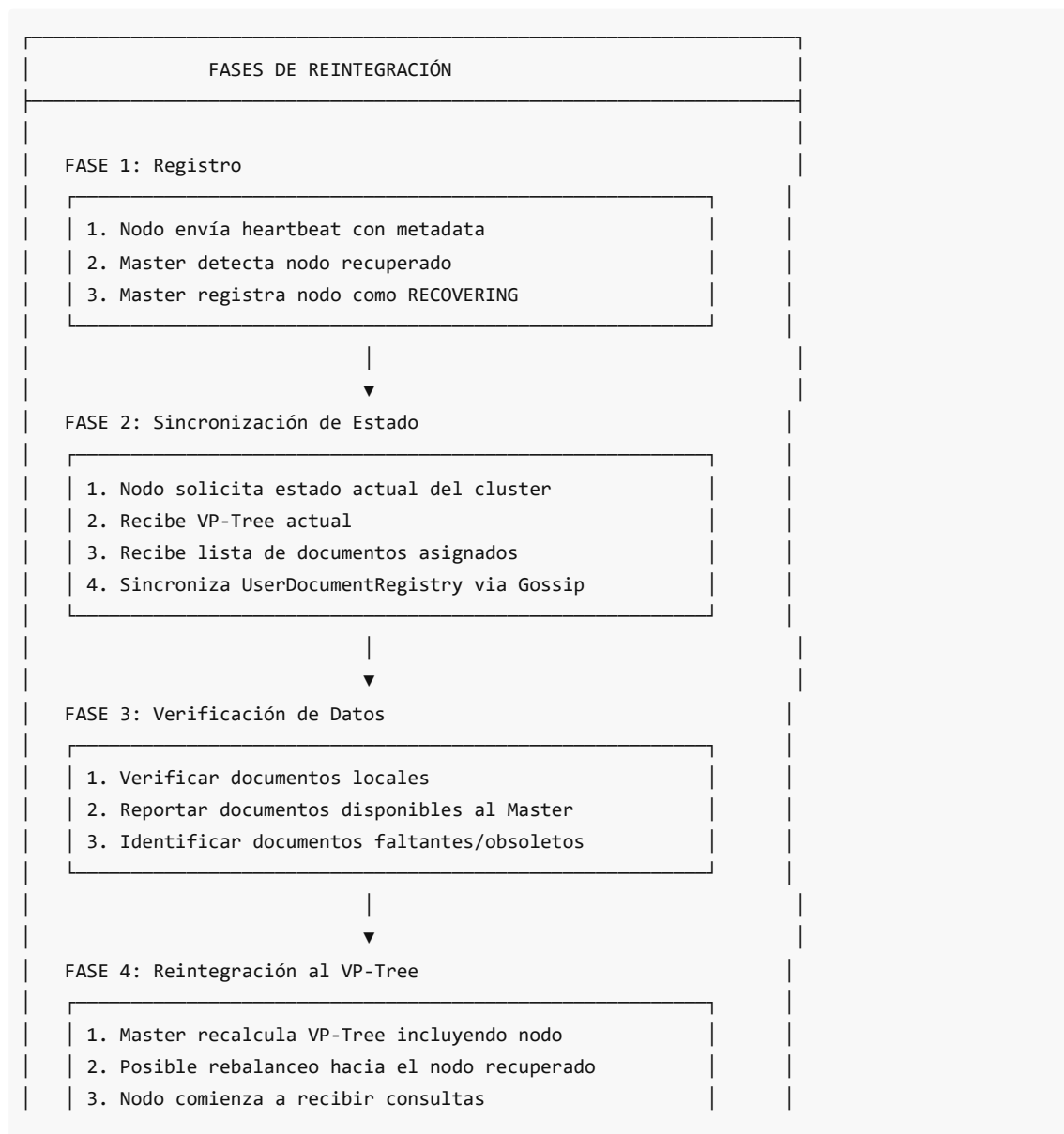
health.last_heartbeat = datetime.utcnow()
health.consecutive_failures = 0
health.status = NodeStatus.HEALTHY

Detectar recuperación
if was_failed:
 self._failed_nodes.discard(node_id)
 health.status = NodeStatus.RECOVERING # Estado transitorio
 logger.info(f"Node {node_id} recovered")

Disparar callback de recuperación
if self._on_recovery:
 asyncio.create_task(self._on_recovery(node_id))

```

## Proceso de Reintegración



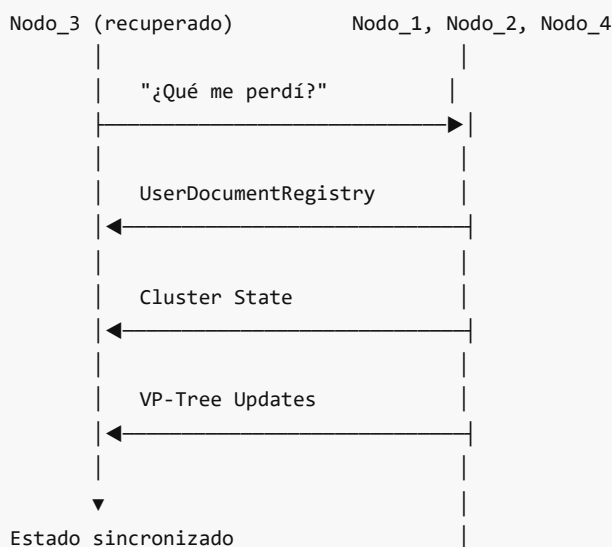
FASE 5: Estado HEALTHY

Nodo completamente operacional

## Sincronización via Gossip

El protocolo Gossip ayuda en la sincronización:

### GOSSIP SYNC DURANTE RECUPERACIÓN



## Manejo de Datos Obsoletos

Cuando un nodo vuelve, puede tener datos obsoletos:

```
async def handle_node_rejoin(self, node_id: str) -> None:
 """
 Handle a previously failed node rejoining.
 """
 # 1. Obtener documentos que el nodo cree tener
 claimed_docs = await self._get_node_documents(node_id)

 # 2. Verificar contra estado actual
 for doc_id in claimed_docs:
 current_replicas = await self._get_replica_info(doc_id)
```

```

if current_replicas is None:
 # Documento fue eliminado mientras nodo estaba caído
 await self._notify_delete(node_id, doc_id)
 continue

current_version = current_replicas.get("version", 0)
node_version = await self._get_node_doc_version(node_id, doc_id)

if node_version < current_version:
 # Versión obsoleta, sincronizar
 await self._sync_document(node_id, doc_id)

```

## Escenarios de Recuperación

### Escenario 1: Recuperación Rápida (<1 minuto)

- Nodo reiniciado por actualizaciones
- Datos locales intactos
- Solo necesita sincronizar estado
- Documentos válidos, sin re-replicación necesaria

### Escenario 2: Recuperación Media (1-10 minutos)

- Nodo caído por problema transitorio
- Datos locales intactos pero posiblemente obsoletos
- Re-replicación ya ocurrió para algunos documentos
- Necesita reconciliar réplicas duplicadas

### Escenario 3: Recuperación Larga (>10 minutos) o Nuevo Nodo

- Nodo reemplazado o datos perdidos
- Sin documentos locales
- Actúa como nodo completamente nuevo
- Recibe documentos via rebalanceo

## Rebalanceo Post-Recuperación

Cuando un nodo vuelve, puede disparar rebalanceo:

```

async def on_node_rejoin(self, node_id: str) -> None:
 """
 Handle node rejoining the cluster.
 """
 # 1. Marcar como HEALTHY
 self.failure_detector.record_heartbeat(node_id)

 # 2. Verificar si hay desbalance
 summary = self.load_calculator.calculate_cluster_summary()

```

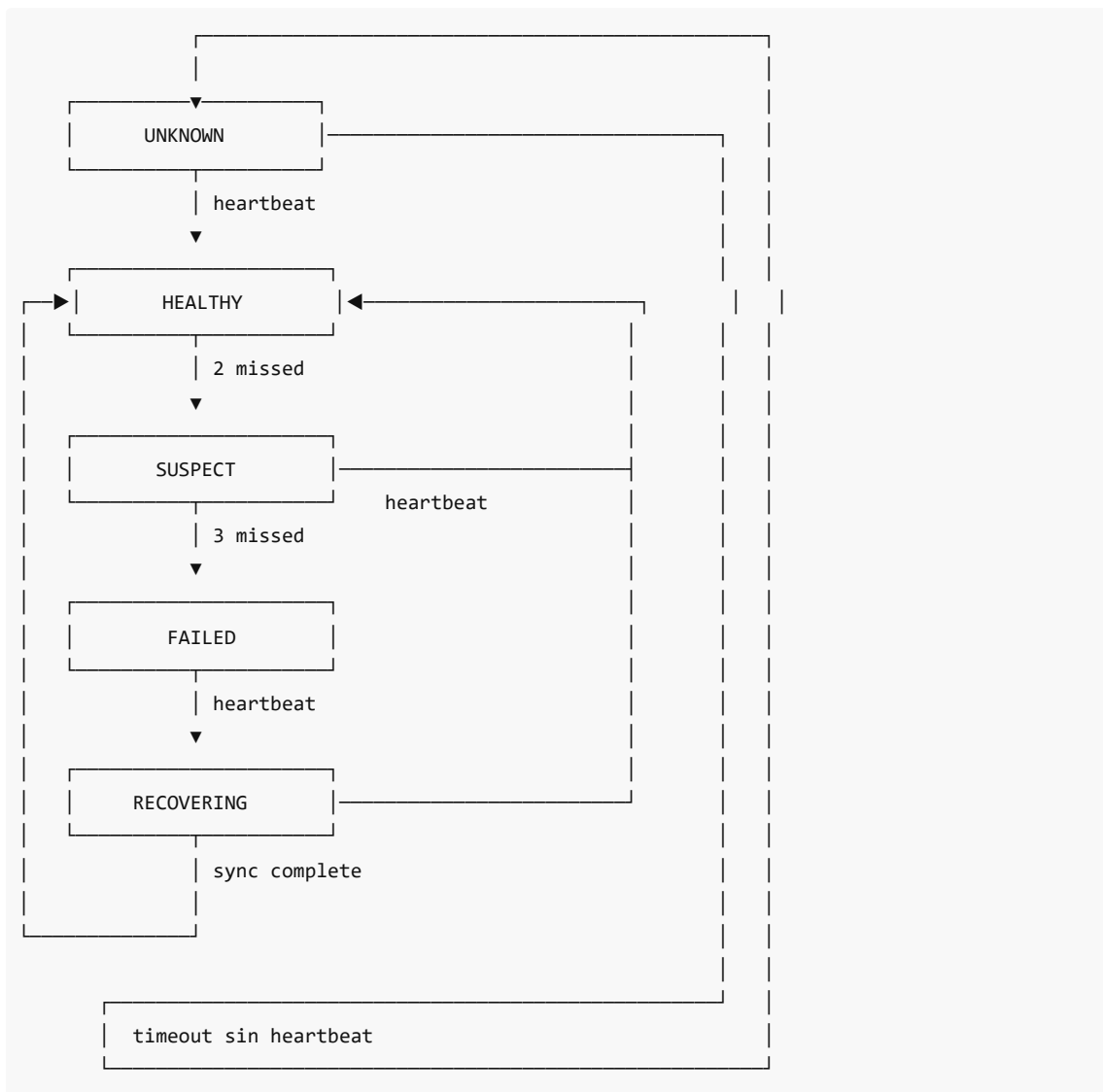
```

El nodo recuperado probablemente tiene pocos documentos
(fueron redistribuidos mientras estaba caído)
node_load = summary.get_node_load(node_id)
avg_load = summary.avg_load_factor

if node_load < avg_load * 0.5: # Significativamente bajo carga
 # Disparar rebalanceo para redistribuir hacia el nodo
 await self.active_rebalancer.execute_rebalance()

```

## Diagrama de Estados Completo



## Troubleshooting de Recuperación

Problemas comunes de la guía de despliegue:

| Problema | Causa | Solución |
|----------|-------|----------|
|----------|-------|----------|



```
|
+-----> Follower <-----+
Leader --(heartbeat AppendEntries)--> Follower
```

En adelante: 01 detalla elección de líder, 02 la replicación de log y 03 el uso de SQLite como máquina de estados replicada.

## Elección de líder (Raft)

### Flujo resumido

1. Cada nodo inicia como `Follower` y arranca un temporizador aleatorio 150–300 ms ( `LeaderElection._get_election_timeout` ).
2. Si no recibe `AppendEntries` (heartbeat) antes de expirar, pasa a `Candidate` , incrementa el término, vota por sí mismo y envía `RequestVote` a todos los peers ( `start_election` ).
3. Los peers otorgan el voto sólo si el término es al menos el actual y el log del candidato está tan actualizado como el suyo ( `_is_log_up_to_date` compara `last_log_term` y `last_log_index` ).
4. Con mayoría simple ( $\lceil N/2 \rceil$ ) el candidato gana y se convierte en `Leader` ( `_win_election` ), resetea el temporizador y arranca heartbeats.
5. Si cualquier RPC revela un término superior, el nodo hace step-down a `Follower` y limpia estado de líder ( `update_term + become_follower` ).

### RequestVote RPC

- Args: `term` , `candidate_id` , `last_log_index` , `last_log_term` .
- Reply: `term` , `vote_granted` .
- Se envía en paralelo a todos los nodos conocidos (excluyendo a sí mismo) y se toleran timeouts (se ignoran respuestas `None` ).

### Prevención de split-brain

- La única vía a liderazgo es mayoría de votos sobre el mismo término; particiones sin mayoría no eligen líder.
- Comparación de logs evita que un nodo atrasado tome liderazgo aunque reciba votos previos.
- Heartbeats frecuentes (50 ms) reinician el temporizador de seguidores; si el líder cae, el timeout aleatorio reduce colisiones de elecciones simultáneas.

### Integración en DistriSearch

- `RaftNode.start()` inicia el temporizador de elección al boot; las llamadas `handle_append_entries` lo reinician cuando se escucha al líder.
- El líder es quien acepta escrituras de metadatos (usuarios, nodos, particiones) y dispara replicación del log; los demás redirigen o esperan.
- Configuración: habilitado por `RAFT_ENABLED=true` en despliegues Swarm; sin mayoría disponible el nodo queda como seguidor pasivo.

### Consideraciones operativas

- Clúster de un solo nodo: gana inmediatamente (se autoelige y pasa a líder).
- Rotación de liderazgo segura: al detectar término mayor en `RequestVote` / `AppendEntries` , el líder vigente se degrada a seguidor.
- Persistencia: `current_term` y `voted_for` se guardan en disco antes de responder, evitando votos duplicados tras reinicios.



---

# Replicación de log (Raft)

## Ciclo del líder

1. Heartbeats cada 50 ms: `AppendEntries` vacío para afirmar liderazgo y reiniciar timers de seguidores ( `_send_heartbeats` ).
2. Al recibir un comando, el líder lo añade al log local con su término ( `submit_command` ) y dispara replicación inmediata.
3. Se esperan acks hasta alcanzar mayoría; sólo entonces el `commit_index` avanza y se aplica a la máquina de estados ( `_maybe_advance_commit_index` ).

## AppendEntries RPC

- Args: `term` , `leader_id` , `prev_log_index` , `prev_log_term` , `entries` , `leader_commit` .
- Reply: `term` , `success` , `match_index` (último índice replicado en el seguidor).
- Cada seguidor valida que `prev_log_index` / `prev_log_term` coincidan con su log; si no, responde `success=False` y el líder decrementa `next_index` para reintentar desde más atrás.

## Garantías de consistencia

- Un líder nunca confirma una entrada hasta que la mayoría la replicó y pertenece al término vigente (regla de seguridad Raft).
- Al aplicar entradas, los seguidores truncan y sobrescriben cualquier divergencia a partir de `prev_log_index` , garantizando logs idénticos tras convergencia.
- Los heartbeats transportan `leader_commit` para que seguidores apliquen entradas ya confirmadas.

## Flujo de aplicación

- Mayoría alcanzada → `commit_index` avanza → evento de apply → máquina de estados (SQLite) ejecuta en orden y actualiza `last_applied` .
- Si un comando no logra mayoría en 5 s ( `replicate_entry` timeout), se reporta fallo pero puede reintentarse cuando el quórum vuelva.

## Recuperación ante fallos

- Líder cae: seguidores detectan falta de heartbeats y reinician elección con timeout aleatorio, evitando doble liderazgo.
- Seguidor rezagado: recibe `AppendEntries` con contexto de término/índice previos; si rechaza, el líder retrocede `next_index` hasta re-alinear.

## Compacción y snapshots

- Actualmente el log se mantiene completo; compacción/snapshots no están implementados, pero la máquina de estados persistente (SQLite) permite incorporarlos sin perder durabilidad.

---

# SQLite replicado con Raft

## Por qué SQLite

- Liger y embebido: cada nodo persiste sus propios metadatos sin depender de un servicio externo.

- Funciona en particiones: escrituras entran sólo vía líder; si no hay quórum, el nodo mantiene lectura local y re-aplica al reunirse.
- Complementa a MongoDB (documentos y vectores) y Redis (cache) guardando credenciales, nodos y particiones críticas.

## Qué se replica

- Usuarios: altas/bajas/actualizaciones ( `CREATE_USER` , `UPDATE_USER` , `DELETE_USER` ).
- Nodos y membresía: `ADD_NODE` , `REMOVE_NODE` , `UPDATE_NODE` (direcciones, roles, estado).
- Particiones VP-Tree: asignaciones y movimientos ( `ASSIGN_PARTITION` , `MOVE_PARTITION` , `REBALANCE` ).
- Configuración y registro de documentos: `UPDATE_CONFIG` , `REGISTER_DOCUMENT` / `UNREGISTER_DOCUMENT` (se cruza con Gossip para convergencia eventual).

## Flujo de escritura

Cliente → Líder → append al log → AppendEntries a seguidores → mayoría confirma → commit\_index avanza → PersistentStateMachine aplica en SQLite

- Cada entrada usa `Command` con `request_id` opcional para deduplicar; se persiste el resultado en `applied_requests` .
- Aplicación es ordenada y serializada con un lock interno; fallos durante apply reintentan al reiniciar porque el log es durable.

## Resiliencia y operación

- Reinicios: `current_term` / `voted_for` y el log están en disco; al boot se reanuda desde el último índice y se continua aplicando `last_applied+1` .
- Particiones de red: sin mayoría no hay nuevos commits, pero la base local sigue disponible para autenticación y lectura de metadatos ya aplicados.
- Integración con despliegue: `RAFT_ENABLED=true` en Swarm activa este modo; el repo SQLite se monta en el contenedor para no perder estado entre recreaciones.

## Beneficios para autenticación y control

- Inicio de sesión y permisos no dependen de MongoDB ni de conectividad total; mientras el líder exista con quórum, las credenciales se replican.
- El registro de nodos y particiones consistente evita que dos masters acepten cambios contradictorios; el líder único serializa las mutaciones.

# 08 Docker Swarm

## Docker Swarm en DistriSearch

Docker Swarm orquesta el despliegue de DistriSearch: gestiona contenedores distribuidos, redes overlay, descubrimiento DNS, balanceo de carga y actualizaciones sin corte de servicio.

## Por qué Swarm y no Kubernetes

- Simplicidad: configuración declarativa en `docker-compose.swarm.yml`; mismo conocimiento de Docker Compose.

- DNS integrado: resolución automática de nombres de servicio (p.ej. "master", "mongodb").
- Routing mesh nativo: cualquier nodo puede recibir tráfico y reenviarlo al servicio correcto.
- Self-healing y rolling updates sin dependencias externas.
- Menor overhead operativo para clusters medianos.

## Servicios principales

| Servicio      | Modo           | Constraint | Descripción           |
|---------------|----------------|------------|-----------------------|
| load-balancer | global         | manager    | Nginx en cada manager |
| master        | global         | manager    | Nodo Raft maestro     |
| slave         | global         | cualquiera | Backend + frontend    |
| mongodb       | global         | manager    | Replica set local     |
| redis         | replicated (1) | manager    | Cache centralizado    |
| coredns       | global         | cualquiera | DNS de respaldo       |
| dns-sync      | replicated (1) | manager    | Sincroniza zona DNS   |

## Redes

- `distrisearch-network` (overlay 10.0.10.0/24): comunicación interna entre servicios.
- `ingress-network` (overlay): routing mesh para tráfico externo.

En los siguientes archivos se profundiza cada concepto.

---

# Conceptos fundamentales de Docker Swarm

## Nodos Manager vs Worker

- **Managers:** mantienen el estado del cluster vía Raft interno, programan tareas, exponen la API y pueden ejecutar contenedores.
- **Workers:** ejecutan contenedores asignados por los managers; no participan en decisiones de orquestación.
- Recomendación: 3 o 5 managers (tolerancia 1 o 2 caídas) y N workers según carga.

## Servicios y tareas

- **Servicio:** definición declarativa (imagen, réplicas, redes, volúmenes, deploy).
- **Tarea:** instancia de un contenedor de un servicio en un nodo específico.

## Funcionalidades clave

| Característica  | Beneficio                                            |
|-----------------|------------------------------------------------------|
| DNS integrado   | "master" → IP sin configurar resolvers externos      |
| Routing mesh    | Petición a cualquier nodo llega al servicio correcto |
| Rolling updates | Actualiza réplicas de a poco sin downtime            |

|              |                                                         |
|--------------|---------------------------------------------------------|
| Self-healing | Reinicia o reubica contenedores caídos                  |
| Secrets      | Variables sensibles cifradas en memoria de contenedores |

## Secrets usados en DistriSearch

- `mongodb-password`, `jwt-secret`, `tls-cert`, `tls-key`, `mongodb-keyfile` (externos; crearlos antes del deploy).

## Inicialización típica

```
docker swarm init --advertise-addr <IP>
docker swarm join-token worker # para agregar workers
docker swarm join-token manager # para agregar managers
docker stack deploy -c docker-compose.swarm.yml distrisearch

Redes Overlay en Docker Swarm

¿Qué es una red overlay?
Una red virtual que abarca múltiples hosts físicos, permitiendo que contenedores en distintas máquinas se comuniquen como si estuvieran en la misma LAN.

Configuración en DistriSearch
```yaml
networks:
  distrisearch-network:
    driver: overlay
    attachable: true
    ipam:
      config:
        - subnet: 10.0.10.0/24
```

- `driver: overlay` : habilita comunicación multi-host.
- `attachable: true` : permite conectar contenedores manuales a la red.
- Subred fija 10.0.10.0/24 evita colisiones y facilita reglas de firewall.

Ingress network

Red overlay especial para el routing mesh; recibe tráfico publicado en puertos (80/443) y lo enruta internamente al servicio.

Comunicación entre servicios

- Los contenedores resuelven nombres de servicio ("master", "mongodb") vía DNS interno 127.0.0.11.
- El tráfico viaja cifrado (VXLAN) entre hosts; dentro de la overlay es transparente.

Aislamiento y seguridad

- Sólo contenedores conectados a `distrisearch-network` pueden alcanzarse.
- Ingress expone únicamente puertos publicados; el resto permanece privado.
- Secrets nunca se escriben en disco, sólo en memoria del contenedor.

Servicios y réplicas

Modos de despliegue

Modo	Comportamiento
<code>replicated</code>	N tareas fijas distribuidas por el scheduler
<code>global</code>	Exactamente 1 tarea por nodo que cumpla constraints

En DistriSearch:

- `slave`, `coredns`, `load-balancer`, `master`, `mongodb` usan **global** para escalar automáticamente al agregar nodos.
- `redis` y `dns-sync` usan **replicated: 1** (singleton).

Endpoint modes

Modo	Resolución DNS
<code>vip</code> (default)	Nombre → IP virtual única; Swarm balancea internamente
<code>dnssr</code>	Nombre → lista de IPs de tareas; cliente elige

- `slave` usa `vip` para balanceo transparente.
- `master` y `mongodb` usan `dnssr` para control fino de conexión (Raft, replica set).

Placement constraints

```
placement:
  constraints:
    - node.role == manager
```

Restringe el servicio a nodos manager (master, load-balancer, mongodb, redis).

Recursos

```
resources:
  limits:
    cpus: '2.0'
    memory: 2G
  reservations:
    cpus: '0.5'
    memory: 512M
```

- **limits:** máximo que puede consumir.
 - **reservations:** mínimo garantizado; el scheduler sólo coloca la tarea si el nodo tiene esos recursos libres.
-

Rolling updates y rollbacks

update_config

```
update_config:
  parallelism: 1
  delay: 30s
  failure_action: rollback
  order: start-first # o stop-first
```

Campo	Significado
parallelism	Cuántas tareas actualizar simultáneamente
delay	Espera entre lotes de tareas
failure_action	pause, continue O rollback si falla
order	start-first (inicia nueva antes de parar vieja) o stop-first

`slave` usa `start-first` para mantener capacidad durante la transición; `master` y `mongodb` usan `stop-first` para evitar conflictos de estado.

rollback_config

```
rollback_config:
  parallelism: 1
  delay: 10s
```

Define cómo revertir si `failure_action: rollback` se dispara.

restart_policy

```
restart_policy:
  condition: on-failure
  delay: 5s
  max_attempts: 5
```

Reintenta contenedores fallidos hasta 5 veces antes de detenerse.

Healthcheck y start_period

Los servicios definen `healthcheck` con `start_period` amplio (30–120 s) para permitir arranque sin falsas alarmas; si el check falla tras reintentos, Swarm reemplaza la tarea.

Ejemplo de actualización

```
docker service update --image distrisearch/slave:v2 distrisearch_slave
```

Swarm aplica las reglas de `update_config` y, si todo es exitoso, finaliza sin corte de servicio.

09 DNS Descubrimiento

DNS y Descubrimiento de Servicios

En un sistema distribuido los contenedores necesitan localizarse dinámicamente. DistriSearch combina el DNS nativo de Docker Swarm con un respaldo CoreDNS para garantizar alta disponibilidad del descubrimiento.

Estrategia de resolución

1. **Docker DNS (127.0.0.11)** – primera opción; automático en Swarm.
2. **CoreDNS backup** – si Docker DNS falla, se consulta `coredns:53`.
3. **Caché local / `/etc/hosts`** – último recurso estático.

Componentes

Componente	RoI
Docker DNS	Resuelve nombres de servicio a VIP o lista de tareas
CoreDNS	Respaldo configurable con zonas <code>distrisearch.local</code> y <code>services.distrisearch.local</code>
dns-sync	Daemon que sincroniza IPs de servicios Swarm a zona CoreDNS cada 30 s

Variables de entorno relevantes

- `DNS_FALLBACK_ENABLED=true` – activa cadena de fallback.
- `COREDNS_HOST=coredns` – dirección del servidor de respaldo.

En los siguientes archivos se profundiza cada capa.

DNS interno de Docker Swarm

Cómo funciona

Cada contenedor en una red overlay tiene `127.0.0.11` como resolver; las consultas se envían al daemon de Docker que mantiene registros de todos los servicios.

Tipos de resolución

Consulta	Resultado
----------	-----------

master	VIP única (si endpoint_mode: vip) balanceada internamente
tasks.master	Lista de IPs de todas las tareas corriendo
master.1.abc123	IP de contenedor específico

endpoint_mode

- **vip** (default): un registro A con IP virtual; Swarm balancea internamente.
- **dnsrr**: múltiples registros A; el cliente elige (usado en `master`, `mongodb` para Raft/replica set).

Ejemplo de flujo

```
Slave → resolver 127.0.0.11 → "master" → 10.0.10.5
      ← respuesta IP          ← Docker DNS
Slave → TCP 10.0.10.5:8001 → Master container
```

Limitaciones

- Dependiente del health del Swarm manager.
- Puede saturarse bajo alto volumen de queries.
- Por eso DistriSearch habilita un respaldo CoreDNS.

CoreDNS como DNS de respaldo

Cuando Docker DNS falla (saturación, partición, bug), CoreDNS responde con zonas pre-sincronizadas.

Archivo Corefile

```
distrisearch.local:53 {
    log
    errors
    file /etc/coredns/zones/distrisearch.local.zone
    loadbalance round_robin
    cache 30
    forward . 127.0.0.11 ; reenvía a Docker DNS si no está en zona
}

services.distrisearch.local:53 { ... }

. {
    forward . 127.0.0.11 8.8.8.8 8.8.4.4
    cache 60
}
```

Plugins clave

Plugin	Función
--------	---------

file	Carga zona desde archivo
loadbalance	Rotación round-robin de registros A
cache	Reduce latencia y carga
forward	Reenvía consultas no resueltas
rewrite	Mapea <code>master.distrisearch.local</code> → <code>master</code> para compatibilidad

dns-sync daemon

`sync_dns_zone.py` corre cada `SYNC_INTERVAL` (30 s):

1. Conecta a Docker API.
2. Lista servicios y extrae IPs de tareas corriendo.
3. Genera zona con registros A dinámicos.
4. Escribe `/zones/distrisearch.local.zone`; CoreDNS recarga automáticamente.

Despliegue

- `coredns` modo global: una instancia por nodo para resolución local rápida.
- `dns-sync` modo replicated (1): singleton en manager con acceso a Docker socket.

Service Discovery y cadena de fallback

Cadena de resolución

1. Docker DNS `127.0.0.11` → éxito → usar IP
↓ fallo
2. CoreDNS `coredns:53` → éxito → usar IP
↓ fallo
3. Caché local / `/etc/hosts` → éxito → usar IP
↓ fallo
4. Error de resolución

Variables de entorno

Variable	Valor ejemplo	Descripción
DNS_FALLBACK_ENABLED	true	Activa cadena
COREDNS_HOST	coredns	Host del servidor backup
MASTER_SERVICE	master	Nombre de servicio master
MONGODB_URI	<code>mongodb://mongodb:27017</code>	Resuelto por DNS

Integración en la aplicación

- Al iniciar, el backend intenta resolver `master`; si falla, usa CoreDNS.

- Conexiones a MongoDB y Redis usan el mismo flujo.
- El healthcheck (`/health`) valida conectividad DNS; si falla, Swarm reinicia el contenedor.

Beneficios

- Resiliencia ante fallos del daemon Docker.
- Permite operar en modo degradado durante particiones de red.
- Caché reduce latencia de resolución en operaciones frecuentes.

10 Load Balancer

Load Balancer en DistriSearch

Nginx actúa como punto de entrada único, distribuyendo tráfico entre los slaves y proporcionando SSL termination, rate limiting y health checks.

Arquitectura

```
Cliente → Nginx (puerto 80/443) → slaves (8000/8443)
                                     → master (8001) para API admin
```

Características

Función	Implementación
Balanceo	Upstreams dinámicos con resolución DNS
SSL	Terminación TLS 1.2/1.3 en load-balancer
Rate limiting	100 req/s API, 10 req/s uploads
Health checks	Verifica <code>/health</code> cada 30 s
Failover	<code>proxy_next_upstream</code> reintenta en otros backends

Despliegue Swarm

- Modo `global` en managers: un contenedor Nginx por manager para HA.
- Usa `ingress-network` para recibir tráfico externo y `distrisearch-network` para backends.

Detalles en los siguientes archivos.

Configuración de Nginx

Resolución dinámica

```
resolver 127.0.0.11 valid=10s ipv6=off;
```

```
resolver_timeout 5s;
```

Usa el DNS interno de Docker; re-resuelve cada 10 s para detectar nuevos slaves.

Upstreams

```
upstream master_api {  
    server master:8001 max_fails=3 fail_timeout=30s;  
    keepalive 16;  
}
```

Se incluyen archivos dinámicos desde `/etc/nginx/conf.d/upstreams/` generados por scripts.

Proxy settings principales

```
location /api/ {  
    proxy_pass http://$slave_api_backend;  
    proxy_http_version 1.1;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_next_upstream error timeout http_502 http_503 http_504;  
    proxy_next_upstream_tries 3;  
}
```

- `proxy_next_upstream` reintenta en otro backend si falla.
- Headers X-Real-IP y X-Forwarded-For preservan IP del cliente.

Optimizaciones

- `worker_connections 4096` , `epoll` , `multi_accept` .
- Gzip para JSON, JS, CSS.
- `client_max_body_size 100M` (500 M para uploads).

Routing Mesh de Docker Swarm

Concepto

Swarm publica puertos en todos los nodos; cualquier petición a `<nodo>:80` se enruta internamente al servicio correcto sin importar dónde corra.

Configuración en compose

```
ports:  
  - target: 80  
    published: 80
```

```
protocol: tcp
mode: ingress
```

- `mode: ingress` : habilita routing mesh (default).
- `mode: host` : expone solo en nodo local (sin mesh).

Flujo

Cliente → 192.168.1.11:80 (worker1) → ingress VIP → load-balancer container (cualquier nodo)

Incluso si load-balancer corre en otro host, la petición llega.

Combinación con Nginx

Nginx recibe vía routing mesh y luego balancea internamente hacia slaves usando upstreams dinámicos.

Health Checks

En Nginx

```
location /health {
    return 200 'OK';
    add_header Content-Type text/plain;
}
```

Endpoint simple para que Swarm verifique el contenedor.

En Docker Swarm

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 10s
```

- `start_period` : ignora fallos durante arranque.
- Tras 3 fallos consecutivos, Swarm reinicia o reemplaza la tarea.

Beneficios

- Backends no saludables se excluyen automáticamente del balanceo.
- Self-healing: Swarm recrea tareas fallidas.

Health en backend

Cada slave expone `/api/v1/health` con conexiones a MongoDB, Redis y estado general; el load-balancer redirige a otros si falla.

SSL/TLS Termination

Terminación en load-balancer

Nginx maneja HTTPS y comunica con backends en HTTP interno; simplifica gestión de certificados.

Configuración

```
ssl_certificate /etc/nginx/ssl/server.crt;
ssl_certificate_key /etc/nginx/ssl/server.key;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-...;
ssl_session_cache shared:SSL:50m;
```

Generación de certificados (desarrollo)

`generate-ssl.sh` :

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout server.key -out server.crt \
  -subj "/CN=distrisearch.local"
```

Para producción usar Let's Encrypt o CA interna.

Secrets en Swarm

```
secrets:
  tls-cert:
    external: true
  tls-key:
    external: true
```

Crear con `docker secret create tls-cert server.crt` antes del deploy.

Headers de seguridad

```
add_header X-Frame-Options "SAMEORIGIN";
add_header X-Content-Type-Options "nosniff";
add_header X-XSS-Protection "1; mode=block";
```

```
# 11 Almacenamiento
```

```
# Almacenamiento en DistriSearch
```

DistriSearch usa una estrategia de almacenamiento multi-capa diseñada para disponibilidad y tolerancia a particiones (AP).

```
## Capas de almacenamiento
```

```
| Capa | Tecnología | Datos | Consistencia |  
|-----|-----|-----|-----|  
| Metadatos críticos | SQLite + Raft | Usuarios, nodos, particiones | Fuerte (mayoría) |  
| Documentos | MongoDB local por nodo | Contenido, vectores | Local |  
| Cache | Redis local | Resultados de búsqueda | Efímero |  
| Registro usuario-docs | UserDocumentRegistry | Mapeo user→docs | Eventual (Gossip) |
```

```
## Por qué almacenamiento local
```

- Cada slave tiene su propio MongoDB: **no** hay punto único de fallo.
- Durante particiones, el nodo sigue sirviendo sus documentos.
- El VP-Tree del master decide la ubicación; **la** replicación crea copias en otros nodos.

En los siguientes archivos se profundiza cada componente.

```
---
```

```
# MongoDB Local por Nodo
```

```
## Arquitectura
```

Cada slave tiene su propia instancia MongoDB corriendo como contenedor o sidecar.

worker1: MongoDB (mongodb-data-worker1) ← slave-worker1 worker2: MongoDB (mongodb-data-worker2) ← slave-worker2 ...

```
## Conexión  
```bash  
MONGODB_URI=mongodb://mongodb-local:27017/distrisearch
```

El nombre `mongodb-local` resuelve al contenedor local vía red overlay.

## Qué se almacena

- Documentos (contenido original, metadatos).
- Vectores TF-IDF, MinHash, LDA.
- Índices para búsqueda local.

## Beneficios

- Sin SPOF: la caída de un MongoDB afecta solo ese nodo.
- Escalabilidad horizontal: agregar nodos = agregar almacenamiento.
- Partición de red: el nodo sigue sirviendo sus datos locales.

## Replica set (opcional)

En despliegues con más disponibilidad, se puede configurar un replica set por zona con `init-replica.js`.

---

# Redis Cache

## Propósito

Acelerar lecturas frecuentes: resultados de búsqueda, embeddings precalculados, sesión de usuario.

## Configuración

```
redis:
 image: redis:7-alpine
 command: ["redis-server", "--appendonly", "yes"]
 volumes:
 - redis-data:/data
```

- `appendonly yes` : persistencia AOF para recuperar datos tras reinicio.

## Conexión

```
REDIS_URL=redis://redis:6379
```

## Estrategia de caché

- TTL cortos (30–60 s) para resultados de búsqueda.
- Invalidación explícita al subir/eliminar documentos.
- Cache aside: si no está en Redis, consultar MongoDB y guardar.

## Despliegue Swarm

- Modo `replicated: 1` en manager (singleton).
  - Para HA se puede usar Redis Sentinel o Redis Cluster, pero agrega complejidad.
- 

# SQLite para Usuarios y Metadatos

## Por qué SQLite

- Embebido: no requiere servidor externo.
- Funciona durante particiones: cada nodo tiene copia local.
- Replicado vía Raft para consistencia fuerte entre mayoría.

## Qué se almacena

- Usuarios: credenciales, roles, permisos.

- Nodos: membresía del cluster, direcciones, estados.
- Particiones: asignaciones VP-Tree.
- Configuración: parámetros del sistema.
- applied\_requests: deduplicación de comandos Raft.

## Flujo de escritura

Cliente → Líder Raft → append log → replicar a seguidores → commit → aplicar en SQLite

## Repositorios

- SQLiteUserRepository : CRUD de usuarios.
- SQLiteNodeRepository : CRUD de nodos.
- SQLitePartitionRepository : CRUD de particiones.

## Beneficios para AP

- Autenticación local: el nodo puede validar JWT con su SQLite aunque esté particionado.
- Metadatos de cluster disponibles para operaciones de sólo lectura.

---

# UserDocumentRegistry

## Propósito

Mapear qué documentos pertenecen a qué usuario y en qué nodo están, permitiendo responder «qué documentos tiene el usuario X» desde cualquier nodo.

## Por qué no en MongoDB

Necesita acceso cross-partition: un usuario puede tener docs en varios nodos. Centralizar crearía SPOF.

## Modelo de datos

```
@dataclass
class DocumentRegistryEntry:
 user_id: str
 document_id: str
 node_id: str
 title: Optional[str]
 is_deleted: bool
 vector_clock: Dict[str, int]
```

## Consistencia eventual (Gossip)

- Cada nodo mantiene su copia local en SQLite.
- Periódicamente intercambia entradas con peers aleatorios.
- Conflictos se resuelven por vector clocks (last-write-wins si empatan).



## Operaciones

Método	Acción
register_document	Agrega entrada local y propaga
unregister_document	Soft-delete (tombstone)
get_user_documents	Lista docs de un usuario desde copia local
sync_with_peer	Intercambio Gossip

## Beneficios

- Lectura local instantánea.
- Tolerante a particiones: eventualmente converge al reunirse.

# 12 Gossip Protocol

## Protocolo Gossip en DistriSearch

Gossip (o epidemic protocol) propaga información de forma descentralizada: cada nodo intercambia datos con peers aleatorios hasta que todos convergen.

### Dónde se usa

- **UserDocumentRegistry**: sincroniza mapeo usuario→documentos entre nodos.
- Potencialmente: health status, estadísticas de carga.

## Características

Aspecto	Valor
Consistencia	Eventual
Tolerancia a fallos	Alta (sin coordinador central)
Latencia de propagación	$O(\log N)$ rondas para $N$ nodos
Overhead	Bajo (mensajes pequeños, periódicos)

## Flujo básico

1. Nodo A selecciona peer aleatorio B.
2. A envía resumen (vector clocks) de sus entradas.
3. B responde con diferencias.
4. Ambos actualizan sus copias locales.
5. Repetir cada intervalo (ej. 5–30 s).

En los siguientes archivos se detalla la implementación.

---

# Consistencia Eventual

## Definición

Si no hay nuevas escrituras, eventualmente todas las réplicas convergen al mismo estado.

## Trade-off CAP en DistriSearch

DistriSearch elige **AP** (Available + Partition-tolerant):

- Durante particiones, cada nodo sigue sirviendo lecturas/escrituras locales.
- Tras reunirse, Gossip sincroniza diferencias.

## Cuándo es aceptable

- Listado de documentos de un usuario: tolera ver un doc nuevo con pequeño retraso.
- Estadísticas de cluster: no crítico que sean instantáneas.

## Cuándo NO es aceptable

- Autenticación: por eso usuarios van en SQLite+Raft (consistencia fuerte).
- Asignación de particiones: también Raft.

## Resolución de conflictos

- Vector clocks: detectan concurrencia.
- Last-write-wins si clocks son incomparables.
- Soft-deletes (tombstones) para evitar resurrección de datos borrados.

---

# Sincronización del UserDocumentRegistry

## Modelo

UserDocumentRegistry mantiene entradas (user\_id, document\_id, node\_id, vector\_clock) en SQLite local.

## Algoritmo Gossip

```
async def sync_with_peer(peer_address):
 my_summary = get_vector_clock_summary()
 peer_summary = await send_summary_to_peer(peer_address, my_summary)
 diff = compute_diff(my_summary, peer_summary)
 entries_to_send = get_entries(diff.missing_on_peer)
 entries_received = await exchange_entries(peer_address, entries_to_send)
 merge_entries(entries_received)
```

## Frecuencia

- Intervalo configurable (default 10–30 s).
- Peer aleatorio por ronda para distribuir carga.

## Merge con vector clocks

- Si mi clock domina → conservo mi versión.
- Si peer domina → adopto versión del peer.
- Si concurrentes → last-write-wins por timestamp.

## Tombstones

- `is_deleted=True` marca borrados.
- Se propagan igual que entradas activas.
- Purgables tras TTL largo (ej. 7 días) para evitar resurrección.

## Beneficios

- Sin coordinador: cualquier nodo puede iniciar sync.
- Escala bien:  $O(\log N)$  rondas para convergencia completa.

# 13 CAP Theorem

## Teorema CAP en DistriSearch

El teorema CAP establece que un sistema distribuido sólo puede garantizar dos de tres propiedades simultáneamente:

- **Consistency**: todos los nodos ven los mismos datos al mismo tiempo.
- **Availability**: toda petición recibe respuesta (aunque no sea la más reciente).
- **Partition tolerance**: el sistema sigue funcionando pese a pérdida de mensajes entre nodos.

## Elección de DistriSearch: AP

DistriSearch prioriza **Disponibilidad** y **Tolerancia a particiones**:

- Un sistema de búsqueda debe responder aunque algunos nodos estén desconectados.
- Consistencia eventual es aceptable para documentos y resultados de búsqueda.
- Donde se requiere consistencia fuerte (usuarios, particiones) se usa Raft.

## Resumen por componente

Componente	Modelo CAP	Justificación
SQLite + Raft	CP	Requiere mayoría para escribir
MongoDB local	AP	Cada nodo sirve sus datos
UserDocumentRegistry	AP	Gossip converge eventualmente
Búsqueda	AP	Responde con datos locales

Detalles en los siguientes archivos.

---

# Disponibilidad y Tolerancia a Particiones

## Por qué AP para búsqueda

- Los usuarios esperan resultados aunque parte del cluster esté caído.
- Un sistema CP bloquearía peticiones hasta recuperar quorum.
- Búsqueda semejante tolera resultados ligeramente desactualizados.

## Operación durante partición

```
[Partición de red]
Grupo A (master + slave1) | Grupo B (slave2, slave3)
- Raft sin quorum → no escribe SQLite
- Lecturas locales OK
- Búsqueda en docs locales OK
```

Cada grupo sigue sirviendo sus documentos. Al reunirse, Gossip sincroniza y Raft elige nuevo líder si es necesario.

## Modo degradado

- Autenticación: funciona si el nodo tiene usuarios en SQLite local (datos replicados previamente).
- Escrituras de usuarios/nodos: bloqueadas sin mayoría Raft.
- Subida de documentos: se almacena localmente; replicación se completa al reunirse.

## Comparación con sistemas CP

Sistema	Comportamiento en partición
CP (ej. Spanner)	Rechaza escrituras sin quorum
AP (DistriSearch)	Acepta localmente, sincroniza después

# Trade-offs de Diseño en DistriSearch

## Decisiones clave

Decisión	Razón
SQLite + Raft para usuarios	Consistencia fuerte evita credenciales divergentes
MongoDB local	Disponibilidad; sin SPOF
Gossip para registry	Baja latencia de lectura, tolerancia a particiones
Replicación por afinidad	Copias en nodos con docs similares para búsquedas rápidas

## Consistencia vs latencia

- Escritura de usuario: espera commit Raft (~50–100 ms con mayoría local).

- Subida de documento: respuesta inmediata tras guardar local; replicación async.
- Búsqueda: lee caché/índice local; resultados en <100 ms típicamente.

## Recuperación tras partición

1. Raft detecta nuevo quorum y elige líder (si cambió).
2. Gossip sincroniza UserDocumentRegistry.
3. Re-replicación de documentos si alguna réplica se perdió.
4. VP-Tree actualiza con nodos disponibles.

## Garantías ofrecidas

- Documentos subidos nunca se pierden si al menos un nodo con copia sobrevive.
- Usuarios autenticados en cualquier nodo que tenga la copia SQLite.
- Búsquedas devuelven resultados del subconjunto accesible.

# 14 Despliegue

## Despliegue de DistriSearch

Esta sección cubre el proceso de despliegue en un cluster Docker Swarm multi-host.

### Resumen de pasos

0. **Configurar firewall** (si hay problemas de red - script dedicado o integrado).
1. Preparar nodos (Docker, límites, directorios).
2. Inicializar Swarm en el manager (incluye configuración de firewall).
3. Unir workers al cluster (incluye configuración de firewall).
4. Desplegar infraestructura (red overlay).
5. Desplegar Master.
6. Desplegar Slaves (Backend + Frontend integrado).
7. Verificar salud del cluster.

### Scripts de despliegue

Los scripts se encuentran en `deploy/manual-swarm/scripts/` :

Script	Descripción
<code>00-configure-firewall.sh</code>	Configura firewall para Docker Swarm (UFW/iptables/firewalld)
<code>01-prepare-node.sh</code>	Prepara nodos con Docker y configuraciones
<code>02-init-swarm.sh</code>	Inicializa Swarm en el manager (configura firewall automáticamente)
<code>03-join-swarm.sh</code>	Une workers al cluster (configura firewall automáticamente)
<code>04-deploy-infrastructure.sh</code>	Crea red overlay
<code>05-deploy-master.sh</code>	Despliega el coordinador Master
<code>06-deploy-slave.sh</code>	Despliega Slaves (opciones: <code>--update</code> , <code>--rebuild</code> , <code>--clean</code> )

07-verify-cluster.sh	Verifica estado del cluster
08-test-distributed-features.sh	Prueba funcionalidades distribuidas
09-cleanup.sh	Limpia completamente el despliegue

## Opciones de 06-deploy-slave.sh

```
./06-deploy-slave.sh [NUM_REPLICAS] [--update] [--rebuild] [--clean]
```

Opción	Descripción
NUM_REPLICAS	Número de slaves a desplegar (default: número de workers)
--update	Forzar actualización de imagen en servicios existentes
--rebuild	Reconstruir imagen antes de desplegar
--clean	Eliminar todos los servicios slave antes de desplegar

## Opciones de despliegue

Método	Uso
docker stack deploy	Usa docker-compose.swarm.yml
Scripts manuales	Control fino, idempotentes

## Puertos requeridos

### Docker Swarm (internos)

Puerto	Uso
2377/tcp	Gestión Swarm
7946/tcp+udp	Comunicación entre nodos
4789/udp	Red overlay VXLAN (Routing Mesh) - <b>CRÍTICO</b>

### DistriSearch

Puerto	Uso
8001/tcp	Master API
8081-8089/tcp	Slave HTTP (Frontend)
4431-4439/tcp	Slave HTTPS (Frontend)
8002-8009/tcp	Slave API (Backend)

27017/tcp	MongoDB (solo red interna)
6379/tcp	Redis (solo red interna)

### Mapeo de puertos por Slave

Slave	HTTP	HTTPS	API
slave-1	8081	4431	8002
slave-2	8082	4432	8003
slave-N	8080+N	4430+N	8001+N

## Troubleshooting de red

Si hay problemas de conectividad en el routing mesh:

1. Ejecutar `00-configure-firewall.sh` en **TODOS** los nodos
2. Reiniciar Docker: `sudo systemctl restart docker`
3. Verificar red overlay: `docker network inspect distrisearch-network`
4. Probar conectividad UDP: `nc -zuv <otro-nodo> 4789`

Detalles paso a paso en los siguientes archivos.

# Configuración del Firewall

## Propósito

El script `00-configure-firewall.sh` configura los puertos necesarios para que Docker Swarm funcione correctamente, especialmente el **Routing Mesh** que requiere el puerto 4789/UDP para la red overlay VXLAN.

## Cuándo usar

Ejecutar este script en **TODOS los nodos** cuando:

- Hay problemas de conectividad entre nodos del Swarm
- Los servicios no son accesibles desde otros nodos
- El Routing Mesh no funciona correctamente
- Los contenedores no pueden comunicarse entre nodos

## Tipos de firewall soportados

El script detecta automáticamente qué firewall está instalado:

Firewall	Distribuciones
UFW	Ubuntu, Debian
firewalld	RHEL, CentOS, Fedora
iptables	Fallback universal

## Puertos configurados

### Docker Swarm (obligatorios)

```
2377/tcp - Gestión del cluster Swarm
7946/tcp - Comunicación entre nodos
7946/udp - Comunicación entre nodos
4789/udp - Red overlay VXLAN (CRÍTICO para Routing Mesh)
```

### DistriSearch

```
8001/tcp - Master API
8081-8089/tcp - Slave HTTP (Frontend Nginx)
4431-4439/tcp - Slave HTTPS (Frontend Nginx)
8002-8009/tcp - Slave API (Backend FastAPI)
```

### Servicios internos (solo red privada)

```
27017/tcp - MongoDB (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
6379/tcp - Redis (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
```

## Ejecución

```
Requiere permisos de root
sudo ./00-configure-firewall.sh
```

## Ejemplo de salida

```
=====
DistriSearch - Configurar Firewall
=====
```

Puertos que se configurarán:

Docker Swarm:

- 2377/tcp : Gestión del cluster
- 7946/tcp : Comunicación entre nodos
- 7946/udp : Comunicación entre nodos
- 4789/udp : Red overlay (VXLAN) - CRÍTICO para routing mesh

DistriSearch:

- 8001/tcp : Master API
- 8081-8089 : Slave HTTP (Frontend)
- 4431-4439 : Slave HTTPS (Frontend)
- 8002-8009 : Slave API (Backend)

[INFO] Configurando UFW...



```
[INFO] UFW configurado correctamente

[INFO] Verificando puertos...

✓ Puerto 2377 (Swarm Management) - Escuchando
o Puerto 8001 (Master API) - No hay servicio escuchando

=====
Firewall Configurado
=====
```

## Integración con scripts 02 y 03

Los scripts `02-init-swarm.sh` y `03-join-swarm.sh` **ya incluyen** configuración automática del firewall. El script `00-configure-firewall.sh` es útil para:

1. Diagnóstico de problemas de red
2. Verificación manual de puertos
3. Nodos que tuvieron problemas iniciales

## Troubleshooting

### El Routing Mesh no funciona

```
1. Ejecutar en TODOS los nodos
sudo ./00-configure-firewall.sh

2. Reiniciar Docker
sudo systemctl restart docker

3. Verificar la red overlay
docker network inspect distrisearch-network

4. Probar conectividad UDP
nc -zuv <otro-nodo> 4789
```

### Verificar reglas aplicadas

```
UFW
sudo ufw status numbered

iptables
sudo iptables -L INPUT -n --line-numbers

firewalld
sudo firewall-cmd --list-all
```

---

## # Preparación de Imágenes Docker

### ## Construir imágenes

```
```bash
cd /ruta/a/DistriSearch
docker build -f docker/master/Dockerfile -t distrisearch/master:latest .
docker build -f docker/slave/Dockerfile -t distrisearch/slave:latest .
```
```

### ## Guardar para transferir

```
```bash
docker save distrisearch/master:latest | gzip > distrisearch-master.tar.gz
docker save distrisearch/slave:latest | gzip > distrisearch-slave.tar.gz
```
```

### ## Transferir a nodos

```
```bash
for host in manager1 worker1 worker2; do
    scp distrisearch-*.tar.gz $host:~
done
```
```

### ## Cargar en cada nodo

```
```bash
gunzip -c ~/distrisearch-master.tar.gz | docker load
gunzip -c ~/distrisearch-slave.tar.gz | docker load
docker images | grep distrisearch
```
```

### ## Alternativa: registry privado

Usar un Docker Registry local evita transferencias manuales:

```
```bash
docker tag distrisearch/master:latest registry.local:5000/distrisearch/master:latest
docker push registry.local:5000/distrisearch/master:latest
```
```

---

## # Inicialización de Docker Swarm

### ## Script automatizado

```
```bash
./02-init-swarm.sh [IP_DEL_MANAGER]
```
```

El script realiza:

1. Detecta IP automáticamente si no se especifica
2. Verifica Docker funcionando

3. **\*\*Configura firewall automáticamente\*\*** (UFW/iptables)
4. Inicializa Swarm
5. Crea red overlay `distrisearch-network` (10.0.10.0/24)
6. Guarda tokens en `/opt/distrisearch/config/swarm-tokens.env`

## ## Configuración de firewall integrada

El script configura automáticamente los puertos requeridos:

- Docker Swarm: 2377/tcp, 7946/tcp+udp, 4789/udp
- DistriSearch: 8001, 8081-8089, 4431-4439, 8002-8009

## ## En el manager principal (manual)

```
```bash
```

```
docker swarm init --advertise-addr 192.168.1.10
```

```
```
```

- `--advertise-addr`: IP accesible por otros nodos.
- Guarda el token de salida para unir workers.

## ## Crear red overlay (manual)

```
```bash
```

```
docker network create \  
  --driver overlay \  
  --attachable \  
  --subnet 10.0.10.0/24 \  
  distrisearch-network
```

```
```
```

## ## Obtener tokens

```
```bash
```

```
docker swarm join-token worker # para workers
```

```
docker swarm join-token manager # para managers adicionales
```

```
```
```

## ## Tokens guardados automáticamente

```
```bash
```

```
cat /opt/distrisearch/config/swarm-tokens.env
```

```
```
```

## ## Unir workers

En cada worker usar `03-join-swarm.sh` o:

```
```bash
```

```
docker swarm join --token SWMTKN-1-xxx... 192.168.1.10:2377
```

```
```
```

## ## Verificar

```
```bash
```

```
docker node ls
```

# ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS
# abc123 *	manager1	Ready	Active	Leader
# def456	worker1	Ready	Active	
# ghi789	worker2	Ready	Active	

```
```
```

```

Alta disponibilidad de managers
Para tolerancia a 1 fallo, usar 3 managers; para 2 fallos, 5 managers.
```bash
docker swarm join --token <manager-token> 192.168.1.10:2377

---

# Despliegue de Servicios

## Script automatizado para Slaves

```bash
./06-deploy-slave.sh [NUM_REPLICAS] [--update] [--rebuild] [--clean]
```

### Opciones disponibles
Opción	Descripción
`NUM_REPLICAS`	Número de slaves (default: número de workers)
`--update`	Forzar actualización de imagen existente
`--rebuild`	Reconstruir imagen desde Dockerfile
`--clean`	Eliminar todos los slaves antes de desplegar

### Qué despliega por cada Slave:
1. **MongoDB LOCAL** (`slave{N}-mongodb`) - Almacena documentos locales
2. **Redis LOCAL** (`slave{N}-redis`) - Caché local
3. **Slave Service** (`distrisearch-slave-{N}`) - Backend+Frontend integrado

## Arquitectura de cada Slave

...



distrisearch-slave-N



Nginx (80/443)  
Frontend



FastAPI (8000)  
Backend



MongoDB LOCAL



Redis LOCAL


...

## Mapeo de puertos por Slave

Slave	HTTP	HTTPS	API
slave-1	8081	4431	8002
slave-2	8082	4432	8003

```

```
| slave-N | 8080+N | 4430+N | 8001+N |
```

```
## Desplegar Master
```

```
```bash
./05-deploy-master.sh
```
```

```
0 manualmente:
```

```
```bash
docker service create \
 --name distrisearch-master \
 --network distrisearch-network \
 --publish published=8001,target=8001 \
 --env NODE_ROLE=master \
 --env RAFT_ENABLED=true \
 distrisearch/master:latest
```
```

```
## Desplegar Slaves (manual)
```

```
```bash
docker service create \
 --name distrisearch-slave-1 \
 --network distrisearch-network \
 --publish published=8081,target=80 \
 --publish published=4431,target=443 \
 --publish published=8002,target=8000 \
 --env NODE_ID=slave-1 \
 --env NODE_ROLE=slave \
 --env MASTER_HOST=distrisearch-master \
 --env MONGODB_URI=mongodb://slave1-mongodb:27017/distrisearch_slave1 \
 --env REDIS_URL=redis://slave1-redis:6379 \
 distrisearch/slave:latest
```
```

```
## Verificar despliegue
```

```
```bash
Ver servicios
docker service ls

Ver tareas
docker service ps distrisearch-slave-1

Ver logs
docker service logs -f distrisearch-slave-1
```
```

```
## Actualizar slaves existentes
```

```
```bash
```

```

Forzar actualización de imagen
./06-deploy-slave.sh --update

Reconstruir imagen y desplegar
./06-deploy-slave.sh --rebuild

Eliminar todo y redespargar
./06-deploy-slave.sh --clean
...

Con docker stack

```bash
docker stack deploy -c docker-compose.swarm.yml distrisearch

---

# Verificación del Cluster

## Comandos básicos
```bash
docker node ls # nodos del Swarm
docker service ls # servicios desplegados
docker service ps <svc> # tareas de un servicio
...

Health checks
```bash
curl -k https://worker1:8443/health
curl http://manager1:8001/health
...

Deben devolver `OK` o JSON con estado.

## Logs
```bash
docker service logs distrisearch_slave --tail 100 -f
docker logs distrisearch-master
...

Probar búsqueda distribuida
```bash
curl -k https://worker1:8443/api/v1/search?q=presupuesto
...

## Troubleshooting común
| Problema | Solución |
|-----|-----|
| Nodo no se une | Verificar puertos 2377, 7946, 4789 |
| DNS no resuelve | Revisar red overlay y CoreDNS |
| MongoDB no conecta | Comprobar nombre en red overlay |
| Master no responde | Ver logs Raft, verificar quorum |

```

```
## Escalar slaves
```

```
```bash
```

```
docker service scale distriSearch_slave=5
```

```
15 API Backend
```

```
API Backend de DistriSearch
```

El backend está construido con **FastAPI** y expone endpoints REST para gestión de documentos, búsqueda, autenticación y administración del cluster.

```
Características
```

- Async/await nativo para alta concurrencia.
- Documentación automática en `/docs` (Swagger) y /redoc`.`
- Validación con Pydantic.
- Middleware: CORS, GZip, rate limiting.
- WebSocket para actualizaciones en tiempo real.

```
Estructura
```

```
```
```

```
backend/app/
```

```
|— main.py          # Entry point, lifespan
|— config.py        # Settings via env vars
|— api/
|   |— router.py    # Agrupa todos los routers
|   |— auth.py      # Login, registro, JWT
|   |— documents.py # CRUD documentos
|   |— search.py    # Búsqueda semántica
|   |— cluster.py   # Admin cluster
|   |— health.py    # Health checks
|   |— websocket.py # WS para dashboard
|— storage/         # MongoDB, SQLite, Redis
|— core/            # Vectorización, particionamiento
|— distributed/     # Raft, Gossip, comunicación
|—
```

Detalles en los siguientes archivos.

```
---
```

```
# Estructura FastAPI
```

```
## Entry point: main.py
```

```
```python
```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
 await init_dependencies(settings)
 yield
 await shutdown_dependencies()
```

```

app = FastAPI(title="DistriSearch API", lifespan=lifespan)
...

El lifespan inicializa conexiones a MongoDB, SQLite, Redis y registra el nodo en el cluster.

Configuración: config.py
Usa `pydantic.BaseSettings` para cargar variables de entorno:
- `NODE_ROLE`, `NODE_ID`, `MONGODB_URI`, `REDIS_URL`, `MASTER_HOST`, etc.

Routers
Cada módulo en `api/` define un `APIRouter`; se incluyen en `router.py`:
```python
api_router.include_router(auth.router, prefix="/auth", tags=["auth"])
api_router.include_router(documents.router, prefix="/documents", tags=["documents"])
api_router.include_router(search.router, prefix="/search", tags=["search"])
```

Middleware
- `CORSMiddleware`: permite orígenes configurables.
- `GZipMiddleware`: comprime respuestas >1 KB.
- Custom middleware para logging, timing, request-id.

Endpoints Principales

Documentos
Método	Ruta	Descripción
POST	/api/v1/upload	Subir documento (multipart, hasta **500MB**, cualquier tipo)
GET	/api/v1/documents	Listar docs del usuario
GET	/api/v1/documents/{id}	Obtener metadatos del documento
GET	/api/v1/documents/{id}/download	**Descargar archivo original**
DELETE	/api/v1/documents/{id}	Eliminar documento

Tipos de archivo soportados
- **Texto/Código**: TXT, MD, JSON, XML, CSV, PY, JS, TS, HTML, CSS, etc.
- **Documentos**: PDF, DOCX, XLSX, PPTX
- **Binarios**: Cualquier tipo (búsqueda por nombre de archivo)

Límites
- Tamaño máximo: **500 MB** por archivo
- Archivos binarios sin contenido extraíble: indexación por nombre de archivo

Búsqueda
Método	Ruta	Descripción
GET	/api/v1/search?q=...	Búsqueda semántica
POST	/api/v1/search	Búsqueda con filtros avanzados

Búsqueda en archivos binarios

```



Para archivos sin contenido textual, la búsqueda se realiza sobre:

- Nombre del archivo
- Extensión
- Metadatos (fecha, tamaño, propietario)

## ## Cluster (admin)

| Método | Ruta                       | Descripción         |
|--------|----------------------------|---------------------|
| -----  | -----                      | -----               |
| GET    | /api/v1/cluster/nodes      | Lista nodos         |
| GET    | /api/v1/cluster/partitions | Particiones VP-Tree |
| POST   | /api/v1/cluster/rebalance  | Disparar rebalanceo |

## ## Health

| Método | Ruta                | Descripción                    |
|--------|---------------------|--------------------------------|
| -----  | -----               | -----                          |
| GET    | /health             | Liveness simple                |
| GET    | /api/v1/health/live | Health check para Docker/Swarm |
| GET    | /api/v1/health      | Detallado (DB, Redis)          |

## ## WebSocket

- `/ws/dashboard`: eventos de cluster en tiempo real.

---

## # Autenticación

### ## Flujo JWT

1. `POST /api/v1/auth/login` con `username` y `password`.
2. Backend valida contra SQLite (usuarios replicados por Raft).
3. Devuelve `access\_token` (JWT) con expiración.
4. Cliente incluye header `Authorization: Bearer <token>` en peticiones.

### ## Registro

`POST /api/v1/auth/register` crea usuario; se replica vía Raft a todos los nodos.

### ## Dependencia de seguridad

```python

```
async def get_current_user(token: str = Depends(oauth2_scheme)) -> UserModel:
    payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    user = await user_repo.get_by_id(payload["sub"])
    if not user:
        raise HTTPException(401)
    return user
```

```

### ## Tokens y secrets

- `JWT\_SECRET` se almacena como Docker secret.
- Expiración configurable (default 24 h).

### ## Autenticación durante particiones

Cada nodo tiene copia local de usuarios en SQLite; puede validar JWT sin conectar al líder Raft (modo AP).

## # 16 Frontend

### # Frontend de DistriSearch

El frontend es una SPA (Single Page Application) construida con **React**, **TypeScript** y **Vite**, estilizada con **Tailwind CSS**.

#### ## Características

- Interfaz responsive para escritorio y móvil.
- Búsqueda en tiempo real con debounce.
- Subida de documentos con drag-and-drop.
- Dashboard de cluster con WebSocket.
- Autenticación JWT integrada.

#### ## Estructura

```
...
frontend/src/
├─ main.tsx # Entry point
├─ App.tsx # Rutas principales
├─ components/ # Componentes reutilizables
├─ pages/ # Vistas (Home, Search, Upload, Dashboard)
├─ services/ # Clientes API
├─ hooks/ # Custom hooks
└─ types/ # Definiciones TypeScript
...
```

#### ## Build

```
```bash
npm install
npm run build # genera dist/
```
```

El bundle se sirve desde Nginx en el contenedor slave.

Detalles en los siguientes archivos.

---

### # React y TypeScript

#### ## Por qué React + TypeScript

- Componentes declarativos y reutilizables.
- Tipado estático reduce errores en tiempo de desarrollo.
- Ecosistema maduro (hooks, context, React Query).

#### ## Configuración Vite

```
`vite.config.ts`:
```ts
```

```

export default defineConfig({
  plugins: [react()],
  server: { proxy: { '/api': 'http://localhost:8000' } }
})
...

Proxy local para desarrollo; en producción Nginx enruta `/api`.

## Tailwind CSS
`tailwind.config.js` define theme, colores y plugins. Clases utilitarias en JSX:
```tsx
<button className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
 Buscar
</button>
```

## Scripts
Comando	Acción
`npm run dev`	Servidor desarrollo con HMR
`npm run build`	Build producción
`npm run preview`	Previsualizar build

---

# Componentes Principales

## Navbar
Barra superior con logo, búsqueda rápida y menú de usuario.

## SearchBar
Input con debounce (300 ms); dispara búsqueda al escribir.

## DocumentCard
Tarjeta que muestra título, fragmento, score y nodo origen.

## UploadDropzone
Zona drag-and-drop para subir archivos; muestra progreso.

## ClusterDashboard
Visualiza nodos, particiones y estado de salud; se actualiza vía WebSocket.

## AuthForms
- `LoginForm`: email + password.
- `RegisterForm`: registro de nuevo usuario.

## Páginas
Ruta	Componente	Descripción
/	Home	Landing con búsqueda
/search	SearchResults	Resultados paginados
/upload	UploadPage	Subir documentos

```

```
| /dashboard | Dashboard | Admin cluster |
| /login | LoginPage | Autenticación |
```

Servicios API

Estructura

`services/api.ts` exporta funciones que llaman al backend:

```ts

```
export const searchDocuments = async (query: string): Promise<SearchResult[]> => {
 const res = await fetch(`/api/v1/search?q=${encodeURIComponent(query)}`, {
 headers: { Authorization: `Bearer ${getToken()}` }
 });
 return res.json();
};
```
```

Servicios principales

```
| Servicio | Funciones |
|-----|-----|
| auth | login, register, logout, refreshToken |
| documents | upload, list, get, delete |
| search | searchDocuments |
| cluster | getNodes, getPartitions, rebalance |
```

Manejo de errores

Interceptor global captura 401 y redirige a login; muestra toast en errores 4xx/5xx.

WebSocket

```ts

```
const ws = new WebSocket('wss://host/ws/dashboard');
ws.onmessage = (e) => updateClusterState(JSON.parse(e.data));
```
```

Recibe eventos de nodos, particiones y alertas en tiempo real.

Caché con React Query

Opcional: usar `useQuery` para cachear resultados de búsqueda y docs.

17 Testing

Testing en DistriSearch

La suite de pruebas cubre desde unidades pequeñas hasta escenarios distribuidos complejos.

Estructura

...

tests/

```
├─ unit/          # Pruebas unitarias (funciones, clases)
├─ integration/   # Pruebas de integración (API, DB)
```

```
└─ distributed/    # Pruebas de cluster (Raft, rebalanceo)
...

## Herramientas
| Herramienta | Uso |
|-----|-----|
| pytest | Runner principal |
| pytest-asyncio | Tests async |
| pytest-cov | Cobertura |
| httpx | Cliente async para API |
| testcontainers | MongoDB/Redis en contenedor |

## Ejecución
```bash
pytest tests/unit -v
pytest tests/integration -v --cov=backend/app
pytest tests/distributed -v -x # falla rápido
```

## Configuración
- `pytest.ini`: markers, opciones por defecto.
- `conftest.py`: fixtures compartidas (app, client, db).

Detalles en los siguientes archivos.

---

# Tests Unitarios

## Objetivo
Probar funciones y clases de forma aislada, sin dependencias externas.

## Ejemplos
```python
def test_vectorizer_tfidf():
 vec = TfidfVectorizer()
 vector = vec.vectorize("documento de prueba")
 assert vector.shape[0] > 0

def test_vp_tree_insert():
 tree = VPtree()
 tree.insert(doc_id="1", vector=[0.1, 0.2])
 assert tree.size() == 1
```

## Mocking
Usar `unittest.mock` o `pytest-mock` para aislar dependencias:
```python
def test_search_service(mock):
 mock.patch('app.storage.mongodb.find', return_value=[...])
 results = search_service.search("query")

```

```

 assert len(results) == 3
...

Ubicación
`tests/unit/` con archivos `test_<modulo>.py`.

Cobertura mínima sugerida
- Vectorizadores: 90%
- Particionamiento: 85%
- Utilidades: 80%

Tests de Integración

Objetivo
Verificar que los componentes interactúan correctamente (API ↔ DB ↔ servicios).

Fixtures
```python
@pytest.fixture
async def client(app):
    async with AsyncClient(app=app, base_url="http://test") as ac:
        yield ac

@pytest.fixture
def mongodb():
    with MongoDBContainer("mongo:6.0") as mongo:
        yield mongo.get_connection_url()
...

## Ejemplos
```python
async def test_upload_and_search(client, mongodb):
 # Subir documento
 resp = await client.post("/api/v1/upload", files={"file": ...})
 assert resp.status_code == 200
 doc_id = resp.json()["id"]

 # Buscar
 resp = await client.get("/api/v1/search?q=contenido")
 assert any(r["id"] == doc_id for r in resp.json())
...

Base de datos de prueba
- Usar testcontainers o MongoDB en memoria.
- Limpiar datos entre tests con fixture `autouse`.

Ubicación
`tests/integration/`

```

---

## # Tests Distribuidos

### ## Objetivo

Validar comportamiento del cluster: Raft, rebalanceo, tolerancia a fallos, Gossip.

### ## Infraestructura

Usar docker-compose.test.yml para levantar mini-cluster:

```
```bash
docker compose -f docker/docker-compose.test.yml up -d
pytest tests/distributed -v
docker compose -f docker/docker-compose.test.yml down
```
```

### ## Ejemplos

```
```python
async def test_raft_leader_election():
    # Parar líder actual
    await stop_container("master-1")
    await asyncio.sleep(5)
    # Verificar nuevo líder
    status = await get_cluster_status()
    assert status["leader"] != "master-1"

async def test_rebalance_after_node_join():
    await add_node("slave-4")
    await trigger_rebalance()
    partitions = await get_partitions()
    assert "slave-4" in [p["node_id"] for p in partitions]
```
```

### ## Fixtures específicas

- ``cluster``: levanta 3 nodos.
- ``faulty_network``: simula particiones con ``tc`` o Toxiproxy.

### ## Ubicación

``tests/distributed/`` y ``backend/tests/test_adaptive_cluster.py``.

## # 18 Control Center

### # Control Center de DistriSearch

El Control Center es una aplicación de administración y monitoreo construida con **Streamlit** (frontend) y **FastAPI** (backend).

### ## Características

- Panel principal con estado del cluster en tiempo real.
- Gestión de contenedores Docker (iniciar, detener, pausar, kill).
- Escenarios de prueba predefinidos (failover, recuperación, particiones).

- Visualización de métricas con Plotly.
- Documentación interactiva de conceptos distribuidos.

## ## Estructura

```

...
control-center/
├── backend/
│ ├── main.py # FastAPI entry point
│ ├── routers/ # Endpoints (nodes, escenarios)
│ └── services/ # Lógica de negocio
├── frontend/
│ ├── 🏠_Panel_Principal.py # Página principal Streamlit
│ └── pages/ # Páginas adicionales
├── docker-compose.yml
└── Dockerfile.*
...

```

## ## Ejecución

```

```bash
# Local
cd control-center/backend && python main.py
cd control-center/frontend && streamlit run 🏠_Panel_Principal.py

```

```

# Docker
docker compose up -d
...

```

- Frontend: <http://localhost:8501>
- Backend: <http://localhost:8888>

Detalles en los siguientes archivos.

Panel de Administración

Tecnología: Streamlit

Streamlit permite crear dashboards interactivos en Python puro:

```

```python
import streamlit as st
st.title("DistriSearch Control Center")
if st.button("Rebalancear"):
 trigger_rebalance()
...

```

### ## Componentes principales

Componente	Función
----- -----	
Sidebar	Navegación entre páginas
Cards de nodos	Estado, CPU, memoria por nodo
Tabla de particiones	Asignaciones VP-Tree
Botón de acciones	Iniciar, detener, pausar contenedores



## ## Escenarios de prueba

1. **Failover de líder**: detiene master y verifica nueva elección.
2. **Recuperación de nodo**: reinicia slave y valida sincronización.
3. **Partición de red**: simula split-brain y observa comportamiento.
4. **Rebalanceo**: agrega nodo y dispara redistribución.

## ## Backend

FastAPI expone endpoints consumidos por Streamlit:

- `GET /nodes`: lista contenedores Docker.
- `POST /nodes/{id}/stop`: detiene contenedor.
- `POST /scenarios/{name}`: ejecuta escenario de prueba.

El backend se comunica con Docker API vía socket.

---

## # Monitoreo del Cluster

### ## Métricas visualizadas

Métrica	Fuente	Visualización
Estado de nodos	Docker API	Iconos verde/rojo
Líder Raft	Endpoint /cluster	Badge destacado
Particiones	Master API	Tabla con node_id
Documentos	MongoDB	Contador por nodo
Latencia	Health checks	Gauge Plotly

### ## Actualización en tiempo real

Streamlit `st.fragment` permite refrescar secciones sin recargar toda la página:

```
```python
```

```
@st.fragment(run_every=5)
```

```
def cluster_status():
```

```
    data = fetch_cluster_status()
```

```
    render_cards(data)
```

```
```
```

### ## Alertas

- Nodo caído: tarjeta roja con tiempo desde último heartbeat.
- Sin líder: banner de advertencia.
- Réplicas insuficientes: indicador en partición afectada.

### ## Gráficos Plotly

- Gauge de salud general.
- Timeline de eventos (failover, rebalanceo).
- Heatmap de carga por nodo.

### ## Integración

El Control Center consulta tanto el backend propio (8888) como los endpoints del cluster DistriSearch (8001, 8000) para obtener datos en vivo.